СРЕДНЕЕ ПРОФЕССИОНАЛЬНОЕ ОБРАЗОВАНИЕ

Л.И. Шеманаева

МИКРОКОНТРОЛЛЕРЫ И ВСТРАИВАЕМЫЕ СИСТЕМЫ

УЧЕБНИК ДЛЯ СПО

Рекомендовано Учебно-методическим отделом СПО в качестве учебника для использования в учебном процессе образовательными учреждениями среднего профессионального образования по укрупненной группе профессий и специальностей «Электроника, радиотехника и системы связи»

Издано при поддержке ИРПО, НАРК, Союза директоров СПО

Саратов
Профобразование
2026

УДК 621.3
ББК 32.84
Ш46

Автор:
Шеманаева Л.И. — канд. техн. наук, доц. кафедры машиностроения
Ковровской государственной технологической академии им. В.А. Дегтярева

Рецензент:
Митрофанов А.А. **—** канд. техн. наук, доц.,
декан факультета автоматики и электроники,
Ковровский государственный технологический университет

Шеманаева, Людмила Ивановна.
Ш46 Микроконтроллеры и встраиваемые системы : учебник для СПО / Л.И. Шеманаева. — Саратов : Профобразование, 2026. — 280 с. — (Среднее профессиональное образование). — Текст : электронный.

ISBN 978-5-4488-2889-8

Учебник посвящен изучению фундаментальных принципов, архитектуры и практического применения микроконтроллеров (МК) — специализированных вычислительных систем на кристалле, являющихся основой современных электронных устройств. Цель изучения: формирование у обучающихся комплексных знаний и практических навыков, необходимых для самостоятельного проектирования, программирования и отладки электронных устройств на основе микроконтроллеров. Учебник имеет выраженную практическую направленность и служит основой для работы в области электроники, киберфизических систем и IoT-разработки.

Подготовлен в соответствии с Федеральным государственным образовательным стандартом среднего профессионального образования.

Учебник предназначен для студентов укрупненной группы профессий и специальностей «Электроника, радиотехника и системы связи», изучающих дисциплину «Микроконтроллеры и встраиваемые системы».

Учебное электронное издание

ISBN 978-5-4488-2889-8

© Шеманаева Л.И., 2026
© ООО «Профобразование», 2026

Учебное издание

Шеманаева Людмила Ивановна

Редактор *М.В. Леман*
Технический редактор, компьютерная верстка *К.С. Бальцюкевич*
Корректор *Т.В. Фетисова*
Обложка *Я.А. Кирсанов, С.С. Сизиумова*, фотобанк Freepik

Подписано к использованию 04.02.2026. Объем данных 8 Мб.

ООО «Профобразование»
8-800-511-14-70 (бесплатный звонок по России)
E-mail: office@profspo.ru, sale@profobr.pro

ОГЛАВЛЕНИЕ

СПИСОК СОКРАЩЕНИЙ 7

ВВЕДЕНИЕ 9

ГЛАВА 1. ОБЩИЕ СВЕДЕНИЯ О МИКРОПРОЦЕССОРНЫХ СИСТЕМАХ 11

1.1. История развития микропроцессоров (МП), современный уровень и тенденции развития микропроцессорных систем (МПС). МП, классификация МП. Структура простейшей МПС, функции МП 11

1.2. Назначение и особенности различных типов МПС. Принстонская и гарвардская архитектуры МПС 17

1.3. Устройства управления с жесткой логикой. Устройства управления с программируемой логикой. Микропрограммное управление 23

1.4. Система команд МП. Рабочий цикл МП 34

1.5. Режимы работы МПС. Программный обмен. Система прерываний МП. Механизм обмена по прерываниям. Обмен в режиме ПДП 42

1.6. Классификация и функции памяти МПС. Классификация ОЗУ, типы и виды ОЗУ. Кэш-память. Классификация ПЗУ, типы и виды ПЗУ. Способы адресации в МПС 60

1.7. Организация связи МПС с внешней средой. Функции устройств ввода-вывода. Принципы построения портов ввода-вывода 78

ГЛАВА 2. ВСТРАИВАЕМЫЕ СИСТЕМЫ НА ОСНОВЕ МИКРОКОНТРОЛЛЕРОВ 93

2.1. Обзор современных микроконтроллеров (МК). Классификация МК. Модульная организация МК 93

2.2. Структура процессорного ядра МК.
Система команд МК. Память МК 109
2.3. Порты ввода-вывода 124
2.4. Таймеры, модуль прерываний МК 128
2.5. Конечные автоматы типа UART, SPI, I2C, USB, CAN
для разгрузки центрального процессора 133
2.6. Минимизация энергопотребления в системах с МК.
Тактовые генераторы МК 144
2.7. Аппаратные средства обеспечения
надежной работы МК 153
2.8. Дополнительные модули МК: последовательного
ввода-вывода, аналогового ввода-вывода 157
2.9. Аппаратные и программные средства
для разработки приложений на базе МК 161
2.10. Функциональные блоки микроконтроллера.
Конфигурирование МК 166

ЛАБОРАТОРНЫЕ РАБОТЫ 177
Лабораторная работа № 1. Структура процессорного ядра
МК. Система команд. Организация памяти 177
Лабораторная работа № 2. Порты ввода-вывода, таймеры,
модуль прерываний МК 181

**ГЛАВА 3. СТРУКТУРА ПРОГРАММЫ
И ОСНОВНЫЕ КОНСТРУКЦИИ ЯЗЫКА СИ 186**
3.1. Вводные понятия языка Си.
Структура программы на Си 186
3.2. Типы данных в Си. Переменные в Си.
Константы в Си 191
3.3. Арифметические и логические операторы языка Си 196
3.4. Операторы ветвления в Си 201
3.5. Циклические конструкции в Си 207
3.6. Указатели и адреса переменных в Си 212

3.7. Работа с функциями в Си. Особенности
передачи данных при обращении к функции в Си 217
3.8. Структуры в Си 222
3.9. Массивы и строки в Си..... 228
3.10. Стандартные функции ввода-вывода в Си..... 233

ЛАБОРАТОРНЫЕ ЗАНЯТИЯ 242
Лабораторное занятие № 1. Выполнение логических
и арифметических команд микропроцессора 242
Лабораторное занятие № 2. Выполнение циклических
конструкций и операторов ветвления на языке Си 245
Лабораторное занятие № 3. Работа с цифровыми портами
ввода-вывода микроконтроллера 251
Лабораторное занятие № 4. Организация циклов
и временных задержек в микроконтроллерах..... 256
Лабораторное занятие № 5. Организация подпрограмм
в языке Си 262
Лабораторное занятие № 6. Работа с макросами
и обработка прерываний в микроконтроллерах 268

ЗАКЛЮЧЕНИЕ 276

СПИСОК ЛИТЕРАТУРЫ 278

СПИСОК СОКРАЩЕНИЙ

ADD — команда сложения

ASIC (Application-Specific Integrated Circuit) — интегральная схема специального назначения

CISC-процессор (англ. Complex Instruction Set Computer — компьютер с полным набором команд) — процессор с архитектурой, характеризующейся большим и разнообразным набором инструкций

CPLD — программируемые логические матрицы с фиксированной архитектурой

CPU — центральный процессор

DAM (Direct Access Memory) — память с прямым доступом

Data Request Ready — флаг готовности регистра

Datasheet — техническая документация

Finite State Machines, FSM — конечный автомат

HDD — жесткий диск

Interrupt-Driven I/O — программный обмен по прерываниям

ISR (Interrupt Service Routine) — обработчик прерываний

Long Mode — 64-битный режим

Polling Mode — режим опроса

Protected Mode — защищенный режим

RAM (Random Access Memory) — память с произвольным доступом

Real Mode — режим реальных адресов

SAM (Sequential Access Memory) — память с последовательным доступом

Virtual Mode, V86 — режим виртуального 8086

АЛУ — арифметико-логическое устройство

ВВ — ввод-вывод

ВЗУ — внешнее запоминающее устройство

ВУ — внешнее устройство

ДОЗУ — динамические оперативные запоминающие устройства

ЗУ — запоминающее устройство

ЗУПВ — запоминающее устройство с произвольной выборкой

ИМС — интегральная микросхема

МП — микропроцессор

МПС — микропроцессорная система

МТ — микропроцессорная техника

НЖМД — накопитель на жестком магнитном диске

ОЗУ — оперативное запоминающее устройство

Оффолдинг — передача специализированной задачи от центрального процессора на выделенный аппаратный блок или сопроцессор

ПДП — прямой доступ к памяти

ПЗУ — постоянное запоминающее устройство

ПК — персональный компьютер

ПКП — программируемый контроллер прерываний

ПЛИС (FPGA) — программируемые логические интегральные схемы

ППЗУ — программируемые постоянные запоминающие устройства

РОН — регистр общего назначения

ЦП — центральный процессор

ВВЕДЕНИЕ

Темпы роста сложности различных технологических объектов, бытовых приборов и повышение требований к качеству управления выполняемыми операциями и их количеству влекут за собой необходимость повышения производительности микроконтроллеров и микропроцессоров, увеличение их многозадачности и множественности режимов управления. Для этого необходимо не только обладать начальными знаниями об их устройстве, но и быть в курсе самых современных разработок.

Микропроцессор (МП) — это электронное цифровое устройство, которое управляется программой. МП предназначен для обработки цифровой информации, полученной с периферийных устройств, и управления обработкой этой информации. Микропроцессор выполняется на одной, а чаще на нескольких интегральных схемах с высокой степенью интеграции электронных элементов.

Первый коммерческий микропроцессор (**Intel 4004**) был представлен на выставке в ноябре 1971 г. компанией Intel. Он содержал 2 300 транзисторов и мог выполнять 60 000 операций в секунду, имел 4-битную архитектуру и предлагался для использования в калькуляторах, но вскоре стал использоваться во многих других устройствах.

В отличие от микропроцессора **микроконтроллер** — это небольшой компьютер на одной интегральной схеме, предназначенный для выполнения строго определенных задач. Он содержит процессор, память и периферийные устройства ввода-вывода.

Первый патент на однокристальную микроЭВМ, которая была предшественником микроконтроллера, получили в 1971 г. инженеры-электронщики Майкл Кокрэн и Гэри Бун, работавшие в американской компании Texas Instruments. Они разместили на одном кристалле процессор и память с устройствами ввода-вывода.

Первый микроконтроллер i8048 выпустила американская фирма Intel в 1976 г. В 1978 г. фирма Motorola представила микроконтроллер MC6801.

Встраиваемые системы (embedded system) — специализированные микропроцессорные системы управления, контроля и мониторинга, предназначенные для работы в устройстве, которым они будут управлять.

В отличие от компьютеров общего назначения встраиваемые системы оптимизированы под конкретные приложения и рассчитаны на работу с ограниченными ресурсами.

ГЛАВА 1. ОБЩИЕ СВЕДЕНИЯ О МИКРОПРОЦЕССОРНЫХ СИСТЕМАХ

1.1. ИСТОРИЯ РАЗВИТИЯ МИКРОПРОЦЕССОРОВ (МП), СОВРЕМЕННЫЙ УРОВЕНЬ И ТЕНДЕНЦИИ РАЗВИТИЯ МИКРОПРОЦЕССОРНЫХ СИСТЕМ (МПС). МП, КЛАССИФИКАЦИЯ МП. СТРУКТУРА ПРОСТЕЙШЕЙ МПС, ФУНКЦИИ МП

За прошедшие полвека можно наблюдать огромный скачок в развитии технологии производства МП, их производительности и размерах.

Краткая хронология развития МП:

- **1971:** Intel 4004 (первый МП, 4-бит);
- **1974:** Intel 8080 (первый успешный 8-бит);
- **1978:** Intel 8086 (начало x86);
- **1985:** Intel 80386 (32-битный x86);
- **1991:** первый ARM-процессор (Acorn Archimedes);
- **1993:** Intel Pentium (суперскалярность);
- **2000-е:** кризис частот;
- **2005:** первые массовые двухъядерники;
- **2007:** первый iPhone (триумф ARM и SoC);
- **2010-е:** эра многоядерности и SoC, начало экспансии ARM на серверы;
- **2020-е:** гибридные архитектуры, доминирование SoC, специализированные ускорители (NPU), война архитектур (x86, ARM, RISC-V). Акцент на эффективность, а не на гигагерцы.

А подробнее это выглядит так.

Предыстория и Рождение (1960-е – 1971).

До появления МП существовали большие и дорогие центральные процессоры (CPU) на основе дискретных транзисторов или простых

ИС. ЭВМ были громоздкими и недоступными. Развитие интегральных схем (ИС) и технологии MOS (металл-оксид-полупроводник).

Кульминация:

• 1971 — Intel 4004. Принято считать первым в мире коммерческим однокристальным микропроцессором. Разработан по заказу японской компании Busicom для калькуляторов;

• характеристики: 4-битный, ~2 300 транзисторов, тактовая частота 740 кГц, изготовлен по 10-мкм техпроцессу. Это был настоящий прорыв — целый центральный процессор на одном кристалле.

Становление и 8-битная эра (1972 – конец 1970-х).

Главная задача — найти применение за пределами калькуляторов. Разработчики осознали универсальность нового устройства. Появились первые операционные системы (CP/M), языки высокого уровня (BASIC, Pascal), сообщество энтузиастов.

Легендарные процессоры:

• Intel 8008 (1972): первый 8-битный МП, но медленный и сложный для подключения;

• Intel 8080 (1974): стал отраслевым стандартом. На его базе появился первый массовый персональный компьютер Altair 8800;

• MOS Technology 6502 (1975): недорогой, мощный, использовался в культовых компьютерах (Apple II, Commodore 64, Nintendo NES);

• Zilog Z80 (1976): усовершенствованный 8080, царил в первых домашних ПК (ZX Spectrum) и промышленности.

16-битная революция и зарождение ПК (конец 1970-х – 1980-е).

Дальнейшее развитие поставило новую задачу: увеличить адресное пространство памяти (>64 КБ) и производительность для сложных бизнес-приложений. Появились IBM PC-совместимых компьютеров как доминирующего стандарта. Начало «войны» архитектур x86 vs. Motorola.

Основные процессоры и стандарты:

• Intel 8086/8088 (1978–1979): первые 16-битные МП от Intel. IBM выбирает 8088 для своего первого персонального компьютера IBM PC (1981). Это решение определило судьбу всей индустрии. Рождается архитектура x86;

• Motorola 68000 (1979): 32-битный внутренне, 16-битная шина. Более элегантная архитектура. Сердце Apple Macintosh, Amiga, Atari ST и ранних рабочих станций.

32-битная эпоха и гонка гигагерц (конец 1980-х – 1990-е).

Развитие поддержки многозадачных ОС (Windows, OS/2, Unix), сложной графики и больших объемов памяти. Появились суперскалярные и суперконвейерные архитектуры, кэш-память 2-го уровня, мультимедийные инструкции (MMX, SSE).

Основные процессоры и тенденции:

• Intel 80386 (1985): первый 32-битный x86-процессор с защищенным режимом и виртуальной памятью. Фундамент для Windows 95/98/NT;

• появление RISC-архитектур: как альтернатива сложным CISC-процессорам (x86). ARM (Acorn), MIPS, SPARC, PowerPC. Они доминировали на рынке рабочих станций, серверов и позже — в мобильных устройствах;

• гонка частот: в конце 1990-х началась интенсивная конкуренция между Intel (Pentium III/4) и AMD (Athlon) за тактовую частоту (гигагерцы).

Многоядерность и кризис частот (2000-е).

Появилась новая проблема, так называемая «Стена частот». После 3–4 ГГц потребляемая мощность и тепловыделение стали недопустимыми (закон Мура работал, но закон Деннарда перестал). Решением стала многоядерность, вместо одного быстрого ядра — несколько более простых, работающих параллельно. Программистам пришлось учиться *параллельному программированию*. Акцент сместился на энергоэффективность (ноутбуки, серверы).

Переход на следующий уровень:

• 2005 — Intel Pentium D, AMD Athlon 64 X2: первые массовые двухъядерные процессоры для ПК;

• 2006 — Intel Core 2 Duo: архитектурный прорыв, доказавший преимущество многоядерности над чистой частотой.

Современная эра: специализация и гетерогенность (2010-е – настоящее время).

Главное направление:

1. Массовая многоядерность: от 2–4 ядер в ПК до десятков и сотен на серверах (Intel Xeon, AMD EPYC).

2. Гибридные архитектуры (big.LITTLE от ARM, Intel Hybrid): сочетание мощных ядер для производительности и энергоэффективных для фоновых задач.

3. Интеграция и SoC (System-on-Chip): процессор перестал быть просто ЦПУ. На один кристалл интегрируются графическое ядро (GPU), контроллер памяти, северный/южный мосты, AI-ускорители (NPU), DSP. Яркий пример — Apple M-series, AMD Ryzen, мобильные Snapdragon.

4. Возрождение и доминирование ARM: благодаря энергоэффективности архитектура ARM захватила рынок смартфонов (99 % доля), а теперь активно проникает в ПК (Apple Silicon) и серверы (AWS Graviton).

5. Эпоха пост-Мура и специализации: тактовая частота почти не растет. Прирост производительности дают специализированные ускорители (для AI, видео, шифрования) и оптимизация архитектуры.

6. Возвращение RISC-V: открытая, свободная и гибкая архитектура RISC-V становится серьезной альтернативой для встраиваемых систем, ИИ-чипов и специализированных процессоров.

То есть логически развитие шло по следующим направлениям.

Повышение степени интеграции — количество транзисторов на одном кристалле.

Повышение тактовой частоты — чем больше тактовая частота, тем больше производительность МП.

Увеличение разрядности — количество символов, которые могут одновременно обрабатываться.

Процесс обработки данных — разделение функций считывания, обработки и вывода на различные устройства МП.

Использование сопроцессоров — таких как математический сопроцессор, выполняющий вычисления, или сопроцессор ввода-вывода данных.

Использование кэш-памяти — увеличение производительности за счет использования небольшой по объему статической памяти на быстродействующих транзисторах (рис. 1.1).

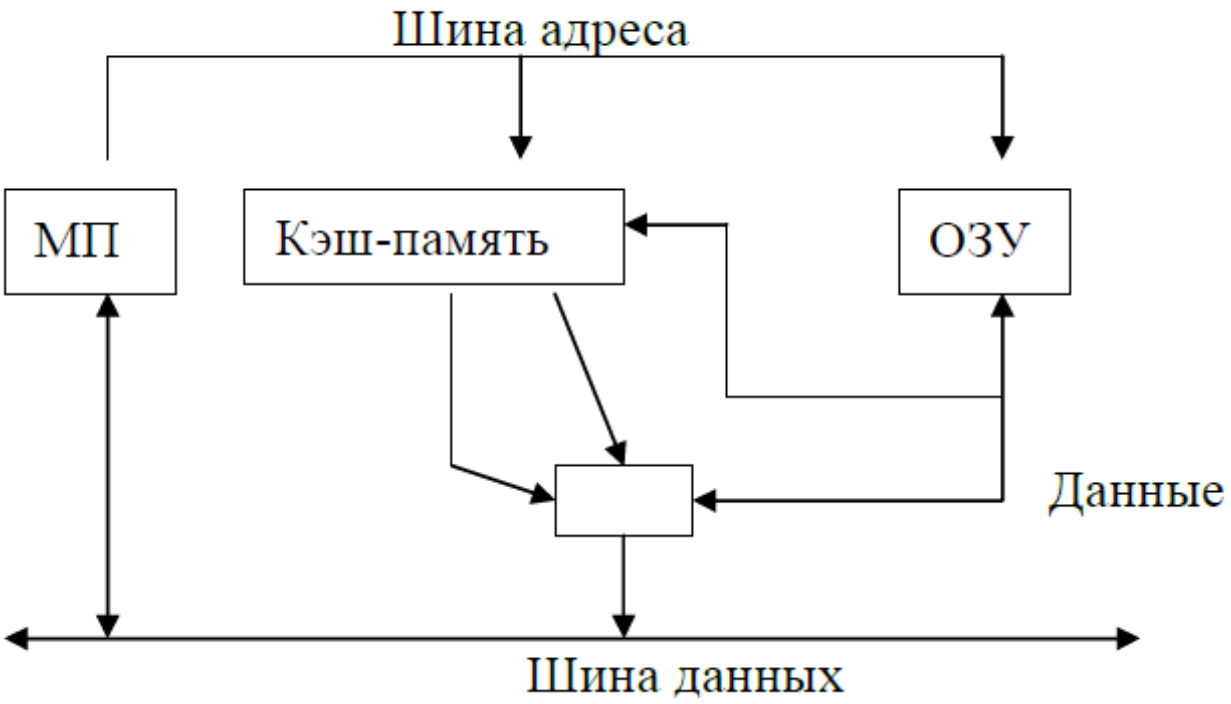


Рис. 1.1. Структура МП с кэш-памятью

Классификация МП обычно проводится по функциональному признаку, т.е. можно выделить МП общего назначения и специализированные (рис. 1.2).

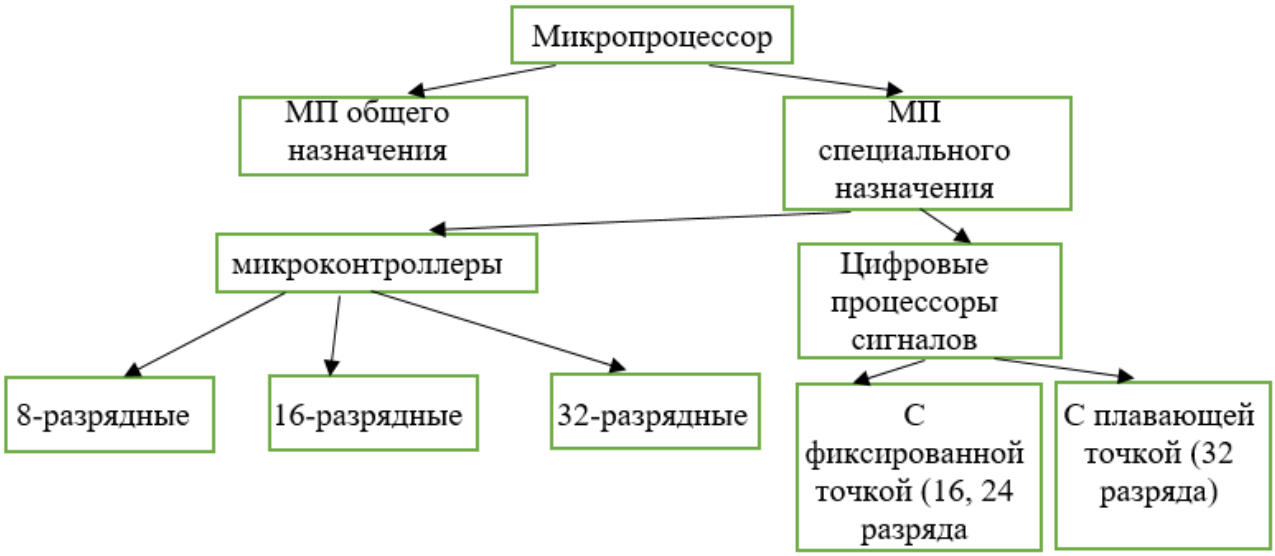


Рис. 1.2. Классификация современных МП
по функциональному признаку

Специализированные процессоры подразделяются на цифровые процессоры сигналов (ЦПС), которые преобразовывают аналоговый сигнал в цифровой, и микроконтроллеры, которые выполняют функции управления различными объектами системы.

МП общего назначения решают широкий круг задач и используются в рабочих станциях, серверах, различных цифровых системах и персональных компьютерах. Могут использоваться и в специализированных системах, где необходима высокая производительность.

Простейшая МПС представлена на рис. 1.3.

По сигналу с блока управления сигнал с шины данных поступает в буферные регистры, способные хранить одно слово информации, один из которых может принимать данные только с шины, другой — либо с шины, либо с аккумулятора. Тип операции задается на АЛУ также с блока управления, результат поступает на аккумулятор. Регистр состояний хранит служебную информацию, состояние аккумулятора, отрицательный результат, деление на ноль; с него сигнал поступает на блок управления о выполнении операции и готовности арифметико-логического устройства (АЛУ) к выполнению следующей.

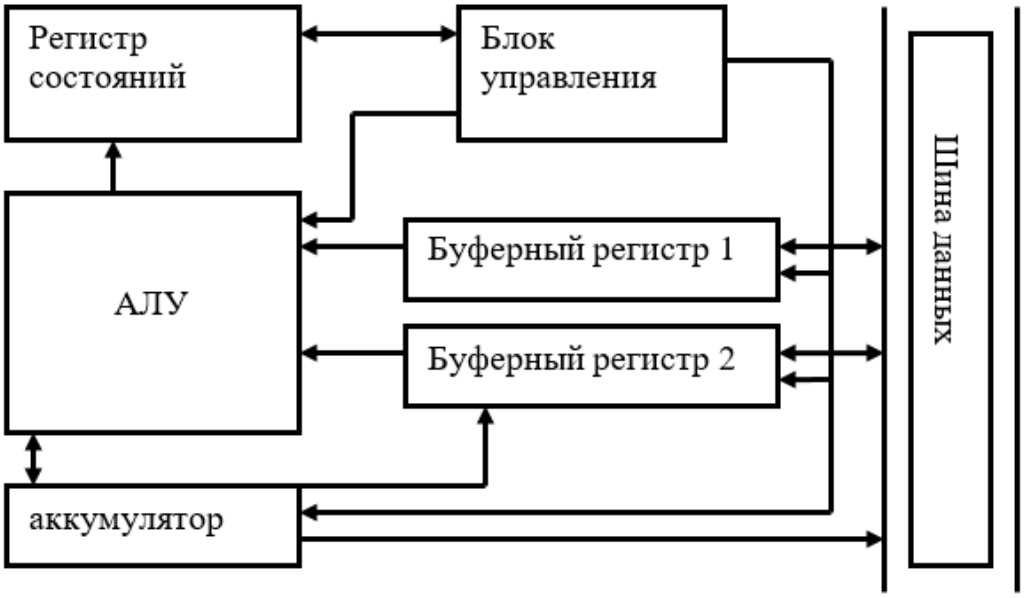


Рис. 1.3. Структура типового процессора

АЛУ — комбинационное логическое устройство; время выполнения операции зависит от частотных свойств элементной базы; основным звеном АЛУ является сумматор.

1.2. НАЗНАЧЕНИЕ И ОСОБЕННОСТИ РАЗЛИЧНЫХ ТИПОВ МПС. ПРИНСТОНСКАЯ И ГАРВАРДСКАЯ АРХИТЕКТУРЫ МПС

Структура — это взаимосвязь входящих в систему элементов, их взаиморасположение и взаимовоздействие.

Структура МП определяет состав основных блоков и устройств и их взаимодействие.

В понятие структуры МП входят:

- внутренняя память — кэш, оперативная и постоянная;
- периферийные устройства — специальные контроллеры, таймеры, аналого-цифровые преобразователи;
- интерфейсный блок — обеспечение выхода на системную шину, обмен данными с внешними устройствами;
- центральный процессор — устройства управления и операционные устройства;

• вспомогательные схемы — схемы для отладки и тестирования, тактовый генератор.

Архитектура МП — это его логическая организация, которая рассматривается с точки зрения пользователя. Архитектура определяет возможности МП в аппаратной, программной и микропрограммной реализации функций, необходимых для построения МПС.

Архитектура МП определяет:

• структуру МП;

• способы передачи данных и их формат;

• способы доступа пользователя ко всем элементам структуры;

• способы приема и обработки внешних данных.

Все МП по типу архитектуры можно разделить:

• на принстонскую (она же фоннеймановская);

• гарвардскую.

Принстонская архитектура — это классическая модель организации МП, предложенная Джоном фон Нейманом в 1945 г. Ее главная особенность — использование единой памяти для хранения и команд, и данных.

Основные принципы:

1. Единая память (однобусная система — все компоненты компьютера подключены к одной общей шине для обмена данными, адресами и управляющими сигналами):

• одинаковое адресное пространство для программ и данных;

• один канал передачи информации между процессором и памятью (узкое место — «фоннеймановская bottleneck»).

2. Последовательное выполнение команд:

• цикл: выборка → декодирование → исполнение;

• нет параллельной обработки команд и данных.

3. Универсальность. Любая ячейка памяти может содержать:

• данные (числа, символы);

• команды (инструкции процессора).

Схема работы представлена на рис. 1.4.

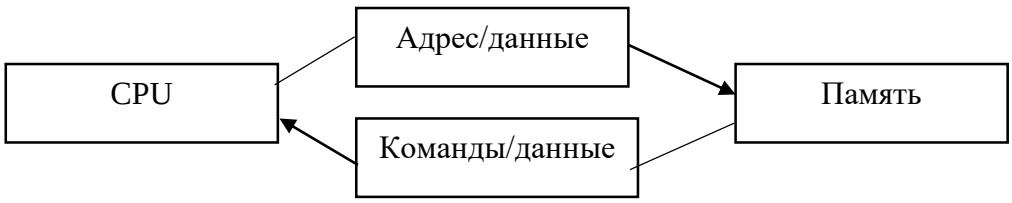


Рис. 1.4. Схема работы фоннеймановской архитектуры

Алгоритм:

1. Процессор запрашивает адрес из памяти.

2. Память возвращает команду или данные.

3. Процессор выполняет операцию и сохраняет результат обратно в память.

Таблица 1.1

Плюсы и минусы фоннеймановской архитектуры

Преимущества	Недостатки
Простота реализации	Низкая производительность (узкое место — одна шина)
Гибкость (память универсальна)	Уязвимость к ошибкам (программа может случайно изменить саму себя)
Дешевизна	Невозможность параллельной обработки

Применяется:

- классические процессоры (Intel x86-64, AMD);
- устаревшие ЭВМ (EDSAC, IBM 704);
- учебные модели (для понимания основ архитектуры).

Современные гибриды (например, ARM) используют модифицированную гарвардскую архитектуру внутри ядра, но сохраняют фоннеймановскую модель для совместимости.

Принстонская архитектура — фундаментальная, но устаревшая концепция. Сегодня ее используют там, где важна простота, а не ско-

рость. Для высокопроизводительных систем (нейросети, DSP) применяют гарвардскую или гибридные модели.

Гарвардская архитектура — это модель организации вычислительной системы, в которой память для команд (инструкций) и память для данных физически разделены. Это устраняет ключевое ограничение фоннеймановской архитектуры — «узкое место» при доступе к памяти.

Схема работы представлена на рис. 1.5.



Рис. 1.5. Схема работы гарвардской архитектуры

Алгоритм:

1. Две независимые шины: для команд и данных.
2. Параллельный доступ: можно одновременно загружать команду и оперировать данными.

Таблица 1.2

Основные компоненты гарвардской архитектуры

Компонент	Назначение
Память программ	Хранение исполняемого кода (только для чтения в классическом варианте)
Память данных	Хранение переменных и результатов вычислений
Процессор	Отдельные блоки для выборки команд и обработки данных

Таблица 1.3

Плюсы и минусы гарвардской архитектуры

Преимущества	Недостатки
Параллелизм: • одновременная загрузка команды и обработка данных; • нет конфликтов доступа к памяти	**Сложность программирования**. Нужно явно распределять память между кодом и данными
Предсказуемость времени выполнения. Критично для систем реального времени (например, в автомобильных ECU)	**Ограниченная гибкость**. Динамическая загрузка кода требует специальных механизмов
Безопасность: • аппаратная защита кода от модификации; • отдельная область памяти для данных	**Увеличенный чип area**. Дублирование интерфейсов памяти
Энергоэффективность. Меньше холостых циклов ожидания	–

Современные модификации: физически разделенные памяти, но единое адресное пространство. Например, ARM Cortex-M (с возможностью обращения к коду как к данным).

Таблица 1.4

Гибридные решения архитектур

Процессор	Реализация
Intel x86	Гарвардская архитектура внутри ядра (кэш L1), фоннеймановская снаружи
Нейронные процессоры	Полностью гарвардская модель (Google TPU)

Области применения.

Микроконтроллеры: STM32, PIC, AVR (Arduino), как надежные и предсказуемые.

Цифровая обработка сигналов (DSP): Texas Instruments C6000, но требует высокую пропускную способность.

Критические системы: авионика (бортовые компьютеры), медицинские приборы.

Ускорители ИИ: Tensor Processing Units (Google), нейроморфные чипы.

Гарвардская архитектура доминирует во встроенных системах и специализированных процессорах, где важны:

- детерминированное выполнение;
- энергоэффективность;
- безопасность кода.

Для задач общего назначения (ПК, серверы) используют **гибридные решения**, сочетающие преимущества обеих архитектур.

Таблица 1.5

Сравнение архитектур

Параметр	**Гарвардская**	**Фоннеймановская**
Организация памяти	Раздельная	Единая
Шины	Минимум 2 (команды + данные)	1 общая
Быстродействие	Выше (параллелизм)	Ниже
Безопасность	Лучше (код нельзя случайно изменить)	Хуже
Сложность	Выше	Ниже
Типичное применение	Микроконтроллеры, DSP	Универсальные CPU

1.3. УСТРОЙСТВА УПРАВЛЕНИЯ С ЖЕСТКОЙ ЛОГИКОЙ. УСТРОЙСТВА УПРАВЛЕНИЯ С ПРОГРАММИРУЕМОЙ ЛОГИКОЙ. МИКРОПРОГРАММНОЕ УПРАВЛЕНИЕ

Устройства с жесткой логикой (Hardwired Control Units) — это цифровые схемы, где алгоритм управления реализован аппаратно (физическими элементами), а не программно. Они обеспечивают максимальное быстродействие, но менее гибки по сравнению с микропрограммными аналогами.

Ключевые особенности.

Принцип работы:

• логика управления формируется физическими элементами:

– логическими вентилями (AND, OR, NOT);

– триггерами;

– дешифраторами;

• нет памяти для хранения команд — все связи постоянны.

Архитектурная схема представлена на рис. 1.6.



Рис. 1.6. Архитектурная схема устройства с жесткой логикой

Области применения:

• процессоры;

• промышленность:

– контроллеры станков ЧПУ;

– системы сигнализации;

– релейная защита в энергетике;

• критические системы:

– авионика (бортовые компьютеры);

– медицинские аппараты ИВЛ;

– АЭС (системы аварийной остановки).

Рассмотрим реализацию на примере.

Управление арифметико-логическим устройством (АЛУ):

```
Verilog // Verilog-код блока управления для 8-битного АЛУ
module ControlUnit(
    input [2:0] opcode,
    output reg alu_src, reg_write, mem_read
);
always @(*) begin
    case(opcode)
        3'b000: {alu_src, reg_write, mem_read} = 3'b101; // ADD
        3'b001: {alu_src, reg_write, mem_read} = 3'b110; // LOAD
        3'b010: {alu_src, reg_write, mem_read} = 3'b000; // NOP
        default: {alu_src, reg_write, mem_read} = 3'bxxx;
    endcase
end
endmodule
```

Релейная схема защиты двигателя

```
plaintext
Термоконтакт --[AND]--> Таймер --[OR]--> Реле отключения
          \--[NOT]--/
```

Таблица 1.6

Преимущества и недостатки устройств с жесткой логикой

Преимущества	Недостатки
Наносекундное время отклика (нет задержек на декодирование)	Сложность модификации (требуется перепроектирование платы)

Преимущества	Недостатки
Детерминированность — предсказуемость на 100 %	Дороговизна при мелкосерийном производстве
Устойчивость к сбоям (нет программного стека)	Ограниченная сложность (не подходит для многоступенчатых алгоритмов)

Современные направления развития.

ПЛИС (FPGA) как компромисс:

- Xilinx Spartan-6: жесткая логика + перепрограммируемые блоки;
- Intel Cyclone V: гибридные ARM+FPGA-системы.

ASIC-решения:

- Google TPU — специализированный тензорный процессор;
- биткоин-майнеры на заказных чипах.

Устройства с жесткой логикой незаменимы там, где нужны:

- максимальная скорость (например, обработка сигналов 5G);
- аппаратная надежность (космические аппараты);
- предсказуемость (медицинская техника).

Устройства управления с программируемой логикой — это электронные компоненты, чья логика работы определяется не физическими соединениями, а загружаемой программой или конфигурацией. Они сочетают гибкость программных решений с быстродействием аппаратных.

Основные типы программируемых устройств.

ПЛИС (FPGA) — матрица программируемых логических блоков и соединений. Имеет следующие преимущества:

- возможность реконфигурации «на лету»;
- параллельную обработку данных.

Недостатки:

- высокое энергопотребление;
- сложность программирования.

Используются в таких устройствах, как Xilinx Virtex, Intel (Altera) Cyclone.

CPLD — программируемые логические матрицы с фиксированной архитектурой. В отличие от FPGA имеют:

- меньшую плотность элементов;
- предсказуемые временные задержки;
- мгновенное включение.

Используются как протокольные преобразователи, простые контроллеры.

Системы на кристалле (SoC FPGA) — совмещают процессорные ядра и программируемую логику.

Используются в таких устройствах, как Xilinx Zynq, Intel Agilex.

Архитектура FPGA представлена на рис. 1.7.



Рис. 1.7. Архитектурная схема устройства с FPGA

Ключевые компоненты архитектуры:

- логические блоки (CLB): базовые элементы (LUT + триггеры);
- межсоединения: программируемая маршрутизация сигналов;
- блоки ввода-вывода (IOB): интерфейс с внешними устройствами;
- тактовые домены: управление синхронизацией.

Алгоритм проектирования:

1. Моделирование (Verilog/VHDL — специализированные языки описания аппаратного обеспечения).

2. Синтез (преобразование в цифровую схему).

3. Размещение и трассировка.
4. Загрузка конфигурации (битстрим).

Пример кода на VHDL:

vhdl
Copy
Download

```
entity counter is
    Port ( clk : in STD_LOGIC;
           count : out STD_LOGIC_VECTOR (3 downto 0));
end counter;

architecture Behavioral of counter is
signal tmp: std_logic_vector(3 downto 0) := "0000";
begin
    process(clk)
    begin
        if rising_edge(clk) then
            tmp <= tmp + 1;
        end if;
    end process;
    count <= tmp;
end Behavioral;
```

Устройства с программируемой логикой используются:

1) для обработки сигналов:
- ЦОС (фильтры, БПФ);
- декодирование видео (H.265);

2) в сетевых технологиях:
- маршрутизаторы 100G+;
- аппаратные фаерволы;

3) для создания искусственного интеллекта:
- ускорители нейросетей;
- тензорные процессоры;

4) в космической технике:

• радиационно-стойкие решения (Xilinx Radiation-Tolerant FPGAs).

Современные направления развития:

• высокоуровневый синтез (C++ → HDL);

• частичная реконфигурация (динамическая смена модулей);

• интеграция с ИИ (например, Xilinx Vitis AI).

Программируемая логика идеальна:

• для прототипирования ASIC;

• систем с меняющимися стандартами;

• высокопроизводительных вычислений.

Для выбора решения необходимо учитывать:

1) требуемую производительность;

2) сроки разработки;

3) бюджет проекта;

4) необходимость последующих модификаций.

Микропрограммное управление.

Микропрограммное управление (Microprogram Control) — это метод реализации управляющего автомата в процессорах и контроллерах, где каждая машинная команда выполняется как последовательность микрокоманд, хранящихся в специальной памяти (ПЗУ микропрограмм).

Базовая архитектура представлена на рис. 1.8.

Ключевые компоненты архитектуры:

• ПЗУ микропрограмм (Control Store) — хранит микрокоманды;

• счетчик микрокоманд (μPC) — указатель текущей микрокоманды;

• дешифратор — преобразует код операции в начальный адрес микропрограммы;

• управляющие сигналы — 50–200 бит на микрокоманду (управление АЛУ, регистрами).

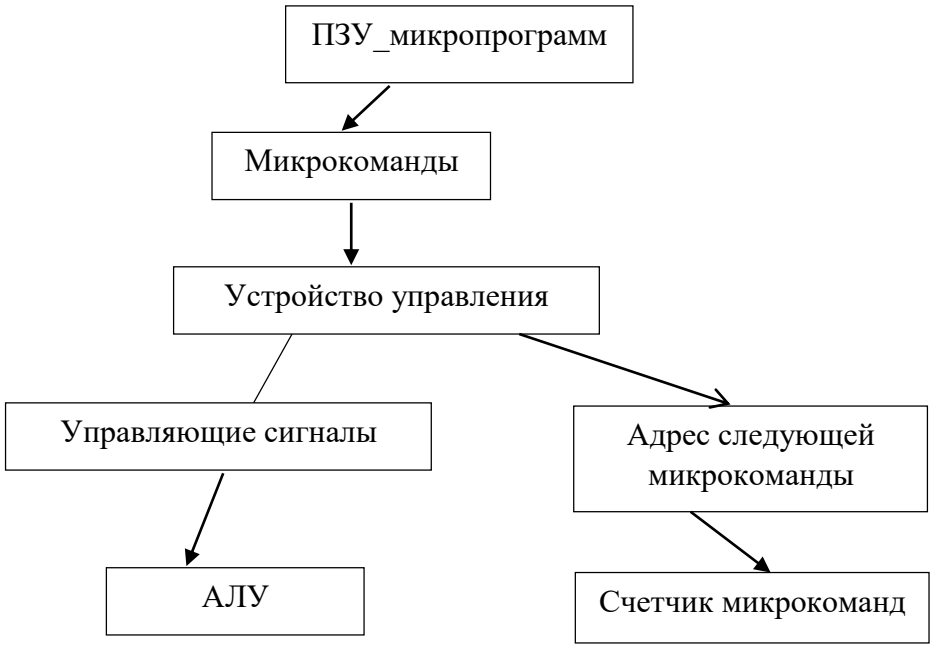


Рис. 1.8. Архитектурная схема устройства
с микропрограммным управлением

Содержание типичной микрокоманды представлено в табл. 1.7.

Таблица 1.7

Формат микрокоманды

Поле	**Размер (бит)**	**Назначение**
Управляющие сигналы	40–120	Непосредственное управление узлами процессора
Адрес перехода	10–20	Условный/безусловный переход
Флаги	4–8	Условия для ветвления

Пример 32-битной микрокоманды:

[ALU_OP:4][REG_SRC:3][REG_DST:3][MEM_EN:1][NEXT_ADDR:12]*[COND:2]*

Алгоритм выполнения:

1. Фаза выборки:

• загрузка машинной команды из RAM;

• дешифрация → начальный адрес в ПЗУ микропрограмм.

2. Фаза выполнения:

while not halt:

микрокоманда = ПЗУ[μPC]

подать_сигналы (микрокоманда.управление)

μPC = вычисление_адреса (микрокоманда.next_addr).

3. Особые случаи:

• прерывания → сохранение μPC в стек;

• конвейеризация → несколько μPC.

Пример микропрограммы для ADD.

Машинная команда: ADD R1, R2 (код операции 0x12) (табл. 1.8).

Таблица 1.8

Микропрограммы

Адрес	Микрокоманда (упрощенно)	Действие
0x120	ALU_A=R1, ALU_B=R2	Загрузка операндов
0x121	ALU_OP=ADD, REG_DST=R1	Сложение, запись
0x122	NEXT=FETCH	Возврат к выборке

Современные реализации:

1. Горизонтальное кодирование:

• каждый бит управляет конкретным узлом;

• пример: IBM System/360 (56-битные микрокоманды).

2. Вертикальное кодирование:

• компактные микрокоманды (8–16 бит);

• требуется дополнительный декодер;
• пример: Intel x86 (начиная с 486).
3. Динамическая микропрограммируемость:
• загрузка микрокода во время работы;
• используется в GPU NVIDIA (Ampere).

Оптимизация микрокода:

1. Вложенные микропрограммы.
Подпрограммы для часто используемых последовательностей.
2. Предикация переходов.
Кэш предсказаний адресов микрокоманд.
3. Компрессия.
LZW-сжатие микрокода (AMD Zen).

Практический пример (VHDL):

```
vhdl
entity control_unit is
port (
    opcode : in  std_logic_vector(7 downto 0);
    clk    : in  std_logic;
    ctrl   : out std_logic_vector(63 downto 0)
);
end entity;

architecture rom_based of control_unit is
type microcode_t is array(0 to 255) of std_logic_vector(63 downto 0);
signal microcode : microcode_t := (
    16#12# => "1100...100", -- ADD
    16#34# => "0010...010", -- LOAD
    others => (others => '0')
);
signal uPC : integer := 0;
begin
    process(clk)
```

```
    begin
        if rising_edge(clk) then
            ctrl <= microcode(uPC);
            uPC <= uPC + 1; -- Упрощенный автомат
        end if;
    end process;
end architecture;
```

Применение в современных процессорах:

1. x86-совместимые:

• микрокодные патчи через Windows Update;

• средний размер: 20–100 КБ на поколение.

2. Графические процессоры:

• управление шейдерными блоками;

• динамическая оптимизация под задачи.

3. Сетевые процессоры. Адаптация под новые протоколы.

Микропрограммное управление имеет следующие достоинства:

• гибкость — исправление ошибок без перепроектирования;

• сложные инструкции — одна команда = много действий;

• масштабируемость — легко добавлять новые функции.

Недостатки:

• задержки на выборку микрокоманд;

• накладные расходы памяти.

В табл. 1.9 для сравнения сведены все параметры управления.

Из всего предыдущего можно сделать следующие выводы.

Жесткая логика — для массового производства с жесткими требованиями к скорости и энергопотреблению (например, мобильные процессоры, IoT-устройства).

Программируемая логика — когда нужны:

• быстрое прототипирование;

• адаптация под меняющиеся стандарты.

Пример: 5G-базовые станции, прототипы ASIC.

Микропрограммное управление — оптимально:

- для сложных инструкций (CISC);
- систем с частыми обновлениями.

Пример: современные x86-процессоры, графические ускорители.

Таблица 1.9

Сравнение типов управления

Параметр	**Жесткая логика**	**Программируемая логика (FPGA/CPLD)**	**Микропрограммное управление**
Принцип реализации	Физические схемы (вентили)	Конфигурируемые логические блоки	Микрокоманды в ПЗУ
Гибкость	✗ Нулевая	✓ Высокая	✓ Средняя (перезапись микрокода)
Быстродействие	✓ 1+ ГГц (макс.)	✓ 100–500 МГц	⚠ 50–300 МГц (зависит от памяти)
Энергопотребление	✓ Минимальное	⚠ Среднее/высокое	⚠ Среднее
Стоимость	✗$$$$ (ASIC), дорого для малых серий	✓$$ (серийное производство)	✓$ (ПЗУ перепрограммируемо)
Сложность разработки	✗ Высокая (требуется фабрикация)	✓ Средняя (HDL-языки)	✓ Средняя (микрокод + декодер)
Типичное применение	RISC-процессоры, DSP	Прототипы, цифровая обработка	CISC-процессоры (x86), GPU

Параметр	Жесткая логика	Программируемая логика (FPGA/CPLD)	Микропрограммное управление
Примеры	ARM Cortex-M0, Intel 8087	Xilinx Virtex, Intel Cyclone	Intel Core (микрокодные патчи), NVIDIA
Время вывода на рынок	✗ 6–18 месяцев	✓ 1–3 месяца	⚠ 3–6 месяцев
Возможность отладки	✗ Только на этапе производства	✓ В реальном времени	✓ Частичная (эмуляция микрокода)
Плотность логики	✓ Максимальная	⚠ Зависит от FPGA	⚠ Средняя (доп. память под микрокод)

Для того чтобы выбрать тип логики для проектируемого устройства, необходимо определиться с целью и задачами, которые будет выполнять это устройство:

1. **Скорость + стабильность** → жесткая логика.
2. **Гибкость + время разработки** → FPGA.
3. **Баланс + обновляемость** → микропрограммное.

Для гибридных решений (например, SoC) часто комбинируют подходы: ядро CPU на микрокоде + FPGA-ускоритель.

1.4. СИСТЕМА КОМАНД МП. РАБОЧИЙ ЦИКЛ МП

Система команд (или Instruction Set Architecture, ISA) — это фундаментальный интерфейс между программным обеспечением и аппаратурой микропроцессора. Это набор всех команд, которые процессор может понять и выполнить.

Ключевые аспекты и определения:

1. ***Система команд*** — это язык, на котором «говорят» программисты (компиляторы, ассемблеры) с процессором. Каждая команда — это указание процессору выполнить простейшую операцию, например сложить два числа, перейти к другой части программы или загрузить данные из памяти.

2. ***Основные философии*** — архитектуры RISC и CISC.

Существует два основных подхода к проектированию систем команд (табл. 1.10).

Таблица 1.10

Подходы к проектированию

Характеристика	**CISC (Complex Instruction Set Computer)**	**RISC (Reduced Instruction Set Computer)**
Идеология	Одна сложная команда делает работу нескольких простых. Много разных команд	Каждая команда простая и выполняется за один такт. Маленький набор команд
Примеры	x86 (Intel, AMD), x86-64	ARM (в смартфонах, Raspberry Pi), MIPS, RISC-V, AVR (в микроконтроллерах Arduino)
Длина команд	Переменная (1–15 байт)	Фиксированная (обычно 4 байта)
Выполнение	Выполняется микрокодом (внутренняя прошивка процессора)	Выполняется непосредственно аппаратно
Регистры	Относительно мало регистров общего назначения	Большое количество регистров общего назначения
Цель	Уменьшить объем кода, упростить компиляторы	Упростить процессор, повысить тактовую частоту и эффективность

Границы между архитектурами сейчас стираются. Современные CISC-процессоры (например, Intel) внутри используют RISC-ядро, преобразуя сложные команды в набор простых микроопераций. А современные RISC-процессоры (например, ARM) добавляют некоторые сложные команды.

3. ***Классификация команд*** (по функциям).

Большинство систем команд можно разделить на несколько основных групп:

1) команды пересылки данных (Data Transfer) — самая часто используемая группа. Перемещают данные между регистрами, памятью и устройствами ввода-вывода:

- LOAD/LDR — загрузить данные из памяти в регистр;
- STORE/STR — сохранить данные из регистра в память;
- MOV — скопировать данные из одного регистра в другой;
- PUSH/POP — работа со стеком (положить/взять значение);

2) арифметические команды (Arithmetic) — выполняют математические операции над данными:

- ADD — сложение;
- SUB — вычитание;
- MUL — умножение;
- DIV — деление;
- INC/DEC — инкремент/декремент (увеличить/уменьшить на 1);

3) логические команды (Logical) — выполняют побитовые операции:

- AND — логическое И;
- OR — логическое ИЛИ;
- XOR — исключающее ИЛИ;
- NOT — инверсия;
- Shift (LSL, LSR) — логические сдвиги битов влево/вправо;
- Rotate (ROR, ROL) — циклические сдвиги битов;

4) команды управления ходом программы (Control Flow) — изменяют порядок выполнения команд (адрес следующей команды):

• JMP/B — безусловный переход;

• CALL/BL — вызов подпрограммы (функции);

• RET — возврат из подпрограммы;

• условные переходы — переход выполняется, только если выполнено условие (проверяются флаги в регистре флагов):

– JE/BEQ — переход, если равно (Zero flag = 1);

– JNE/BNE — переход, если не равно (Zero flag = 0);

– JG/BGT — переход, если больше;

– JL/BLT — переход, если меньше;

– JGE/BGE — переход, если больше или равно;

5) команды управления процессором (Processor Control) — управляют состоянием самого процессора:

• NOP — пустая операция (задержка, выравнивание);

• HLT — остановка процессора;

• прерывания — команды для работы с аппаратными и программными прерываниями (например, **INT**, **IRET**).

4. ***Способы адресации*** (Addressing Modes) — это способы указания операндов (данных) для команды (от них сильно зависит гибкость программирования):

1) непосредственная адресация: операнд (число) указан прямо в коде команды:

ADD R1, R2, #10 // Прибавить к регистру R2 число 10 и результат поместить в R1;

2) регистровая адресация: операнд находится в регистре:

ADD R1, R2, R3 // Сложить содержимое R2 и R3, результат в R1;

3) прямая адресация: в команде указан прямой адрес в памяти, где находится операнд:

LOAD R1, [0x1000] // Загрузить в R1 данные из ячейки памяти с адресом 0x1000:

4) косвенная адресация: в регистре содержится не сам операнд, а адрес, по которому он находится:

LOAD R1, [R2] // Загрузить в R1 данные из памяти, адрес которых хранится в R2;

5) относительная адресация: адрес операнда вычисляется относительно текущего значения счетчика команд (PC) — используется для команд перехода в пределах текущей процедуры;

6) индексная адресация и базированная: адрес = базовый адрес (в регистре) + смещение (в команде или другом регистре):

LOAD R1, [R2 + #4] // Идеально для работы с массивами. R2 — адрес начала массива, #4 — смещение к 4-му элементу.

5. ***Примеры популярных систем команд:***

- x86/x86-64: доминирует на рынке персональных компьютеров и серверов. Яркий пример CISC-архитектуры. Огромное наследство, обратная совместимость;
- ARM: доминирует на рынке мобильных устройств (смартфоны, планшеты), встраиваемых систем и одноплатных компьютеров (Raspberry Pi). Современный пример RISC-архитектуры. Высокая энергоэффективность;
- RISC-V: открытая и свободная RISC-архитектура. Набирает огромную популярность в академической среде, IoT и как альтернатива ARM из-за отсутствия лицензионных отчислений;
- AVR: 8-битная RISC-архитектура от компании Microchip (Atmel). Крайне популярна в мире микроконтроллеров (например, Arduino).

В сухом итоге: система команд — это ДНК процессора. Она определяет:

- как будет написан компилятор;
- как процессор спроектирован на аппаратном уровне;
- насколько он будет быстрым и энергоэффективным;
- с каким программным обеспечением он будет совместим.

Понимание системы команд — ключ к низкоуровневому программированию, оптимизации кода и проектированию hardware.

Рабочий цикл микропроцессора (также известный как цикл выборки-исполнения или Fetch-Decode-Execute cycle) — это фундаментальный процесс, который описывает, как процессор получает, понимает и выполняет каждую команду программы.

Это сердце работы любого процессора — от простейшего микроконтроллера до мощного CPU в сервере.

Основная идея — процессор выполняет программу, которая представляет собой последовательность команд, хранящихся в памяти. Он делает это, бесконечно повторяя три основных шага, таких как:

1) выборка (Fetch) — достать команду из памяти;

2) дешифрация (Decode) — понять, что эта команда означает;

3) исполнение (Execute) — выполнить требуемое действие.

Этот цикл синхронизируется тактовыми импульсами (тактовой частотой). Каждый шаг может занимать один или несколько тактов.

Подробное описание шагов цикла.

Для работы цикла критически важны два специальных регистра:

1) счетчик команд (Program Counter, PC) — содержит адрес в памяти следующей команды для выполнения;

2) регистр команд (Instruction Register, IR) — хранит текущую команду, которую нужно выполнить.

Шаг 1. Выборка (Fetch).

Процессор получает команду из оперативной памяти (ОЗУ):

1. Значение из счетчика команд (PC) (адрес следующей команды) помещается на адресную шину.

2. Процессор посылает сигнал чтения по шине управления.

3. Память, получив адрес и сигнал чтения, находит нужные данные и помещает их на шину данных.

4. Процессор считывает команду с шины данных и загружает ее в регистр команд (IR).

5. Значение счетчика команд (PC) увеличивается, чтобы указывать на следующую команду в памяти. (Размер шага зависит от архитектуры, обычно 1–4 байта.)

Шаг 2. Дешифрация (Decode).

Процессор «понимает», что ему делать:

1. Команда из IR поступает в блок дешифратора команд.

2. Дешифратор анализирует биты команды, чтобы определить:

a) *какую операцию выполнять* (ADD, MOV, JMP и т.д.);

b) *где находятся операнды* (в регистрах, в памяти, прямо в коде команды);

c) *куда сохранить результат*.

3. На основе этого анализа дешифратор активирует конкретные цепи внутри процессора, которые будут выполнять эту конкретную операцию на следующем шаге.

Шаг 3. Исполнение (Execute).

Процессор выполняет decoded-операцию:

1. Активированные на предыдущем шаге схемы процессора выполняют работу.

2. Действие зависит от команды:

• арифметическая/логическая операция: АЛУ (арифметико-логическое устройство) выполняет вычисление над данными из регистров;

• пересылка данных: данные копируются из регистра в регистр или между регистром и памятью;

• операция с памятью: процессор снова работает с шинами (как на шаге выборки), но теперь может как читать, так и писать данные (для команд LOAD/STORE);

• операция перехода: если команда условного перехода (например, JZ — jump if zero), процессор проверяет флаги. Если условие истинно, значение счетчика команд (PC) изменяется на новый адрес из команды. Это прерывает последовательное выполнение программы.

После завершения шага исполнения весь цикл повторяется снова для следующей команды, адрес которой уже находится в PC.

Взаимодействие с подсистемами процессора.

Во время цикла процессор активно взаимодействует со своими внутренними компонентами:

- блоком управления (Control Unit, CU): «дирижер» оркестра. Управляет всеми шагами цикла, посылает управляющие сигналы другим компонентам;
- арифметико-логическим устройством (ALU): «вычислительный цех». Выполняет математические и логические операции по команде от блока управления;
- регистровым файлом: набор быстрых ячеек памяти внутри процессора. Основной источник и приемник данных для операций;
- шинами (Bus): система «автодорог» для передачи данных, адресов и управляющих сигналов:

– шина данных: передает сами данные;

– адресная шина: указывает, *куда* или *откуда* передаются данные;

– шина управления: передает сигналы (чтение, запись, готовность и т.д.).

Пример для команды ADD R1, R2, R3.

Предположим, эта команда означает «сложить содержимое регистров R2 и R3, результат поместить в R1»:

1. Fetch:

- PC указывает на адрес этой команды в памяти;
- команда считывается в регистр IR;
- PC увеличивается.

2. Decode:

- дешифратор видит код операции ADD;
- понимает, что первый операнд — регистр R2, второй — R3, а результат нужно положить в R1;
- готовит АЛУ и пути передачи данных между регистрами.

3. Execute:

• блок управления подает сигналы: значения из R2 и R3 отправляются на входы АЛУ;

• АЛУ выполняет операцию сложения;

• результат с выхода АЛУ записывается в регистр R1;

• цикл завершен. Процессор переходит к следующей команде, адрес которой уже в PC.

Важно о конвейерной обработке (Pipelining).

В современных процессорах этот цикл не является последовательным для разных команд. Пока одна команда находится на стадии исполнения, следующая уже может дешифрироваться, а команда после нее — выбираться из памяти. Это похоже на конвейер на заводе, где на разных стадиях сборки находятся разные изделия. Это кардинально повышает производительность, так как в один момент времени процессор обрабатывает несколько команд одновременно на разных этапах.

1.5. РЕЖИМЫ РАБОТЫ МПС. ПРОГРАММНЫЙ ОБМЕН. СИСТЕМА ПРЕРЫВАНИЙ МП. МЕХАНИЗМ ОБМЕНА ПО ПРЕРЫВАНИЯМ. ОБМЕН В РЕЖИМЕ ПДП

Режимы работы микропроцессорной системы (МПС) (табл. 1.11) — это различные состояния и конфигурации, в которых может функционировать процессор и система в целом. Эти режимы определяют, какие ресурсы и инструкции доступны, как процессор обращается к памяти и как он обрабатывает задачи.

Переключение между режимами позволяет эффективно разделять и защищать ресурсы системы, обеспечивая стабильность и безопасность.

Таблица 1.11

Сводная таблица режимов работы МПС (на примере x86)

Режим	Разрядность	Защита памяти	Виртуальная память	Основное применение
Real Mode	16-бит	**Нет**	Нет	Загрузка, старые ОС (DOS)
Protected Mode	32-бит	**Да** (сегменты + страницы)	Да	Современные 32-битные ОС
Virtual 8086	16-бит (внутри 32-бит)	Да (через ОС)	Да	Запуск DOS-программ в Windows
Long Mode	64-бит	**Да** (страницы)	Да	Современные 64-битные ОС
System Management Mode	Любая	Изолирован	Нет	Управление питанием, hardware

I. Режим реальных адресов (Real Mode).

Это самый простой и **унаследованный режим**, в котором работали первые процессоры x86 (начиная с Intel 8086). Он включает в себя:

1. Адресацию: используется сегментная адресация. Физический адрес вычисляется как Сегмент * 16 + Смещение. Это позволяет 16-битному процессору адресовать до 1 МБ памяти (2^20 = 1 048 576 байт).

2. Память: нет защиты памяти. Любая программа может получить доступ и изменить любой участок памяти, включая память опе-

рационной системы. Это основная причина «падений» старых операционных систем (например, MS-DOS, Windows 3.x) — сбойная программа могла записать что угодно куда угодно.

3. Многозадачность: нет встроенной аппаратной поддержки многозадачности. Переключение между задачами должно было реализовываться программно.

4. Применение: современные процессоры включают этот режим для обеспечения обратной совместимости. Он используется:

a) при старте компьютера (на этапе BIOS/UEFI);

b) для загрузки старых ОС (например, через DOS-эмулятор);

c) в некоторых системах низкоуровнего программирования (например, загрузчики).

II. Защищенный режим (Protected Mode).

Это основной режим работы всех современных операционных систем (Windows, Linux, macOS) на платформе x86/x86-64. Он был введен с процессором Intel 80286 и значительно расширен в 80386; включает в себя:

1. Защиту памяти: ключевая особенность. Процессор использует глобальную таблицу дескрипторов (GDT). Каждой программе (задаче) выделяется свой сегмент памяти, доступный только ей. Попытка выйти за пределы своего сегмента вызывает исключение (прерывание), и ОС завершает программу с ошибкой (например, Segmentation Fault в Linux). Это обеспечивает стабильность системы.

2. Виртуальную память: поддержка механизма подкачки страниц (paging). Каждой программе кажется, что у нее есть собственное непрерывное адресное пространство размером 4 ГБ (для 32-бит), хотя физическая память может быть меньше. Это изолирует программы друг от друга.

3. Многозадачность: аппаратная поддержка. Процессор сам может переключаться между задачами, сохраняя и восстанавливая их контекст (состояние регистров).

4. Привилегированные уровни: реализована система колец защиты (Ring 0 — Ring 3). Это определяет уровень доверия к коду:

a) Ring 0 (режим ядра): максимальные привилегии. Здесь работает ядро ОС и драйверы устройств. Доступны все инструкции и ресурсы;

b) Ring 3 (пользовательский режим): минимальные привилегии. Здесь работают прикладные программы (браузер, текстовый редактор). Попытка выполнить привилегированную инструкцию или обратиться к чужой памяти вызовет исключение.

III. Режим виртуального 8086 (Virtual 8086 Mode, V86).

Это подрежим защищенного режима, который позволяет запускать несколько программ, написанных для Real Mode, в изолированных «песочницах» внутри современной многозадачной ОС.

Суть: ОС создает для каждой такой DOS-программы виртуальную машину, которая эмулирует среду Real Mode (адресация 1 МБ, нет защиты), но при этом изолирована от основной ОС и других программ благодаря механизмам Protected Mode.

Применение: именно этот режим использовался в Windows XP и более ранних версиях для работы с устаревшими DOS-приложениями.

IV. Режим управления системой (System Management Mode, SMM).

Специальный «скрытый» режим для выполнения низкоуровневых функций управления аппаратным обеспечением:

1. Независимость: работает в отдельном адресном пространстве, изолированном от основной ОС.

2. Активация: вызывается специальным аппаратным прерыванием (SMI — System Management Interrupt), которое обычно генерируется чипсетом материнской платы.

3. Применение:

a) управление энергопотреблением (спящий режим, гибернация, отключение неиспользуемых компонентов);

b) контроль температуры и вращения вентиляторов;

c) реализация функций hardware security;

d) эмуляция устаревшего оборудования (например, PS/2-порта для клавиатуры на современной материнской плате).

4. Код, выполняемый в этом режиме, называется SMI-обработчик и прошивается в firmware (BIOS/UEFI).

V. Long Mode (64-битный режим).

Эволюционное развитие Protected Mode для 64-битных архитектур (x86-64, AMD64):

1. Увеличенное адресное пространство: теоретически позволяет адресовать до 2^{64} байт памяти (16 эксабайт). Практически сейчас ограничивается 48 битами.

2. Новые регистры: добавлены новые 64-битные регистры (R8–R15) и расширены старые (EAX -> RAX).

3. Новые возможности: введены новые инструкции и более эффективные механизмы адресации.

4. Совместимость: в рамках Long Mode есть два подрежима:

a) 64-битный режим: для выполнения нативного 64-битного кода;

b) режим совместимости (Compatibility Mode): позволяет выполнять большинство старых 16-битных и 32-битных приложений без перезагрузки в Real Mode. Это более безопасная и современная замена режиму Virtual 8086.

Переключение между этими режимами — основа загрузки современного компьютера:

1. Процессор стартует в Real Mode.

2. Загрузчик и firmware подготавливают систему и переводят процессор в Protected Mode (или сразу в Long Mode).

3. ОС загружает ядро в режим ядра (Ring 0).

4. Запускает прикладные приложения в пользовательском режиме (Ring 3).

5. По сигналу от аппаратуры (например, закрытие крышки ноутбука) процессор может временно перейти в **SMM**, чтобы выполнить необходимые действия и вернуться обратно.

Программный обмен — это один из базовых способов организации ввода-вывода данных в МПС, при котором ЦП активно управляет всем процессом передачи данных, выполняя специальную программу.

Суть программного обмена.

При программном обмене процессор сам, выполняя команды программы, опрашивает состояние внешнего устройства, проверяя, готово ли оно к обмену. Если устройство готово, процессор выполняет команду чтения или записи для передачи данных. Весь процесс ложится на ЦП, что отвлекает его от решения других задач.

Основные схемы реализации.

Существует две основные схемы программного обмена:

I. Программный обмен в режиме опроса (Polling Mode) или готовности (readiness).

Это самый простой метод. Процессор циклически проверяет флаг готовности периферийного устройства.

Алгоритм работы:

1. Устройство ввода-вывода (ВВ) имеет регистр состояния, в котором есть специальный бит (флаг) готовности (например, DRQ — Data Request Ready для готовности к передаче или BSY — BusY для занятости).

2. Программа, которая хочет передать или принять данные, входит в цикл опроса.

3. Цикл опроса:

a) ЦП читает регистр состояния устройства;

b) проверяет значение бита готовности;

c) если устройство не готово → ЦП возвращается к началу цикла и продолжает опрашивать состояние;

d) если устройство готово → ЦП выходит из цикла и выполняет команду чтения данных из регистра данных устройства или записи в него.

Алгоритм для вывода данных (CPU → Устройство):

MOV CX, 1000 ; Количество данных для передачи

LEA SI, DataBuffer ; Указатель на данные в памяти

Poll_Output:

IN AL, StatusPort ; Прочитать регистр состояния устройства (например, порт 0x37)

TEST AL, 80h ; Проверить бит 7 (флаг «Буфер пуст, можно передавать»)

JZ Poll_Output ; Если флаг не установлен, продолжать опрос

MOV AL, [SI] ; Если готово, взять байт данных из буфера

OUT DataPort, AL ; и записать его в регистр данных устройства (например, порт 0x36)

INC SI ; Перейти к следующему байту

LOOP Poll_Output ; Повторить для всех данных

Достоинство этого метода — простота реализации. Не требуется сложная аппаратная поддержка.

Недостатки:

- низкая эффективность. Процессор тратит много времени на бесполезный опрос в цикле ожидания вместо выполнения полезной работы. Это называется активное ожидание;
- низкая скорость обмена. Скорость определяется не максимальными возможностями устройства, а скоростью опроса.

Используется в простых системах, где нет высоких требований к производительности, или для работы с очень медленными устройствами.

II. Программный обмен по прерываниям (Interrupt-Driven I/O) (табл. 1.12).

Это более advanced (передовой) метод, который позволяет процессору не тратить время на опрос, а заниматься другими задачами, пока устройство не будет готово.

Таблица 1.12

Сравнение с другими методами

Параметр	**Программный обмен (опрос)**	**Программный обмен (по прерываниям)**	**Прямой доступ к памяти (DMA)**
Задействование ЦП	Максимальное (постоянно занят)	Среднее (занят только во время передачи)	Минимальное (инициирует обмен)
Скорость	Низкая	Средняя	Высокая
Сложность	Низкая	Средняя	Высокая (аппаратная)
Применение	Простые и медленные устройства	Устройства средней скорости	Высокоскоростные устройства (HDD, SSD, сетевая карта)

Алгоритм работы:

1. Процессор запускает операцию обмена (например, дает команду устройству начать чтение).

2. Вместо опроса процессор переходит к выполнению другой полезной работы.

3. Устройство, выполнив свою работу и будучи готовым к обмену (например, получив порцию данных), формирует сигнал запроса прерывания (IRQ).

4. Получив этот сигнал, процессор:

a) приостанавливает выполнение текущей программы;

b) сохраняет ее контекст (значения регистров);

c) переходит к выполнению специальной подпрограммы — обработчика прерывания (ISR — Interrupt Service Routine).

5. Внутри обработчика прерывания происходит непосредственная передача данных (чтение или запись через команды IN/OUT).

6. По завершении обработчика прерывания процессор восстанавливает контекст и возвращается к выполнению прерванной программы.

Алгоритм для ввода данных (Устройство → CPU):

• основная программа:

... ; ЦП выполняет любую полезную работу

; В любой момент его может прервать устройство

• обработчик прерывания (ISR):

PUSH AX ; Сохранить регистры, которые будем использовать

PUSH SI

IN AL, DataPort ; Прочитать пришедший из порта устройства байт данных

MOV [SI], AL ; Сохранить его в буфер, в оперативной памяти

INC SI ; Увеличить указатель буфера

... ; Возможно, послать устройству команду на подготовку след. байта

POP SI ; Восстановить регистры

POP AX

IRET ; Вернуться из прерывания

Достоинства:

• высокая эффективность. ЦП не простаивает в цикле ожидания, а занимается полезной работой;

• своевременная реакция. Устройство само «оповещает» процессор о своей готовности, что сокращает задержки.

Недостатки:

• более сложная программная реализация (написание корректных обработчиков прерываний;

• требуется аппаратная поддержка (контроллер прерываний);

• на обработку самого прерывания (сохранение/восстановление контекста) также тратится время.

Является самым распространенным методом для работы с большинством периферийных устройств (сетевые карты, диски, клавиатуры, мыши) в современных системах.

Программный обмен — это фундаментальный метод управления вводом-выводом, при котором процессор выполняет программу для передачи каждого байта/слова данных. *Он делится на две ключевые разновидности:*

1. Опрос (Polling) — простой, но расточительный по времени ЦП.

2. По прерываниям — более эффективный, позволяющий ЦП работать параллельно.

Для оффлодинга процессора от рутинных задач по пересылке больших объемов данных используется более advanced метод — прямой доступ к памяти (DMA).

Система прерываний МП — это механизм, позволяющий процессору реагировать на асинхронные события, временно приостанавливая выполнение текущей программы для обработки специальной процедуры, вызванной этим событием, с последующим возвратом к прерванной задаче.

Проще говоря, это как система «срочных вызовов» для процессора. Когда происходит важное событие (например, пользователь нажал клавишу), устройство «звонит» процессору. Тот бросает текущую работу, обрабатывает вызов и возвращается к тому, что делал.

Прерывания необходимы:

• для эффективной работы с устройствами ввода-вывода (I/O): устройства (клавиатура, мышь, сетевой контроллер) работают намного медленнее процессора, поэтому без прерываний ЦП должен был бы постоянно их опрашивать (режим «polling»), тратя время впустую. С прерываниями устройство само уведомляет ЦП о своей готовности;

• реакции на асинхронные события: обработка нажатий клавиш, движения мыши, прихода сетевого пакета;

• обработки исключительных ситуаций: реакция на ошибки (деление на ноль, отсутствие страницы в памяти, неверная команда);

• вызова функций операционной системы: программы запрашивают сервисы ОС (например, доступ к файлу) через программные прерывания.

Классификация прерываний:

А. По источнику возникновения:

• аппаратные (внешние) прерывания:

– источник: внешние устройства (клавиатура, таймер, дисковый контроллер);

– сигнал: запрос на прерывание (IRQ — Interrupt ReQuest) поступает на специальные выводы процессора;

– особенность: асинхронны, т.е. могут произойти в любой момент выполнения программы;

– цель: обслуживание периферийных устройств;

• программные (внутренние) прерывания:

– источник: выполнение специальной команды в программе (например, INT 21h в x86);

– особенность: синхронны, так как инициируются самим кодом программы;

– цель: вызов сервисов операционной системы (API), обработчиков ошибок;

• исключения (Exceptions):

– источник: сам процессор при обнаружении аномальной ситуации при выполнении команды;

– примеры: деление на ноль, отсутствие страницы памяти, недействительный код операции;

– особенность: синхронны (возникают при попытке выполнить конкретную команду).

Б. По возможности запрета:

• маскируемые: можно временно игнорировать (запрещать) с помощью установки флага IF (Interrupt Flag) в регистре флагов про-

цессора. Команды CLI (сброс IF — запрет) и STI (установка IF — разрешение). Большинство аппаратных прерываний — маскируемые;

• немаскируемые (NMI — Non-Maskable Interrupt): обрабатываются всегда, независимо от состояния флага IF. Используются для сообщения о критических аппаратных ошибках (например, ошибка четности памяти, аварийное отключение питания).

Принцип работы и аппаратная поддержка.

Для работы системы прерываний требуется как аппаратная поддержка, так и программное обеспечение.

Аппаратные компоненты:

• контроллер прерываний (PIC — Programmable Interrupt Controller или его современный аналог APIC):

– задача: принимать множество запросов IRQ от разных устройств, определять их приоритет и отправлять на процессор один главный запрос;

– важность: позволяет процессору иметь всего один-два входа для запросов прерываний вместо десятков. В современных системах это Advanced PIC (APIC), интегрированный прямо в чипсет и процессор;

– регистр флагов процессора: содержит бит IF (Interrupt Enable Flag), который разрешает или запрещает обработку маскируемых прерываний;

• векторная таблица прерываний (Interrupt Vector Table — IVT):

– это массив адресов, находящийся в оперативной памяти по фиксированному адресу (в реальном режиме x86 это адрес 0x0000:0x0000);

– каждому типу прерывания присвоен свой номер (вектор). Например, вектор 0 — деление на ноль, вектор 1 — отладка, вектор 9 — клавиатура;

– по этому номеру процессор находит в таблице адрес программы-обработчика для данного прерывания.

Процесс обработки прерывания:

1. Запрос: устройство посылает сигнал IRQ на контроллер прерываний.

2. Разрешение: если прерывания разрешены (IF = 1), контроллер определяет приоритет и посылает сигнал процессору.

3. Завершение команды: процессор завершает выполнение текущей команды.

4. Сохранение контекста:

• процессор автоматически сохраняет в стеке адрес возврата (значения регистров CS:IP или RIP) и регистр флагов (FLAGS/RFLAGS);

• сбрасывает флаг IF (запрещает новые маскируемые прерывания, если не используется механизм вложенных прерываний).

5. Определение вектора: процессор получает от контроллера номер вектора прерывания.

6. Поиск обработчика: процессор использует номер вектора как индекс в таблице векторов прерываний (IVT или IDT) и загружает адрес обработчика.

7. Переход к обработчику: процессор начинает выполнение кода обработчика прерывания (ISR — Interrupt Service Routine).

8. Работа обработчика (ISR):

• программа, написанная разработчиком ОС или драйвера;

• сохраняет в стеке значения всех регистров, которые будет использовать (AX, BX и т.д.);

• выполняет необходимые действия (читает scan code из клавиатуры, посылает данные в сеть);

• в конце посылает специальную команду контроллеру прерываний (End of Interrupt — EOI), сообщая, что обработка завершена;

• восстанавливает из стека сохраненные регистры.

9. Возврат: выполняется команда IRET (Interrupt Return). Процессор автоматически восстанавливает из стека регистр флагов и адрес возврата и продолжает выполнение прерванной программы с того же места.

Что делать, если прерывания приходят одновременно? Для этого существует система приоритетов:

1. Аппаратно — приоритеты назначаются контроллером прерываний (например, в классическом PIC IRQ0 имел высший приоритет, а IRQ7 — низший).

2. Обработка прерывания с высшим приоритетом может прервать обработку прерывания с низшим приоритетом (вложенные прерывания).

3. Исключения и NMI обычно имеют наивысший приоритет.

Схема цикла работы МП с учетом прерываний.

Типичный цикл выполнения команды процессора выглядит так:

выборка (Fetch) → дешифрация (Decode) → исполнение (Execute).

С учетом прерываний цикл модифицируется:

1. После этапа Исполнения (и перед следующей Выборкой) процессор проверяет наличие запросов на прерывание.

2. Если есть запрос и прерывания разрешены, процессор не начинает новую команду, а инициирует процедуру обработки прерывания.

3. После возврата из прерывания (IRET) процессор продолжает стандартный цикл с Выборки команды прерванной программы.

Система прерываний — это мощный механизм, который:

- повышает эффективность работы системы, избавляя ЦП от пустого опроса устройств;
- обеспечивает многозадачность и быструю реакцию на внешние события;
- позволяет обрабатывать ошибки и исключительные ситуации;
- обеспечивает интерфейс между приложениями и операционной системой.

Без этого механизма современные операционные системы и сложные многозадачные приложения были бы невозможны.

Обмен в режиме ПДП (прямого доступа к памяти) — это высокоскоростной метод передачи данных между устройствами ввода-

вывода (ВВ) и оперативной памятью (ОЗУ) без участия центрального процессора (ЦП) на время выполнения самой пересылки.

ЦП лишь инициирует и завершает процесс, но не тратит свои такты на пересылку каждого байта. Это классический пример оффлодинга — разгрузки процессора.

Суть и решаемая проблема.

Проблема: при программном обмене (через прерывания или тем более опрос) ЦП активно участвует в каждой операции: читает данные из порта устройства и записывает в память (или наоборот). Это отвлекает его от выполнения полезной работы и снижает общую производительность системы, особенно при передаче больших объемов данных (например, чтение файла с диска, передача пакета по сети).

Решение: передать функцию управления шиной и пересылки данных специализированному аппаратному модулю — контроллеру ПДП (DMA-контроллеру).

Аппаратная основа: контроллер ПДП (DMA-контроллер).

DMA-контроллер — это отдельная микросхема (или блок внутри чипсета), которая может временно «становиться хозяином» системной шины. Он умеет:

- формировать адреса ячеек памяти;
- формировать управляющие сигналы для чтения/записи;
- передавать данные по шине;
- вести подсчет переданных байтов.

Типичные регистры внутри DMA-контроллера:

- регистр адреса: содержит начальный адрес в памяти, откуда или куда нужно передавать данные;
- регистр счетчика: содержит количество байтов/слов для передачи;
- регистр управления: задает направление передачи (устройство → память или память → устройство), режим работы и другие параметры.

Принцип работы (последовательность действий).

Рассмотрим классический пример пересылки данных от периферийного устройства в оперативную память (например, чтение с диска):

1. Инициализация (настройка ЦП):

• ЦП загружает в контроллер ПДП параметры будущей операции:

– адрес в памяти, куда нужно поместить данные;

– количество байтов для передачи;

– направление передачи (чтение из устройства, запись в память);

– идентификатор канала и устройства, с которым будет идти обмен;

• после настройки ЦП разрешает работу DMA-контроллера и переходит к выполнению других задач.

2. Запрос на передачу (от устройства). Периферийное устройство (например, жесткий диск) подготавливает порцию данных для передачи и выставляет сигнал DRQ (DMA Request) на соответствующем канале DMA-контроллера.

3. Запрос на захват шины (от DMA). Контроллер ПДП, получив DRQ, посылает процессору сигнал HRQ (Hold Request), запрашивая разрешение на временный захват системной шины.

4. Ответ процессора. ЦП заканчивает выполнение текущей машинной команды, освобождает системные шины (адресную, данных и управления) и посылает в ответ сигнал HLDA (Hold Acknowledge), сообщая: «Шина в вашем распоряжении».

5. Передача данных (управляет DMA):

• DMA-контроллер становится хозяином шины;

• он выполняет всю работу по пересылке:

– выставляет на адресную шину адрес ячейки ОЗУ;

– выставляет на шину управления сигналы чтения для устройства и записи для памяти;

– получает данные от устройства по шине данных и записывает их в память;

– увеличивает значение регистра адреса и уменьшает значение регистра счетчика;

• этот цикл повторяется для каждого байта/слова.

6. Завершение передачи. Когда регистр счетчика достигает нуля (все данные переданы), контроллер ПДП снимает сигнал HRQ.

7. Прерывание от DMA (опционально). После завершения блока передачи DMA-контроллер может послать ЦП сигнал прерывания, чтобы уведомить его о завершении операции.

8. Восстановление работы ЦП. Процессор, видя снятый сигнал HRQ, снимает свой сигнал HLDA и снова «становится хозяином» шины, продолжая работу с того места, где его прервали.

Режимы работы ПДП.

Контроллер ПДП может работать в нескольких режимах, определяющих, как часто он будет отвлекать ЦП.

1. Одиночная передача (Single Transfer Mode):

• за один захват шины передается всего один байт/слово;

• после каждой передачи DMA возвращает шину ЦП. Это минимально мешает процессору, но скорость передачи не максимальна.

2. Пакетный режим (Burst Mode):

• DMA запрашивает шину и передает весь блок данных (пока счетчик не обнулится) за один сеанс;

• это самый быстрый режим, но ЦП «замораживается» на все время передачи большого блока данных.

3. Пошаговый режим (Demand Mode). Передача идет блоками до тех пор, пока устройство не снимает сигнал DRQ (например, его буфер опустел или переполнился). Как только устройство снова готово, передача возобновляется.

4. Совместный режим (Transparent Mode). DMA-контроллер работает только в те моменты, когда ЦП не использует шину (например, во время внутренних операций процессора, не требующих обращения к шине). Это самый «вежливый», но и самый медленный режим.

Режим ПДП (табл. 1.13) — это краеугольный камень высокопроизводительных computing systems. Он абсолютно необходим для эффективной работы современных компьютеров, серверов и любых других устройств, где требуется быстрая пересылка больших массивов данных без простоя главного процессора.

Таблица 1.13

Преимущества и недостатки прямого доступа к памяти

Преимущества	Недостатки
Кардинально снижает нагрузку на ЦП. Процессор свободен для вычислений, пока данные пересылаются	**Аппаратная сложность.** Требуется дополнительный контроллер на материнской плате (в современных системах встроен в чипсет)
Высокая скорость передачи. Специализированный контроллер optimized for data transfer и может работать с максимальной скоростью шины	**Проблемы с когерентностью кэша.** Данные, переписанные в память через DMA, могут не знать about обновления в кэше процессора. Это требует механизмов синхронизации
Позволяет эффективно работать с высокоскоростными устройствами (сетевые карты, SSD, дисковые массивы)	**Конфликт за шину.** В режиме burst ЦП может надолго лишаться доступа к памяти, что может приводить к задержкам

Практически все высокоскоростные периферийные устройства (жесткие диски SATA/SSD, сетевые карты Gigabit/Ethernet, звуковые карты) используют DMA для обмена с оперативной памятью.

1.6. КЛАССИФИКАЦИЯ И ФУНКЦИИ ПАМЯТИ МПС. КЛАССИФИКАЦИЯ ОЗУ, ТИПЫ И ВИДЫ ОЗУ. КЭШ-ПАМЯТЬ. КЛАССИФИКАЦИЯ ПЗУ, ТИПЫ И ВИДЫ ПЗУ. СПОСОБЫ АДРЕСАЦИИ В МПС

Классификация и функции памяти МПС.

Память является иерархической структурой, от которой напрямую зависят быстродействие и эффективность всей системы.

Функции памяти в МПС.

Память в МПС выполняет три критически важные функции:

1. Хранение данных и программ: в памяти находятся как инструкции выполняемой программы, так и данные, которые она обрабатывает.

2. Буферизация данных: обеспечивает временное хранение информации при обмене между устройствами с разной скоростью работы (например, между CPU и HDD).

3. Обеспечение работы стеков: специальные области памяти, используемые для организации вызовов подпрограмм, передачи параметров и хранения контекста прерываний.

Классификация памяти МПС.

Память можно классифицировать по множеству признаков. Вот основные из них:

А. По назначению и функциональности (табл. 1.14).

Б. По способу доступа:

- память с произвольным доступом (RAM — Random Access Memory): время доступа к ячейке не зависит от ее физического расположения. Это ОЗУ и кэш;
- память с последовательным доступом (SAM — Sequential Access Memory): для доступа к нужной ячейке необходимо последовательно прочитать все предыдущие. Например, магнитные ленты;
- память с прямым доступом (DAM — Direct Access Memory): комбинация двух предыдущих. Сначала находится дорожка (быстро,

как произвольный доступ), а затем данные на дорожке ищутся последовательно (как последовательный доступ). Например, жесткие диски (HDD).

В. По устойчивости данных (энергозависимость):

• энергозависимая (Volatile): данные теряются при отключении питания;

– ОЗУ (оперативное запоминающее устройство): SRAM, DRAM;

– кэш-память: обычно на основе SRAM;

– регистровая память;

• энергонезависимая (Non-Volatile): данные сохраняются после отключения питания:

– ПЗУ (постоянное запоминающее устройство): ROM, PROM, EPROM, EEPROM, Flash-память (SSD, USB-флешки);

– магнитные диски (HDD), оптические диски (CD/DVD/Blu-Ray).

Г. По возможности записи:

• память только для чтения (ROM — Read-Only Memory):

– Mask ROM (масочные ПЗУ): записываются once при изготовлении;

– PROM (Programmable ROM): пользователь может записать данные один раз;

– EPROM (Erasable PROM): стирается ультрафиолетом;

– EEPROM (Electrically Erasable PROM): стирается электрически (например, BIOS);

– Flash-память: разновидность EEPROM, стирается блоками (SSD, карты памяти);

• память для чтения и записи (Read-Write Memory) — ОЗУ (RAM).

Д. По физическому принципу действия:

• полупроводниковая: вся память, находящаяся на кристалле процессора или в микросхемах (кэш, ОЗУ, ПЗУ, регистры);

• магнитная: жесткие диски (HDD), магнитные ленты;

- оптическая: CD-, DVD-, Blu-Ray-диски;
- магнитооптическая: устаревший гибридный формат.

Таблица 1.14

Классификация памяти МПС по назначению и функциональности

Тип памяти	Назначение	Характеристики	Аналог из жизни
Регистровая память	Внутренняя память процессора. Хранит операнды, адреса, служебную информацию	Очень высокая скорость, небольшой объем (несколько десятков-сотен байт), энергозависимая	Блокнот на столе
Кэш-память	Буфер для ускорения доступа к оперативной памяти. Хранит копии часто используемых данных и команд из ОЗУ	Очень высокая скорость, небольшой объем (КБ — МБ), энергозависимая, делится на уровни (L1, L2, L3)	Полка с документами
Основная память (ОЗУ)	Хранение выполняемых программ и их данных. Непосредственно взаимодействует с процессором	Высокая скорость, большой объем (ГБ), энергозависимая	Шкаф с папками
Внешняя память (ПЗУ, НЖМД)	Долговременное хранение программ и данных, не используемых в данный момент	Низкая скорость, очень большой объем (ГБ — ТБ), энергонезависимая	Архив в другом здании

Тип памяти	Назначение	Характеристики	Аналог из жизни
Стек	Специальная область ОЗУ для временного хранения адресов возврата, параметров функций, содержимого регистров	Организована по принципу LIFO (Last-In, First-Out). Управляется указателем стека (SP)	Стопка тарелок

Иерархия памяти (Memory Hierarchy) (табл. 1.15).

Ключевой принцип организации памяти в МПС — иерархия. Цель — создать иллюзию большой, быстрой и дешевой памяти путем комбинации различных ее типов.

Таблица 1.15

Сводная таблица по основным типам иерархии памяти

Тип памяти	Регистры	Кэш (SRAM)	ОЗУ (DRAM)	Flash (SSD)	HDD	ПЗУ (BIOS)
Технология	Триггеры	SRAM	DRAM	NAND Flash	Магнитные	EEPROM
Скорость	Очень высокая	Очень высокая	Высокая	Средняя	Низкая	Низкая
Объем	Десятки байт	КБ — МБ	ГБ	ГБ — ТБ	ТБ	МБ
Энергозависимость	Да	Да	Да	Нет	Нет	Нет
Назначение	Внутренние операции процессора	Буфер между CPU и RAM	Основная рабочая память	Внешняя память (быстрая)	Внешняя память (дешевая, большая)	Хранение firmware, загрузчика

Уровни иерархии (от самого быстрого и дорогого к самому медленному и дешевому):

1. Регистры процессора → находятся внутри ядра ЦП.

2. Кэш-память 1-го уровня (L1) → разделяется на кэш команд и кэш данных. Находится на кристалле ЦП.

3. Кэш-память 2-го уровня (L2) → находится на кристалле ЦП (часто общий для ядер).

4. Кэш-память 3-го уровня (L3) → находится на кристалле ЦП, общий для всех ядер.

5. Оперативная память (ОЗУ, DRAM) → отдельные микросхемы на материнской плате.

6. Внешняя память (SSD, HDD, сетевые хранилища) → устройства хранения данных.

Принцип работы иерархии: при обращении процессора к данным он сначала ищет их в кэше L1. Если их там нет («промах» кэша), поиск продолжается в L2, затем в L3, затем в ОЗУ и в худшем случае на медленном диске. Найденные данные поднимаются на более высокие уровни иерархии для ускорения последующих обращений.

Классификация памяти в МПС отражает компромисс между тремя ключевыми параметрами:

1) скоростью (быстродействие);

2) объемом (емкость);

3) стоимостью.

Не существует универсальной памяти, которая была бы одновременно огромной, сверхбыстрой и дешевой. Поэтому современные системы используют сложную иерархическую структуру, где каждый уровень памяти компенсирует недостатки предыдущего, создавая в итоге сбалансированную и высокопроизводительную систему.

Классификация ОЗУ, типы и виды ОЗУ, кэш-память — это ключевые компоненты для понимания производительности любой вычислительной системы.

Классификация и типы ОЗУ (RAM).

ОЗУ (оперативное запоминающее устройство) или RAM (Random Access Memory) — это быстрая энергозависимая память, ис-

пользуемая для временного хранения данных и команд, необходимых процессору для выполнения текущих операций:

1. Классификация по физическому принципу (технологии) (табл. 1.16).

Таблица 1.16

Главная классификация, определяющая быстродействие и стоимость памяти

Признак	**DRAM (Dynamic RAM)**	**SRAM (Static RAM)**
Принцип работы	Хранение заряда в конденсаторе. Заряд быстро «стекает», поэтому данные постоянно регенерировать (обновлять)	Хранение данных на триггерах (стабильная схема из нескольких транзисторов). Не требует регенерации
Быстродействие	Относительно низкое (наносекунды, ~100 нс). Медленнее из-за циклов регенерации	Очень высокое (единицы наносекунд, ~10 нс). Быстрее DRAM в ~5–10 раз
Плотность / стоимость	Высокая плотность, низкая стоимость на бит. В одном чипе можно разместить много ячеек	Низкая плотность, высокая стоимость на бит. Ячейка состоит из 4–6 транзисторов
Энергопотребление	Умеренное, но требует энергии для постоянной регенерации	Низкое в статическом режиме, но высокое при активной работе
Основное применение	Основная оперативная память в ПК, серверах, ноутбуках (модули DIMM/DDR)	Кэш-память процессора (L1, L2, L3), буферы в высокоскоростных устройствах

Вывод: SRAM — быстрая и дорогая, используется там, где скорость критична, а объем небольшой. DRAM — медленнее, но дешевле и плотнее, используется как основная память большого объема.

2. Классификация по типу модуля (форм-фактор):

• SIMM (Single In-line Memory Module): устаревший формат. Контакты с двух сторон модуля были соединены;

• DIMM (Dual In-line Memory Module): современный стандарт. Контакты на каждой стороне модуля являются независимыми. Используется в ПК и серверах. SODIMM (Small Outline DIMM): уменьшенная версия DIMM для ноутбуков и компактных устройств.

3. Классификация по архитектуре (для серверов и высокопроизводительных систем):

• обычная (Unbuffered) память: модули напрямую подключены к memory controller. Используется в большинстве ПК;

• регистровая память (Registered, RDIMM): содержит буферный регистр (RCD) между memory controller и чипами DRAM. RCD уменьшает электрическую нагрузку на контроллер, позволяя установить больше модулей памяти на один канал. Повышает стабильность и надежность. Используется в серверах;

• память с полной буферизацией (Fully Buffered, FB-DIMM): более advanced и дорогая технология, чем Registered. Использует advanced memory buffer (AMB) для последовательного соединения модулей. Увеличивает пропускную способность и максимальный объем, но добавляет задержку и тепло. Сейчас используется реже.

Виды и эволюция DRAM (основной оперативной памяти) (табл. 1.17).

Поколения несовместимы между собой! У каждого свой ключ-вырез в контактной площадке и разные рабочие напряжения.

Кэш-память (Cache Memory).

Кэш-память — это небольшой объем сверхбыстрой SRAM, расположенный на кристалле процессора и действующий как буфер между сверхбыстрым ядром процессора и относительно медленной оперативной памятью (DRAM).

Таблица 1.17

Основные поколения DDR SDRAM

Поколение	Название	Напряжение, В	Скорость передачи данных (МТ/с*)	Ключевая особенность
DDR	DDR SDRAM	2,5	200–400	Передача данных 2 раза за такт (по фронту и срезу сигнала)
DDR2	DDR2 SDRAM	1,8	400–1 066	Удвоенная частота шины, большие задержки (тайминги), новый форм-фактор
DDR3	DDR3 SDRAM	1,5	800–2 133	Еще более низкое напряжение, еще более высокая частота, еще большие задержки
DDR4	DDR4 SDRAM	1,2	1 600–3 200	Увеличенная плотность модулей, новый 288-контактный разъем, более высокая эффективность
DDR5	DDR5 SDRAM	1,1	3 200–6 400+	Удвоенная предвыборка, еще более высокая плотность и скорость, встроенная схема управления питанием (PMIC)

** МТ/с (MegaTransfers per second) — миллион передач в секунду. Не путать с МГц (частота).*

Принцип работы (принцип локальности).

Работа кэша основана на двух эмпирических правилах:

1. Пространственная локальность: если процессор обратился к какой-то ячейке памяти, он, скорее всего, в ближайшее время обратится к соседним ячейкам.

2. Временная локальность: если процессор обратился к какой-то ячейке памяти, он, скорее всего, обратится к ней снова в ближайшее время.

Кэш предварительно подгружает в себя данные из ОЗУ, которые, вероятно, понадобятся процессору.

Иерархия кэш-памяти.

Современные процессоры имеют многоуровневую кэш-память (обычно 3 уровня) (табл. 1.18).

Таблица 1.18

Многоуровневая кэш-память (3 уровня)

Уровень	**L1**	**L2**	**L3**
Расположение	Внутри каждого ядра	Внутри каждого ядра (или общий на группу ядер)	Общий на кристалле
Кто использует	Только одно ядро	Только одно ядро (или группа ядер)	Все ядра процессора
Скорость	Самый быстрый	Быстрый	Медленнее, чем L2
Объем	Самый маленький (32–64 КБ)	Больше, чем L1 (256 КБ — 1 МБ)	Самый большой (8–64 МБ и более)
Назначение	Разделен на L1i (кэш инструкций) и L1d (кэш данных)	Универсальный кэш для инструкций и данных	Служит для синхронизации данных между ядрами и является последним буфером перед обращением к ОЗУ

Принципы «Попадание» и «Промах»:

• кэш-попадание (Cache Hit): запрошенные процессом данные найдены в кэше. Это идеальная ситуация, обрабатывается максимально быстро;

• кэш-промах (Cache Miss): Запрошенных данных нет в кэше. Процессору приходится обращаться к медленной оперативной памяти. При этом нужные данные подгружаются в кэш, вытесняя оттуда какие-то старые данные (по специальным алгоритмам; например, LRU — Least Recently Used).

Эффективность кэша измеряется коэффициентом попаданий (Hit Rate) — процентом обращений, которые были обслужены из кэша. Хороший показатель — 90–95 %.

Итак:

• ОЗУ (DRAM) — это основная рабочая память большого объема, но относительно медленная;

• кэш (SRAM) — это сверхбыстрый буфер, скрывающий задержки оперативной памяти. Его объем мал, но он критически важен для производительности;

• вместе они образуют сбалансированную иерархическую систему, которая обеспечивает высокую скорость работы процессора, не требуя от основной памяти быть невероятно быстрой и дорогой.

Классификация ПЗУ, типы и виды ПЗУ.

ПЗУ — это тип памяти, предназначенный для хранения данных, которые не изменяются в процессе эксплуатации устройства и сохраняются после отключения питания (энергонезависимая память).

Классификация ПЗУ.

ПЗУ можно классифицировать по нескольким ключевым признакам:

1. По возможности записи и перезаписи (эволюция технологий).

В табл. 1.19 дана самая распространенная классификация ПЗУ, отражающая ее историческое развитие.

Таблица 1.19

Историческое развитие технологий ПЗУ

Тип ПЗУ	ROM (Mask ROM)	PROM	EPROM	EEPROM	Flash-память
Технология	Масочное ПЗУ	Programmable ROM	Erasable PROM	Electrically Erasable PROM	Разновидность EEPROM
Возможность записи	Завод-изготовитель (один раз)	Пользователь (один раз)	Пользователь (много раз)	Пользователь (много раз)	Пользователь (много раз)
Возможность стирания	Нет	Нет	Да, УФ-излучением	Да, электрически	Да, электрически
Способ стирания	Невозможно	Невозможно	Ультрафиолетовая лампа. Чип имеет окошко для стирания	Электрический сигнал. Стирание и запись без извлечения из схемы	Стирание блоками (блочное). Быстрее, чем побайтное в EEPROM
Типичное применение	Массовое производство. Прошивается once при изготовлении чипа. Дешево в больших тиражах	Прототипирование. Содержит предохранители, которые «пережигаются» программатором	Разработка и отладка. Перед повторной записью чип must be извлечен и помещен under УФ-лампу на ~20 мин	Хранение настроек. BIOS/UEFI, микропрограммы (firmware) устройств, конфигурационные данные	Массовое хранение данных. USB-флешки, SSD-диски, карты памяти (SD, microSD), firmware в смартфонах

Ключевые отличия Flash от EEPROM:

• EEPROM стирается и перезаписывается побайтно;

• Flash-память стирается только целыми блоками (например, по 4 КБ), но работает быстрее и дешевле в производстве на большой объем.

2. По способу программирования (доступа):

• параллельные ПЗУ: для записи/чтения используется шина из множества линий (адресная и data шина). Обеспечивают высокую скорость доступа. Пример: классические EPROM (27C512), EEPROM и Flash в составе embedded systems;

• последовательные ПЗУ: для обмена данными используется последовательный интерфейс (обычно I²C или SPI). Требуют меньше выводов у микроконтроллера, но работают медленнее. Пример: микросхемы EEPROM (24C02, 25AA160A) для хранения небольших объемов данных (калибровочные коэффициенты, настройки).

3. По месту расположения в системе:

• внутреннее ПЗУ: расположено на кристалле микропроцессора или микроконтроллера. Например, память программ в микроконтроллерах AVR (Arduino) или STM32;

• внешнее ПЗУ: отдельная микросхема на плате. Например, чип BIOS на материнской плате ПК или Flash-память в отдельном корпусе.

В табл. 1.20 приведена сводная таблица типов постоянной памяти.

Таблица 1.20

Сводная таблица типов ПЗУ

Параметр	**ROM (Mask)**	**PROM**	**EPROM**	**EEPROM**	**Flash**
Запись	На заводе	1 раз	Много раз	Много раз	Много раз

Параметр	ROM (Mask)	PROM	EPROM	EEPROM	Flash
Стирание	Невозможно	Невозможно	УФ-излучением	Электрически	Электрически (блоками)
Скорость доступа	Высокая	Высокая	Высокая	Средняя	Высокая (чтение) / средняя (запись)
Стоимость	Очень низкая (в mass production)	Низкая	Средняя	Средняя/высокая	Очень низкая (на бит)
Гибкость	Нет	Низкая	Средняя	Высокая	Очень высокая
Основное применение	Готовые массовые продукты	Прототипы	Разработка и отладка	Настройки, firmware	Накопители данных, firmware

Функции и применение ПЗУ в МПС.

1. Хранение firmware (микропрограммы):

• BIOS/UEFI в компьютерах: программа, которая инициализирует hardware при включении и обеспечивает базовые функции ввода-вывода;

• прошивка микроконтроллеров, маршрутизаторов, телевизоров, смартфонов и другой бытовой электроники.

2. Загрузчик (Bootloader).

Небольшая программа, которая запускается самой первой после включения питания. Ее задача — инициализировать hardware и загру-

зить основную операционную систему или более сложную firmware из другой памяти.

3. Хранение констант и данных.

Таблицы символов, шрифты, прошитые звуки или изображения в игровых автоматах и бытовой технике (например, «пищалка» в микроволновке).

4. Хранение конфигурационных данных.

Калибровочные коэффициенты, серийные номера, настройки оборудования, которые не должны теряться при отключении питания. Для этого идеально подходит EEPROM.

Эволюция ПЗУ шла по пути увеличения гибкости и удобства для разработчика:

- от однократно прошиваемых на заводе масок (ROM);
- через чипы, однократно программируемые пользователем (PROM) и стираемые ультрафиолетом (EPROM);
- к современной электрически перезаписываемой памяти (EEPROM и Flash), которая стала основой для всей современной портативной электроники и систем хранения данных.

Flash-память, являясь экономически эффективной и обладающей высокой плотностью, стала доминирующей технологией для энергонезависимого хранения информации в современных устройствах.

Способы адресации в МПС.

Способы адресации — это правила, по которым процессор определяет источник (операнд-источник) и место назначения (операнд-приемник) для данных при выполнении команды.

Проще говоря, это «указания», которые говорят процессору, где взять данные для операции и куда положить результат. От способа адресации напрямую зависят гибкость программирования и эффективность кода.

Классификация способов адресации.

Существует множество способов адресации, но все они могут быть разделены на две большие группы:

1) адресация операндов в памяти;

2) адресация операндов в регистрах.

Рассмотрим основные и наиболее распространенные способы:

1. Непосредственная адресация (Immediate Addressing).

Сам операнд (необходимое значение) содержится непосредственно в коде команды. Например, в рецепте прямо указано: «добавить 100 грамм сахара». Число 100 — это и есть непосредственное значение.

Преимущество: очень быстрый способ, так как данные уже находятся в команде, что не требует дополнительных обращений к памяти или регистрам.

Недостаток: операнд нельзя изменить без изменения самой команды.

Пример в ассемблере (x86):

MOV AX, 0F5h *; Загрузить в регистр AX hexadecimal число F5 (245 в десятичной).*

; Вторая часть команды (0F5h) — это непосредственный операнд.

2. Прямая (абсолютная) адресация (Direct Addressing).

В коде команды указан физический адрес ячейки памяти, в которой находится операнд. Например, в рецепте указано: «взять ингредиент из ящика № 5». № 5 — это прямой адрес.

Преимущество: позволяет напрямую обратиться к любой ячейке памяти.

Недостаток: размер команды увеличивается, так как должен включать в себя полный адрес, который может быть длинным. Медленнее, чем регистровая адресация.

Пример в ассемблере (x86):

MOV AX, [1050h] *; Прочитать содержимое ячейки памяти с адресом 1050h и загрузить его в регистр AX.*

3. Косвенная адресация (Indirect Addressing).

В команде указан адрес не данных, а регистра или ячейки памяти, где *хранится адрес* операнда. Регистр или ячейка выступают в роли указателя. Например, в рецепте указано: «взять ингредиент из

ящика, номер которого записан в блокноте». Сначала нужно заглянуть в блокнот, чтобы узнать номер ящика.

Преимущество: высокая гибкость. Одна и та же команда может обращаться к разным ячейкам памяти, просто меняя значение в регистре-указателе. Широко используется для работы с массивами, строками и структурами данных.

Недостаток: требует дополнительного обращения к памяти для чтения адреса, что замедляет выполнение.

Пример в ассемблере (x86):

MOV SI, 1050h *; Загрузить в регистр-указатель SI адрес 1050h.*

MOV AX, [SI] *; Прочитать данные по адресу, который хранится в SI (т.е. из ячейки 1050h), и поместить их в AX.*

4. Регистровая адресация (Register Addressing).

Операнды находятся внутри регистров процессора. Например, работа с ингредиентами, которые уже лежат на столе прямо перед поваром.

Преимущество: самый быстрый способ адресации, так как регистры находятся прямо внутри процессора и доступ к ним происходит за 1 такт.

Недостаток: очень ограниченное количество регистров.

Пример в ассемблере (x86):

MOV AX, BX *; Скопировать содержимое регистра BX в регистр AX.*

ADD CX, DX *; Сложить содержимое регистров CX и DX, результат поместить в CX.*

5. Относительная адресация (Relative Addressing).

Эффективный адрес операнда вычисляется как сумма содержимого регистра (чаще всего указателя команд IP/EIP/RIP) и смещения (offset), заданного в команде. В основном используется для команд перехода (JMP) и вызова подпрограмм в пределах текущего сегмента кода. Позволяет создавать перемещаемый код (код, который можно загрузить в память по любому адресу).

Пример в ассемблере (x86):

JMP +0A0h *; Перейти на инструкцию, расположенную на 160 байт (A0h) вперед от текущей позиции.*

6. Индексная и базированная адресация (Indexed & Based Addressing) (табл. 1.21).

Эти способы являются вариациями косвенной адресации и идеально подходят для работы с массивами и структурами данных.

Эффективный адрес формируется путем сложения:

- базового адреса (начальный адрес массива, хранящийся в базовом регистре, например BX или BP);
- индекса (смещение до нужного элемента массива, хранящееся в индексном регистре, например SI или DI);
- смещения (offset) (константное значение из команды, например, для доступа к полю структуры).

Разновидности:

- базированная: ЭА = Базовый_регистр + Смещение;
- индексная: ЭА = Индексный_регистр + Смещение;
- базированно-индексная: ЭА = Базовый_регистр + Индексный_регистр + Смещение.

Пример в ассемблере (x86) для доступа к элементу массива:

MOV BX, offset Array *; BX = адрес начала массива (база).*

MOV SI, 4 *; SI = 4 (индекс, хотим 5-й элемент, т.к. индексация часто с 0).*

MOV AX, [BX + SI] *; AX = Array[4] (эффективный адрес = BX + SI).*

Пример для доступа к полю структуры:

; Предположим, есть структура Person с полями age (смещение 0), name (смещение 2).

MOV BX, offset Person1 *; BX указывает на начало структуры.*

MOV AX, [BX] *; AX = Person1.age (чтение по прямому адресу BX + 0).*

MOV AX, [BX + 2] *; AX = Person1.name (чтение по адресу BX + 2).*

Таблица 1.21

Сводная таблица адресации

Способ адресации	Формат операнда (пример)	Преимущества	Недостатки	Применение
Непосредственная	5 или 0F5h	Высокая скорость	Нельзя изменить	Загрузка констант
Прямая	[1050h]	Прямой доступ к памяти	Медленно, длинная команда	Работа с глобальными переменными
Косвенная	[BX]	Гибкость, работа с указателями	Дополнительное обращение к памяти	Строки, динамические структуры
Регистровая	AX	Максимальная скорость	Мало регистров	Внутренние операции процессора
Относительная	[IP+0A0h]	Перемещаемость кода	Ограниченный диапазон	Команды перехода
Индексная/базированная	[BX+SI+02]	Эффективная работа с массивами	Сложность вычисления адреса	Массивы, структуры, стек

Понимание способов адресации критически важно:

- для низкоуровневого программирования (ассемблер);
- написания компиляторов;

• оптимизации кода (выбор более быстрого способа адресации ведет к ускорению программы).

Компиляторы часто используют комбинации этих способов для генерации максимально эффективного машинного кода.

1.7. ОРГАНИЗАЦИЯ СВЯЗИ МПС С ВНЕШНЕЙ СРЕДОЙ. ФУНКЦИИ УСТРОЙСТВ ВВОДА-ВЫВОДА. ПРИНЦИПЫ ПОСТРОЕНИЯ ПОРТОВ ВВОДА-ВЫВОДА

Организация связи МПС с внешней средой — это комплекс методов и аппаратных средств, обеспечивающих взаимодействие микропроцессора с периферийными устройствами (внешней средой). Это одна из ключевых задач при проектировании любой микропроцессорной системы.

Основные задачи и проблемы связи.

Главная задача — организовать обмен данными между быстрым центральным процессором и относительно медленными внешними устройствами (ВУ). Основные проблемы, которые при этом решаются:

• согласование скоростей: скорость работы ЦП намного выше скорости большинства ВУ (клавиатура, мышь, дисковод);

• согласование форматов данных: ЦП оперирует двоичными словами (байтами, словами), а ВУ могут использовать аналоговые сигналы, последовательные коды и т.д.;

• синхронизация: ЦП должен знать, когда устройство готово к обмену данными;

• согласование уровней сигналов: логические уровни ЦП (например, 0В и 3.3В) могут не совпадать с уровнями сигналов ВУ.

Аппаратная основа: порты ввода-вывода.

Для связи с внешним миром в МПС используются порты ввода-вывода (I/O ports). Это специальные адресуемые ячейки, которые бывают двух типов:

1. Порты ввода: через них ЦП читает данные от внешнего устройства (например, состояние кнопки).

2. Порты вывода: через них ЦП передает данные на внешнее устройство (например, включает светодиод).

Физически порты могут быть:

• выделенными: отдельное адресное пространство (используются специальные команды IN/OUT в x86);

• отображаемыми на память (Memory-Mapped I/O): порты представляются как ячейки обычной оперативной памяти. Для работы с ними используются стандартные команды работы с памятью (MOV, LDR/STR в ARM). Этот способ более гибкий и распространен в современных архитектурах (ARM, AVR).

Способы (режимы) организации обмена данными.

Существует три основных способа организации обмена, которые отличаются степенью участия процессора:

1. Программный обмен (управляемый ЦП).

Весь обмен данными осуществляется под непосредственным контролем процессора путем выполнения специальной программы:

А. Обмен в режиме опроса (Polling): ЦП периодически (в цикле) опрашивает регистр состояния внешнего устройства, проверяя специальный флаг готовности.

Алгоритм: «Готово? → Нет → Жду → Готово? → Да → Передаю данные».

Плюс: простота реализации.

Минус: низкая эффективность, так как ЦП тратит много времени на бесполезный опрос.

Используется в простых системах, работа с очень медленными устройствами.

Б. Обмен по прерываниям (Interrupt-Driven I/O): ВУ, когда оно готово к обмену, самостоятельно инициирует процесс, посылая ЦП запрос на прерывание (IRQ).

Алгоритм: ЦП выполняет свою программу → ВУ посылает IRQ → ЦП прерывает текущую работу → выполняет процедуру обработки прерывания (ISR) → по завершении возвращается к прерванной программе.

Плюс: ЦП не простаивает, а занимается полезной работой в ожидании готовности ВУ (высокая эффективность).

Минус: более сложная программная и аппаратная реализация (нужен контроллер прерываний).

Это наиболее распространенный способ для работы с большинством устройств (клавиатура, мышь, таймер).

2. Обмен в режиме ПДП (прямого доступа к памяти).

Для пересылки *больших блоков данных* между ВУ и памятью привлекается специальный аппаратный модуль — контроллер ПДП (DMA-контроллер). ЦП лишь инициализирует этот контроллер, а затем освобождается для другой работы.

Алгоритм:

1. ЦП настраивает DMA: задает адрес в памяти, количество байт, направление передачи.

2. ВУ готово → посылает запрос DMA-контроллеру.

3. DMA-контроллер запрашивает у ЦП право на использование системной шины.

4. ЦП подтверждает запрос и «отпускает» шину.

5. DMA-контроллер напрямую (без участия ЦП) управляет шиной и пересылает данные между ВУ и памятью.

6. По окончании передачи DMA-контроллер сообщает ЦП о завершении операции (часто через прерывание).

Плюсы: максимальная скорость обмена, минимальная нагрузка на ЦП.

Минус: сложная аппаратная реализация.

Используется для работы с высокоскоростными устройствами (сетевые карты, жесткие диски, SSD, звуковые карты).

3. Обмен через специализированные процессоры ввода-вывода.

Для управления сложными и высокоскоростными устройствами (например, видеокарта, современный SSD) используются их собственные встроенные специализированные процессоры (контроллеры).

Алгоритм: ЦП посылает такому контроллеру высокоуровневые *команды* (например, «отрендери эту 3D-сцену» или «прочитай дан-

ные из этих секторов»), а весь объемный низкоуровневый обмен данными контроллер устройства берет на себя, часто используя свой собственный DMA.

Используется как графические ускорители (GPU), сложные сетевые адаптеры, контроллеры дисковых массивов.

4. Схема организации связи (на примере обмена по прерываниям) (табл. 1.22):

1. Аппаратный интерфейс:

• устройство подключается к порту ввода-вывода и к линии запроса прерывания на системной шине;

• контроллер прерываний (PIC/APIC) управляет приоритетами запросов от разных устройств.

2. Программный интерфейс:

• в памяти существует таблица векторов прерываний, где для каждого типа прерывания хранится адрес своей процедуры обработки прерывания (ISR — Interrupt Service Routine);

• программист пишет эти процедуры, которые непосредственно взаимодействуют с портами устройства.

3. Процесс обмена:

• инициализация: ЦП разрешает прерывания от устройства;

• ожидание: ЦП выполняет свою программу;

• запрос: устройство готово → выставляет сигнал IRQ;

• обработка: срабатывает ISR, которая читает/пишет данные через порты;

• завершение: ISR посылает устройству команду завершения и возвращает управление основной программе.

Организация связи МПС с внешней средой — это всегда поиск компромисса между скоростью, нагрузкой на процессор и сложностью проектирования. Правильный выбор способа обмена критически важен для производительности и эффективности всей системы.

Таблица 1.22

Сравнение способов обмена

Способ обмена	Загрузка ЦП	Скорость обмена	Сложность реализации	Применение
Программный (опрос)	Максимальная	Низкая	Низкая	Простейшие системы, медленные устройства
По прерываниям	Средняя	Средняя	Средняя	Основной способ для большинства устройств
ПДП (DMA)	Минимальная	Высокая	Высокая	Высокоскоростные устройства (диски, сеть, звук)
Через спец. процессоры	Очень низкая	Очень высокая	Очень высокая	Графика, сложные накопители

Функции устройств ввода-вывода (УВВ) — это набор задач, которые выполняют эти устройства и их контроллеры для обеспечения корректного и эффективного взаимодействия между внешним миром и центральным процессором (ЦП) микропроцессорной системы (МПС).

УВВ выступают в роли посредников или переводчиков между человеком, окружающей средой и цифровым миром процессора.

Ключевые функции устройств ввода-вывода (табл. 1.23):

1. Преобразование сигналов и данных (Signal Conversion & Data Transformation).

Это основная и фундаментальная функция:

• физический → цифровой (для устройств ввода):

– аналого-цифровое преобразование (АЦП): преобразование непрерывного аналогового сигнала (например, напряжение с микрофона, температура с датчика) в дискретный цифровой код, понятный процессору;

– дискретизация: преобразование механических или других физических воздействий в цифровые сигналы (например, нажатие клавиши на клавиатуре замыкает контакт, что интерпретируется как логическая '1');

• цифровой → физический (для устройств вывода):

– цифроаналоговое преобразование (ЦАП): преобразование цифрового кода от процессора в аналоговый сигнал (например, напряжение для динамика, чтобы воспроизвести звук);

– «оживление» данных: преобразование цифровых данных в физическое действие (например, цифровой код превращается в магнитное поле для записи на диск, в управляющий импульс для двигателя или в пиксель на экране).

2. Буферизация данных (Buffering).

Временное хранение данных в специальной памяти (буфере) для компенсации разницы в скорости работы ЦП и внешнего устройства.

Необходимо:

• для входных устройств: данные от медленного устройства (например, клавиатуры) накапливаются в буфере, и, когда их набирается достаточно (или по сигналу прерывания), процессор считывает их сразу пачкой;

• выходных устройств: процессор быстро записывает данные в буфер (например, видеопамять), а контроллер дисплея самостоятельно, с постоянной скоростью считывает их оттуда для формирова-

ния изображения. Это освобождает ЦП от необходимости выводить каждый пиксель вручную.

3. Согласование протоколов и форматов данных (Protocol & Format Matching).

Преобразование данных из формата, удобного для процессора (параллельные шины данных), в формат, требуемый для конкретного канала связи, и наоборот. Например:

• последовательный ↔ параллельный: процессор оперирует данными «целыми словами» (например, 32 бита сразу). Для передачи по последовательной линии связи (например, UART, USB, Ethernet) контроллер УВВ преобразует эти данные в последовательность битов. На приемной стороне другой контроллер выполняет обратное преобразование;

• кодирование/декодирование: контроллер сетевой карты добавляет к данным служебную информацию (заголовки пакетов, контрольные суммы) для передачи по сети и удаляет ее при приеме.

4. Синхронизация потоков данных и управление ими (Synchronization & Flow Control).

Обеспечение того, что данные передаются тогда, когда получатель готов их принять.

Алгоритм реализации:

• через статусные регистры: УВВ имеет регистр состояния, где выставляются флаги (например, «буфер пуст», «буфер полон», «ошибка»). ЦП может опрашивать эти флаги (режим опроса) или получать уведомление через прерывание;

• аппаратные сигналы: использование специальных линий для управления потоком, например RTS/CTS (Request to Send / Clear to Send), в последовательной связи.

5. Гальваническая развязка и защита (Isolation & Protection).

Электрическая изоляция чувствительной микропроцессорной схемы от потенциально опасных или «шумных» внешних цепей.

Необходимо для защиты МПС от высоких напряжений, бросков тока, электромагнитных помех, которые могут присутствовать в про-

мышленных сетях, или от мощных исполнительных механизмов (двигатели, реле).

Реализуется с помощью оптронов (оптоэлектронные развязки), трансформаторов, специализированных микросхем.

6. Управление и конфигурирование (Control & Configuration).

УВВ часто имеют регистры управления, через которые ЦП может гибко настраивать их работу, не меняя аппаратную часть.

Примеры настроек:

- скорость обмена (Baud Rate);
- размер данных (7–8 бит);
- четность (Parity);
- режим прерываний (разрешить/запретить);
- направление передачи (для портов ввода-вывода, которые могут работать в обоих направлениях).

7. Логическое управление внешними устройствами (Device Control).

Прямое управление состоянием подключенного внешнего устройства путем подачи на него управляющих сигналов.

Примеры:

- подача импульса на электромагнитное реле для включения двигателя;
- управление шаговым двигателем (подача импульсов на обмотки);
- инициирование процесса чтения/записи в подключенное устройство (например, команда «прочитать сектор» для SD-карты).

Устройства ввода-вывода — это не просто «проводники» данных. Это интеллектуальные интерфейсы, которые выполняют сложную работу по преобразованию, буферизации, синхронизации и защите данных, позволяя центральному процессору эффективно взаимодействовать с бесконечно разнообразным внешним миром, не вдаваясь в его физические подробности.

Таблица 1.23

Сводная таблица функций

Функция	Устройство ввода (пример: мышь)	Устройство вывода (пример: монитор)
Преобразование	Механическое движение → цифровые координаты. (Оптический сенсор → АЦП → данные)	Цифровой код цвета → интенсивность свечения пикселя. (ЦАП → транзистор → жидкий кристалл / светодиод)
Буферизация	Накопление данных о перемещении и кликах перед отправкой пакетом по USB	Кадровый буфер (видеопамять). Хранит весь кадр изображения, который постоянно сканируется контроллером
Согласование	Преобразование данных в пакеты протокола USB	Прием данных по DisplayPort/HDMI, декодирование видеопотока
Синхронизация	Отправка запроса на прерывание, когда есть данные для передачи	Синхронизация с источником сигнала (частота кадров, разрешение)
Управление	Регистры для настройки частоты опроса, разрешения датчика	Регистры для настройки яркости, контрастности, выбора источника сигнала

Принципы построения портов ввода-вывода — это фундаментальные правила и методы, по которым организуется аппаратный интерфейс для связи микропроцессора с внешними устройствами.

Порты ввода-вывода (I/O ports) — это специальные аппаратные ячейки, выступающие в роли «окон» между цифровым миром процессора и внешней средой.

Базовые принципы:

1. Адресуемость.

Каждый порт (или регистр порта) имеет уникальный адрес в пространстве ввода-вывода. Это позволяет процессору однозначно обращаться к конкретному устройству, используя команды ввода-вывода (например, IN и OUT в x86) или обращаясь к ячейке памяти (Memory-Mapped I/O).

2. Направление передачи данных:

• порты ввода: предназначены для чтения данных процессором от внешнего устройства (например, состояние кнопки);

• порты вывода: предназначены для записи данных процессором во внешнее устройство (например, включение светодиода);

• двунаправленные порты: могут динамически настраиваться на работу как на ввод, так и на вывод. Направление обычно задается через специальный регистр управления направлением (DDR — Data Direction Register).

3. Согласование уровней и нагрузочной способности.

Порты должны обеспечивать электрическую совместимость между логическими уровнями процессора (например, 3.3 В) и подключаемыми устройствами (которые могут работать от 5 В или иметь аналоговую природу). Для этого используются:

• буферы (Buffer) и трансиверы (Transceiver): усиливают сигнал, увеличивают нагрузочную способность порта;

• преобразователи уровней (Level Shifter): преобразуют логические уровни между разными семействами логики;

• оптроны (Optocoupler): обеспечивают гальваническую развязку, защищая процессор от высоких напряжений и помех.

Схемотехническая организация портов.

Порты ввода-вывода строятся на основе триггеров (для хранения данных) и буферов (для усиления сигнала и управления направлением):

1. Организация порта вывода.

Порт вывода строится на D-триггере с выходным буфером.

Принцип работы:

• шина данных процессора подключена к входам D-триггеров;

• дешифратор адреса и схемы управления формируют сигнал записи WR для конкретного порта;

• по сигналу WR данные с шины процессора фиксируются (защелкиваются) в триггере;

• выход триггера через буфер постоянно подает сохраненное значение на внешнее устройство до следующей операции записи.

Это обеспечивает хранение выходного сигнала даже после того, как процессор перешел к другим задачам.

Упрощенная схема порта вывода на D-триггере.

2. Организация порта ввода.

Порт ввода строится на трехстабильных буферах (буферах с состоянием высокого импеданса).

Принцип работы:

• внешние сигналы подключены к входам буферов;

• выходы буферов подключены к шине данных процессора;

• дешифратор адреса и схемы управления формируют сигнал чтения RD для конкретного порта;

• по сигналу RD буферы активируются и передают текущее состояние внешних сигналов на шину данных процессора;

• в отсутствие сигнала RD выходы буферов переходят в высокоимпедансное состояние (Z-state), effectively отключаясь от шины и позволяя другим устройствам использовать ее.

Упрощенная схема порта ввода на трехстабильных буферах.

3. Организация двунаправленного порта.

Это комбинация порта ввода и порта вывода, где направление передачи данных контролируется отдельным триггером (регистром DDR):

• если в DDR записана '1': соответствующий бит порта работает на выход. Внутренний выходной буфер включается;

• если в DDR записана '0': соответствующий бит порта работает на вход. Выходной буфер отключается (переходит в Z-state), и порт начинает принимать внешние сигналы.

Способы адресации портов ввода-вывода.

Существует два основных архитектурных подхода к организации доступа к портам:

1. Изолированный ввод-вывод (Port-Mapped I/O).

Порты ввода-вывода имеют отдельное, независимое адресное пространство, не связанное с адресным пространством оперативной памяти.

Для доступа к портам используются специальные команды процессора (IN, OUT в x86).

Преимущества:

- не занимает адресное пространство памяти;
- более короткие и быстрые команды;
- четкое разделение между доступом к памяти и к устройствам.

Недостаток: меньший набор команд для работы с портами.

2. Ввод-вывод, отображаемый на память (Memory-Mapped I/O).

Порты ввода-вывода отождествляются с ячейками оперативной памяти. Регистрам устройств назначаются конкретные адреса в общем адресном пространстве.

Для доступа к портам используются те же команды, что и для работы с памятью (MOV, LDR, STR).

Преимущества:

- более богатый набор команд для работы с данными (арифметические, логические операции) можно напрямую применять к регистрам устройств;
- упрощенная декодировка адресов;
- более гибкая и простая для программирования модель.

Недостатки:

- «съедает» часть адресного пространства памяти;
- требует более сложной схемы дешифрации адресов.

Этот подход доминирует в современных архитектурах (ARM, AVR, RISC-V).

Дешифрация адресов.

Чтобы активировать конкретный порт при обращении процессора к его адресу, используется дешифратор адресов. Это комбинационная логическая схема, которая:

1) анализирует старшие биты адресной шины;

2) активирует сигнал выбора микросхемы (CS — Chip Select) для определенного устройства, только когда адрес попадает в назначенный ему диапазон;

3) младшие биты адреса используются для выбора конкретного регистра внутри микросхемы устройства.

Ключевые принципы построения.

Аппаратная защелка (latch) для вывода: данные на выходе должны сохраняться до следующей команды записи.

Буферизация для ввода: данные на входе должны подключаться к шине данных только в момент чтения, чтобы не мешать работе других устройств.

Управление направлением: возможность программно настраивать назначение выводов (вход–выход) через регистр DDR.

Четкая адресация: каждый порт должен иметь уникальный адрес для точного выбора процессором.

Электрическое согласование: порты должны обеспечивать корректную работу с внешними цепями через буферы, преобразователи уровней и развязку.

Выбор модели доступа: решение об использовании изолированного I/O или memory-mapped I/O является фундаментальным архитектурным выбором.

Эти принципы лежат в основе работы всех микроконтроллеров и интерфейсных блоков современных процессоров.

КОНТРОЛЬНЫЕ ВОПРОСЫ ПО ТЕМЕ «МИКРОПРОЦЕССОРНЫЕ СИСТЕМЫ»

Базовый уровень:

1. Что такое микропроцессорная система? Назовите ее основные компоненты.

2. Дайте определение микропроцессору. Каковы его основные функции?

3. Что такое системная шина? Из каких основных шин она состоит и какую функцию выполняет каждая из них?

4. Чем отличается ОЗУ от ПЗУ? Их основные характеристики (энергозависимость, скорость, назначение).

5. Что такое порт ввода-вывода? Приведите примеры устройств ввода и вывода.

Средний уровень:

6. Опишите цикл выполнения команды микропроцессором (Fetch-Decode-Execute).

7. Что такое прерывание? Опишите процесс обработки прерывания. Для чего нужен контроллер прерываний (PIC)?

8. Что такое прямой доступ к памяти (ПДП)? Какую проблему он решает и как работает?

9. Объясните разницу между микропроцессором (МП) и микроконтроллером (МК). Где typically применяется каждый из них?

10. Что такое система команд (ISA)? В чем разница между архитектурами RISC и CISC? Приведите примеры процессоров каждого типа.

Продвинутый уровень:

11. Опишите иерархию памяти в МПС. Почему она организована именно так (объясните компромисс «скорость — объем — стоимость»)?

12. Что такое кэш-память? Какие уровни кэша существуют и чем они отличаются? Что такое «попадание» и «промах» кэша?

13. Объясните понятие «способ адресации». Опишите и приведите примеры команд для следующих способов: непосредственного, прямого, регистрового, косвенного.

14. Что такое конвейеризация команд? Как она повышает производительность процессора?

15. Назовите основные режимы работы современного процессора (например, в архитектуре x86). Чем защищенный режим отличается от реального?

Вопросы на практическое понимание:

16. Почему для работы с медленными устройствами выгоднее использовать прерывания, а не режим опроса?

17. Какой способ адресации наиболее эффективен для доступа к элементам массива? Почему?

18. Где в МПС может храниться программа начальной загрузки (bootloader) и почему именно там?

19. Почему в современных системах используется несколько уровней кэш-памяти (L1, L2, L3), а не один большой?

20. Каковы преимущества и недостатки передачи управления шиной контроллеру ПДП?

ГЛАВА 2. ВСТРАИВАЕМЫЕ СИСТЕМЫ НА ОСНОВЕ МИКРОКОНТРОЛЛЕРОВ

2.1. ОБЗОР СОВРЕМЕННЫХ МИКРОКОНТРОЛЛЕРОВ (МК). КЛАССИФИКАЦИЯ МК. МОДУЛЬНАЯ ОРГАНИЗАЦИЯ МК

Обзор современных микроконтроллеров — это взгляд на ключевые игроки, архитектуры и тренды в мире «электронных мозгов», которые управляют всем вокруг нас — от зубной щетки до промышленного робота.

Современный микроконтроллер — это не просто процессор. Это законченный компьютер на одной микросхеме (SoC — System on a Chip), который включает:

- ядро процессора (CPU);
- оперативную память (RAM);
- память программ (Flash);
- тактовый генератор;
- периферийные устройства: таймеры, АЦП/ЦАП, интерфейсы связи (UART, I2C, SPI, USB, CAN, Ethernet), контроллеры дисплеев и многое другое.

Современные тренды:

- низкое энергопотребление: основная битва идет за микроамперы в режиме сна;
- высокая производительность: появление ядер с тактовой частотой под 1 ГГц;
- ннтеграция: в одну микросхему добавляют WiFi, Bluetooth, криптоускорители;
- экосистема: важность не только «железа», но и удобства разработки, поддержки сообщества.

Ключевые архитектуры ядер (табл. 2.1).

Таблица 2.1

Характеристики ARM Cortex-M и RISC-V

Архитектура	**ARM Cortex-M**	**RISC-V**
Суть	Проприетарная архитектура. Компания ARM разрабатывает ядра и лицензирует их производителям чипов	Открытая архитектура. Любой может использовать и дорабатывать ее без лицензионных отчислений
Преимущества	Де-факто стандарт в индустрии. Огромная экосистема инструментов и кодовая база. Высокая производительность и энергоэффективность	Открытость и бесплатность (нет роялти). Полная кастомизация ядра под задачи. Быстрорастущее сообщество и поддержка
Недостатки	За лицензию и использование ядер нужно платить ARM и производителю	Меньше готовых оптимизированных решений, экосистема моложе
Примеры	STM32 (ST), nRF52 (Nordic), RP2040 (Raspberry Pi, как Cortex-M0+)	Микроконтроллеры от GD32, HiFive1, WCH, а также кастомные чипы многих компаний

Также существуют проприетарные архитектуры (например, AVR от Microchip, PIC, MSP430 от TI), но их доля рынка сокращается в пользу ARM и RISC-V.

Обзор популярных семейств и производителей (табл. 2.2):

1. Для общего применения и обучения: STM32 (производитель — STMicroelectronics).

Имеют архитектуру: ARM Cortex-M (M0, M3, M4, M7, M33). Большую популярность приобрели из-за невероятно широкого ассор-

тимента (от дешевых 8-пиновых чипов до мощных с DSP и FPU). Отличная документация и поддержка. Огромное сообщество.

По сложности — средняя и высокая. Требует понимания периферии.

Предназначены: для инженеров, студентов, любителей. Фактически промышленный стандарт.

2. Для встраиваемого WiFi/Bluetooth: ESP32 (Espressif Systems).

Имеют архитектуру: Xtensa (собственная) или RISC-V (в новых версиях) + ядра для беспроводного стека. Популярны благодаря феноменальному соотношению цены, производительности и наличия беспроводных интерфейсов (Wi-Fi, Bluetooth, BLE). Огромное сообщество.

По сложности — низкая. Очень прост в освоении благодаря Arduino-платформе.

Предназначены: для IoT-проектов, умного дома, сети датчиков.

3. Для сверхнизкого энергопотребления: nRF52/nRF53 (Nordic Semiconductor).

Имеют архитектуру: ARM Cortex-M4/M33. Лидеры в области энергоэффективности и качества реализации Bluetooth Low Energy (BLE). Профессиональный инструмент.

По сложности — средняя. Сильный акцент на работу с радиочастью.

Предназначены: для коммерческих IoT-устройств, носимых гаджетов (wearables), медицинских приборов, работающих от батареи.

4. Для обучения и хобби: Arduino (на базе AVR/ARM) & RP2040 (Raspberry Pi).

Имеют архитектуру: AVR (ATmega328p) или ARM (в более новых платах), а также собственное ядро в RP2040.

Невероятно простая среда разработки (Arduino IDE), огромное количество уроков и библиотек. RP2040 от Raspberry Pi — это современный, дешевый и очень мощный МК с уникальной периферией (PIO). Очень низкая сложность.

Предназначены: для начинающих, любителей, быстрых прототипов.

5. Мощные и специализированные: микроконтроллеры от TI, Microchip, Infineon, NXP.

Имеют архитектуру: ARM Cortex-M, RISC-V, проприетарные ядра (детали архитектуры — коммерческая тайна). Имеют высокую сложность.

Популярны, так как эти компании доминируют в промышленной, автомобильной и аэрокосмической отраслях. Их МК предлагают:

- высокую надежность и стойкость к внешним воздействиям;
- специализированную периферию (моторы, силовая электроника, CAN FD, EtherCAT);
- долгую доступность компонентов (long-term supply).

Предназначены: для профессиональных инженеров, работающих над промышленными, автомобильными и медицинскими проектами.

Таблица 2.2

Сравнительная таблица современных МК

Семейство/ производитель	**Архитектура ядра**	**Ключевые преимущества**	**Недостатки**	**Идеальная сфера применения**
STM32 (ST)	ARM Cortex-M	Широчайший выбор, сообщество, docs	Огромный выбор усложняет поиск	Универсальная, промышленная, авто
ESP32 (Espressif)	Xtensa/RISC-V	Цена, Wi-Fi/BT, сообщество	Не для high-end RF задач	IoT, умный дом, сети датчиков

Семейство/ производитель	Архитектура ядра	Ключевые преимущества	Недостатки	Идеальная сфера применения
nRF52/53 (Nordic)	ARM Cortex-M	Энергопотребление, качество BLE	Цена выше, чем у ESP	BLE-гаджеты, носимые устройства
RP2040 (Raspberry Pi)	Dual-Core ARM M0+	Цена, PIO, сообщество	Нет встроенной Flash, нет аналогов	Хобби, образование, уникальные задачи
PIC/AVR (Microchip)	PIC/AVR	Простота, надежность, долгая поставка	Устаревающая архитектура	Простая автоматика, образование
MSP430 (TI)	Propietary 16-bit	Рекордно низкое энергопотребление	Низкая производительность	Батарейные устройства с малой логикой

Критерии выбора современного микроконтроллера.

Производительность: тактовая частота, наличие FPU (блок вычислений с плавающей точкой), DSP-инструкций.

Память: объем Flash (для кода) и RAM (для данных). Важно для сложных протоколов и графики.

Энергопотребление: ток в активном режиме и, что важнее, в режимах ожидания (sleep, deep sleep).

Периферия: нужные вам интерфейсы (UART, I2C, ADC/DAC, USB, Ethernet, CAN, LCD).

Стоимость: цена за чип в нужных вам объемах.

Наличие и сроки поставки: крайне актуальный вопрос в эпоху чипового кризиса.

Экосистема: наличие готовых библиотек, качественной документации, активного сообщества и удобных инструментов разработки (IDE, отладочные платы, программаторы).

Современный рынок микроконтроллеров переживает ренессанс. ARM Cortex-M остается индустриальным стандартом, но RISC-V активно набирает обороты благодаря своей открытости:

- для хобби и стартапов идеальны ESP32 и Raspberry Pi RP2040;
- для профессиональной разработки универсальным выбором остается STM32;
- для устройств с батарейным питанием нет равных Nordic nRF52;
- для промышленности и авто доминируют TI, NXP, Infineon и Microchip.

Главный тренд — переход от простых «чипов с портами» к сложным системам на кристалле, которые берут на себя все больше задач, оставляя разработчику работу на уровне программной логики и протоколов.

Классификация микроконтроллеров — это структурирование их по ключевым признакам, что помогает инженерам и разработчикам быстро выбрать подходящий чип для конкретной задачи.

Классифицировать МК можно по множеству критериев. Рассмотрим основные из них:

1. По разрядности (битности) ядра (табл. 2.3).

Это наиболее общая и известная классификация, определяющая ширину данных, с которой может работать процессор за один такт.

2. По архитектуре системы команд.

Определяет, как организована память и как процессор обращается к данным:

- гарвардская архитектура;
- фоннеймановская архитектура.

Таблица 2.3

Классификация данных, обрабатываемых процессором

Разрядность	Характеристики	Примеры архитектур	Область применения
8-битные	Низкая стоимость и энергопотребление. Ограниченная производительность и объем памяти. Простая архитектура	AVR (ATmega), PIC, 8051, HC08	Простейшие задачи: пульты ДУ, игрушки, датчики, управление двигателями, кнопки/ светодиоды
16-битные	Баланс между производительностью и энергопотреблением. Более развитая периферия	MSP430 (TI), PIC24 (Microchip), RL78 (Renesas)	Более сложные устройства: медицинские датчики, измерительные приборы, цифровые термостаты, системы сбора данных
32-битные	Высокая производительность. Большие объемы памяти (Flash, RAM). Сложная периферия (Ethernet, USB, LCD). Часто есть FPU (блок вычислений с плавающей точкой)	ARM Cortex-M (M0, M3, M4, M7, M33), RISC-V, PIC32 (MIPS), Tricore (Infineon)	Сложные системы: IoT-устройства, смартфоны (как сопроцессоры), промышленная автоматика, сети, мультимедийные интерфейсы

Разрядность	Характеристики	Примеры архитектур	Область применения
64-битные	Увеличенная разрядность регистров. Огромный объем памяти до 16 эксабайт (ЭБ). Большее количество регистров. Новые инструкции и технологии: SSE2/SSE3, AES-NI, виртуализации (Intel VT-x, AMD-V) и др. Обратная совместимость	• x86-64 (AMD64 / Intel 64); • AArch64 (ARM64); • SPARC64, POWER/ PowerPC, RISC-V; • IA-64 (Itanium); • MIPS64, Alpha	Персональные компьютеры и ноутбуки. Серверы и центры обработки данных: x86-64 (Intel Xeon, AMD Epyc), ARM64 (AWS Graviton, Ampere Altra), POWER, SPARC. Высокопроизводительные вычисления (HPC) и суперкомпьютеры. Мобильные устройства. Встраиваемые системы и Интернет вещей (IoT). Игровые и мультимедийные консоли

3. По набору инструкций (ISA):

• CISC (Complex Instruction Set Computer): архитектура со сложным набором команд. Одна команда может выполнять несколько низкоуровневых операций.

Примеры: 8051, архитектура x86.

Плюс: плотный код (меньше занимает места в памяти).

Минусы: сложное устройство процессора, разное время выполнения команд;

• RISC (Reduced Instruction Set Computer): архитектура с сокращенным набором команд. Команды простые, выполняются за один такт.

Примеры: AVR, ARM Cortex-M, RISC-V, MSP430, PIC18.

Плюсы: высокая и предсказуемая производительность, проще конвейеризация.

Минус: код менее плотный.

Современный тренд: подавляющее большинство новых МК используют RISC-архитектуру (особенно ARM и RISC-V) из-за ее эффективности.

4. По производителю и семейству (табл. 2.4).

Это практическая классификация, по которой чаще всего и происходит выбор.

Таблица 2.4

Классификация МК по производителю и семейству

Производитель	**Популярные семейства (проприетарные/ARM)**	**Особенности и ниша**
Microchip	PIC (8/16/32-bit), AVR (8-bit), SAM (ARM)	Невероятно широкий ассортимент, надежность, долгосрочная доступность. Промышленность, авто
STMicroelectronics (ST)	STM32 (ARM Cortex-M)	Лидер по популярности. Огромное сообщество, документация, выбор. Универсальное применение

Производитель	Популярные семейства (проприетарные/ARM)	Особенности и ниша
Texas Instruments (TI)	MSP430 (16-bit), CC13xx/26xx (ARM Cortex-M)	Лидер в сверхнизком энергопотреблении (MSP430) и беспроводных решениях
Espressif Systems	ESP32 (Xtensa/RISC-V)	Лидер по доступным Wi-Fi-, BT-решениям. Огромная популярность в IoT и у любителей
Nordic Semiconductor	nRF52/nRF53 (ARM Cortex-M)	Лидер в сегменте Bluetooth Low Energy (BLE). Профессиональные wearable- и IoT-устройства
Raspberry Pi	RP2040 (ARM Cortex-M0+)	Уникальная периферия (PIO), низкая цена, большое сообщество. Хобби и образование
Renesas	RL78, RX, RA (ARM)	Мощные решения для автомобильной и промышленной электроники
Infineon	PSoC, AURIX	Высокая интеграция (PSoC — «программируемая система на кристалле»), автомобильный сектор
NXP	LPC, Kinetis (ARM)	Мощные МК для сложных промышленных и потребительских применений

Отдельно стоит рассмотреть 64-битные МК, так как они сейчас почти вытеснили остальные архитектуры.

Продолжение таблицы 2.4

Классификация МК по производителю и семейству 64-битных МК

Производитель	**Ключевое 64-битное семейство**	**Базовая архитектура (ядро)**	**Типичные области применения**
NXP	i.MX 8/9, Layerscape	ARM Cortex-A (A72, A53, A55)	Автомобиль (панели), пром. HMI, сетевые шлюзы
STMicro	Stellar	ARM Cortex-R52 (64-бит)	Автоэлектроника (безопасность, двигатели)
Microchip	PolarFire SoC FPGA	RISC-V (SiFive U54)	Телеком, защищённые системы, IoT-шлюзы
TI	Sitara AM62x/AM64x, Jacinto	ARM Cortex-A (A53, A72)	Промышленность, робототехника, ADAS
Renesas	RZ/G2, RZ/V2	ARM Cortex-A (A57, A53, A55)	Промышленность со сложным GUI, встроенный ИИ
Espressif	ESP32-P4 (анонс)	RISC-V (T-Head C6)	Продвинутые IoT-устройства, прототипирование
Allwinner	H-серия, D-серия	ARM Cortex-A (A53, A7)	Бюджетные одноплатники, мультимедиа

5. По сфере применения и специализации:

• универсальные: предназначены для широкого круга задач. Имеют стандартный набор периферии.

Примеры: STM32, AVR, PIC;

• специализированные: оптимизированы для конкретных задач:

– сверхнизкое энергопотребление (Ultra-Low Power): MSP430, STM32L, nRF52;

– беспроводная связь (Wireless): SP32 (Wi-Fi/BT), nRF52 (BLE), CC13xx (Sub-1 GHz);

– управление двигателями (Motor Control): имеют advanced таймеры, ШИМ-контроллеры, компараторы;

– автомобильные (Automotive): соответствуют стандарту AEC-Q100, работают в расширенном температурном диапазоне;

– сенсорные функции (Touch): имеют встроенные контроллеры емкостных сенсорных кнопок.

6. По объему памяти.

• микроконтроллеры с малой памятью: Flash: 1–16 КБ, RAM: 0.1–2 КБ (8-битные PIC, AVR);

• микроконтроллеры со средней памятью: Flash: 32–256 КБ, RAM: 4–32 КБ (STM32F1, многие ESP32);

• микроконтроллеры с большой памятью: Flash: 512 КБ — 2 МБ+, RAM: 128–512 КБ+ (STM32H7, PIC32MZ). Фактически это уже одноплатные компьютеры (SBC).

Итог: как выбрать?

Классификация помогает сузить круг поиска, задавая правильные вопросы:

1. Какая нужна производительность? → выбираем разрядность (8/16/32-bit).

2. Какая периферия нужна? (USB, Ethernet, ADC, etc.) → смотрим на семейства производителей (STM32, PIC, ESP32).

3. Какое энергопотребление критично? → ищем специализированные ULP-серии (MSP430, STM32L).

4. Нужна ли беспроводная связь? → сразу смотрим на ESP32 или nRF52.

5. Какой бюджет и объем производства? → для хобби берем ESP32 или STM32, для массового продукта считаем цену за чип и смотрим на PIC или 8051.

Грамотный выбор МК — это всегда компромисс между стоимостью, производительностью, энергопотреблением и наличием необходимой периферии.

Модульная организация микроконтроллера — это ключевой принцип его архитектуры, который делает его не просто процессором, а законченной вычислительной системой на одном кристалле (System-on-a-Chip, SoC).

Суть в том, что микроконтроллер состоит из набора функционально законченных блоков (модулей), соединенных внутренними шинами. Понимание этой организации критически важно для его эффективного программирования.

Общая схема модульной организации.

Любой современный МК можно представить следующей структурой (рис. 2.1).

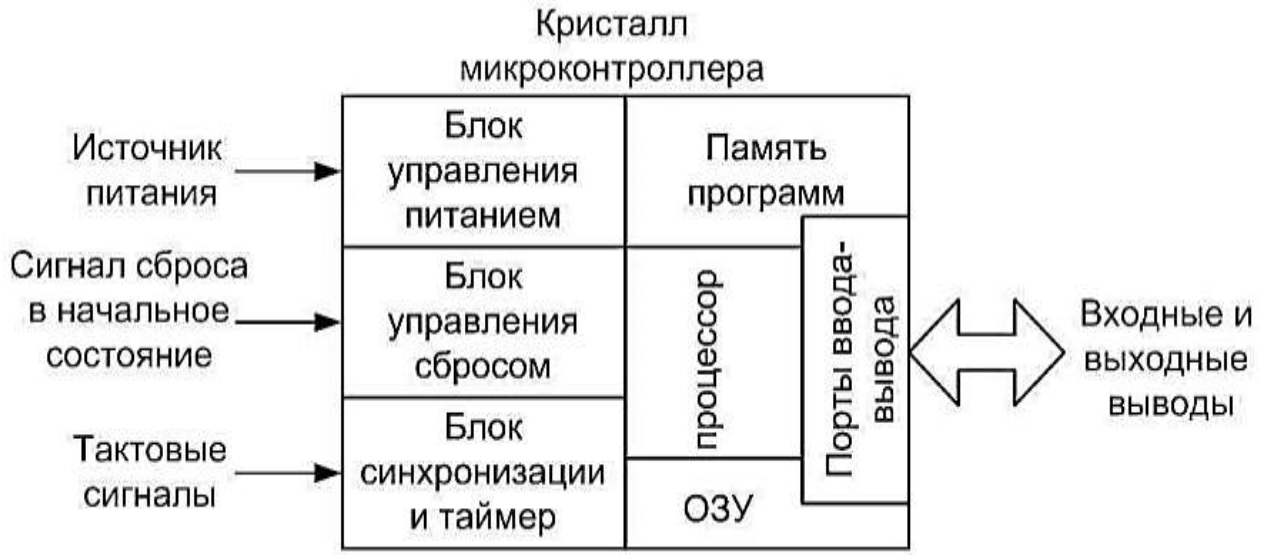


Рис. 2.1. Структура современного МК

Детальное описание ключевых модулей:

1. Вычислительное ядро (CPU — Central Processing Unit):

• функция: «мозг» системы. Выполняет команды программы из памяти;

• что содержит: АЛУ (арифметико-логическое устройство), регистры общего назначения, схему управления;

• важно: ядро может быть разных архитектур (ARM Cortex-M, RISC-V, AVR, PIC и т.д.).

2. Память (Memory):

• память программ (Flash): энергонезависимая память. Здесь хранится прошивка (ваша программа). Сохраняется после отключения питания;

• ОЗУ (RAM — Random Access Memory): энергозависимая память. Здесь хранятся временные данные (переменные, стек, буферы). Стирается при отключении питания;

• ППЗУ (EEPROM): энергонезависимая память для хранения данных, которые могут меняться в процессе работы, но должны сохраняться после выключения (настройки, калибровочные коэффициенты).

3. Системные модули:

• тактовый генератор (Clock Generator): задает темп работы всей системы. Может использовать внутренний RC-генератор (неточный, но дешевый) или внешний кварцевый резонатор (точный);

• схема сброса (Reset Circuit): обеспечивает корректный запуск МК при подаче питания и по команде;

• сторожевой таймер (WDT — Watchdog Timer): специальный таймер, который должен периодически сбрасываться программой. Если программа зависла и не сбросила его, WDT инициирует перезагрузку МК. Критически важен для надежности;

• контроллер прерываний (Interrupt Controller): управляет внешними и внутренними событиями (прерываниями), которые требуют немедленной реакции процессора.

4. Периферийные модули (модули ввода-вывода).

Это «органы чувств» и «инструменты» МК для взаимодействия с внешним миром:

• порты ввода-вывода (GPIO — General Purpose Input/Output): универсальные выводы, которые можно программно настроить на вход (читать состояние кнопки) или выход (подать напряжение на светодиод);

• таймеры/счетчики (Timers/Counters):

– счетчик импульсов: подсчет внешних событий (например, оборотов двигателя);

– таймер: формирование точных временных интервалов;

– генерация ШИМ-сигнала (PWM): управление яркостью светодиодов, скоростью моторов;

• аналого-цифровой преобразователь (АЦП — ADC): позволяет измерять аналоговые величины (напряжение с датчика температуры, освещенности) и преобразовывать их в цифровой код для обработки процессором;

• цифроаналоговый преобразователь (ЦАП — DAC): выполняет обратную задачу — преобразует цифровой код в аналоговое напряжение (например, для генерации звука);

• интерфейсы связи: модули для обмена данными с другими устройствами:

– UART (USART): асинхронный последовательный интерфейс (общение между МК и компьютером, GPS-модулем);

– I^2C (TWI): двухпроводной интерфейс для связи с множеством датчиков и периферийных микросхем;

– SPI: высокоскоростной последовательный интерфейс для связи с дисплеями, картами памяти, ADC;

– USB: для подключения к компьютеру в качестве устройства;

– CAN: промышленный интерфейс для автомобилей и автоматизации.

5. Системная шина (System Bus).

Это «магистраль», которая соединяет все вышеперечисленные модули с ядром и друг с другом. Она состоит из шины адреса, шины данных и шины управления.

Принцип работы с модулями.

Программист взаимодействует с периферийными модулями через регистры специальных функций (SFR — Special Function Registers). Каждому модулю выделен свой набор регистров:

- регистр управления: настройка режима работы модуля (например, включить таймер, выбрать источник тактирования);
- регистр данных: чтение или запись данных (например, чтение результата преобразования АЦП, отправка байта по UART);
- регистр статуса: проверка состояния модуля (например, «данные готовы», «передача завершена»).

Пример: чтобы измерить напряжение, программа:

1) записывает в регистр управления АЦП команду на запуск преобразования;

2) периодически проверяет регистр статуса АЦП, ожидая флага «преобразование завершено».

3) читает результат из регистра данных АЦП.

Модульная организация — это сила микроконтроллера. Она позволяет:

1) экономить пространство и энергопотребление, интегрируя все на одном кристалле;

2) упрощать разработку: программист может использовать готовые модули, не проектируя схемы на дискретных компонентах;

3) создавать гибкие решения: выбирая МК с нужным набором модулей, можно оптимально решить задачу без переплаты за ненужную функциональность.

Понимание этой организации — первый шаг к эффективному программированию МК не на уровне «зажечь светодиод», а на уровне управления аппаратными ресурсами.

2.2. СТРУКТУРА ПРОЦЕССОРНОГО ЯДРА МК. СИСТЕМА КОМАНД МК. ПАМЯТЬ МК

Структура процессорного ядра микроконтроллера (МК) — это архитектура его «мозга», которая определяет, как он выполняет команды, работает с данными и управляет периферией.

В отличие от процессоров общего назначения (CPU в ПК), ядра МК максимально оптимизированы для низкого энергопотребления, детерминированного времени отклика и эффективной работы с периферией.

Ключевые компоненты структуры ядра.

Структуру любого процессорного ядра МК можно представить в виде блок-схемы, которая показывает поток данных и команд (рис. 2.2).

Детальное описание ключевых компонентов:

1. Арифметико-логическое устройство (АЛУ — Arithmetic Logic Unit):

• функция: это «вычислительный цех» ядра. Выполняет все арифметические (+, -, *, /) и логические (И, ИЛИ, НЕ, XOR) операции, а также операции сдвига и сравнения;

• особенность в МК: часто оптимизировано для работы с целыми числами (integer). Наличие блока вычислений с плавающей точкой (FPU) — это опция, характерная для более мощных 32-битных ядер (например, Cortex-M4F/M7).

2. Регистровый файл (Register File):

• функция: набор сверхбыстрой памяти *внутри* процессора для временного хранения данных, с которыми он работает *прямо сейчас*;

• важность: скорость доступа к регистрам намного выше, чем к ОЗУ. Эффективное использование регистров — ключ к производительности;

• типы регистров:

– регистры общего назначения (R0-R12): для хранения данных и адресов;

– специальные регистры:

○ счетчик команд (PC — Program Counter): хранит адрес *следующей* команды для выполнения;

○ регистр команд (IR — Instruction Register): хранит *текущую* выполняемую команду;

○ регистр стека (SP — Stack Pointer): указывает на вершину стека в ОЗУ. Стек используется для временного хранения данных и адресов возврата из подпрограмм;

○ регистр состояния (SR — Status Register): содержит флаги (flags), которые устанавливаются после операций АЛУ:

✓ Z (Zero): флаг нуля (устанавливается в 1, если результат операции равен нулю);

✓ C (Carry): флаг переноса (устанавливается при переполнении разрядной сетки);

✓ N (Negative): флаг отрицательного числа;

✓ V (Overflow): флаг переполнения для знаковых операций.

3. Устройство управления (УУ — Control Unit).

Функция: «дирижер» оркестра. Координирует работу всех блоков ядра. Его главная задача — декодировать команду из регистра команд (IR) и сгенерировать управляющие сигналы для АЛУ, регистров и шин, чтобы эта команда была выполнена.

4. Шины (Buses):

• функция: «автодороги» для передачи данных и команд внутри ядра и между ядром и памятью;

• типы шин:

– шина данных: передает собственно данные;

– шина адреса: указывает, *откуда* или *куда* передаются данные:

– шина управления: передает управляющие сигналы от УУ (чтение, запись, выбор модуля).

Архитектуры ядра: гарвардская, фоннеймановская.

Это фундаментальное различие (табл. 2.5, 2.6) в организации памяти, которое напрямую влияет на структуру ядра.

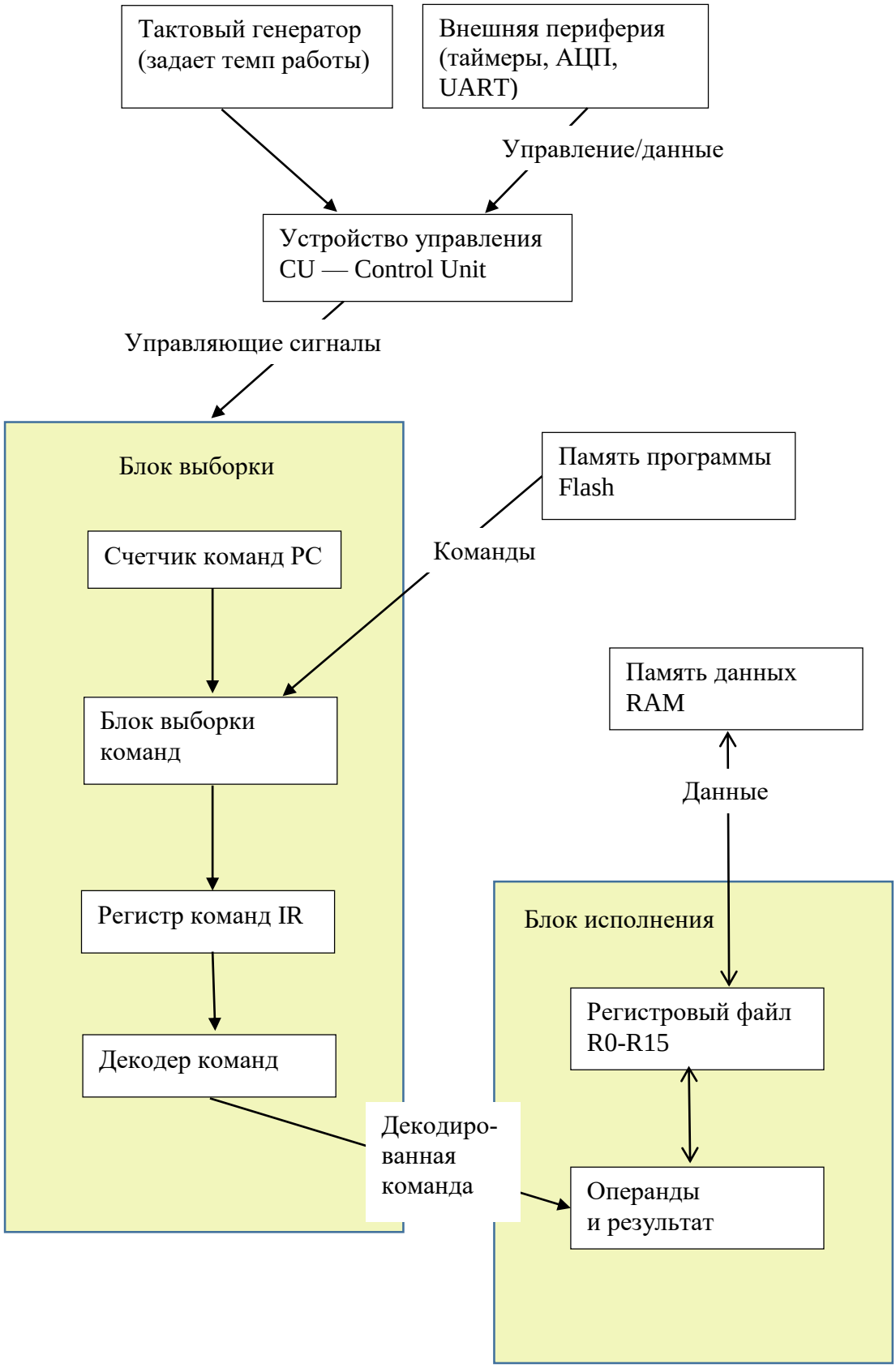


Рис. 2.2. Блок-схема потока данных и команд в МК

Таблица 2.5

Различие архитектур

Параметр	Гарвардская архитектура	Фоннеймановская архитектура
Принцип	Раздельные шины и память для команд и данных	Общая шина и память для команд и данных
Преимущества	Выше производительность: возможность параллельной выборки команды и данных (за один такт)	Проще и гибче: проще в проектировании, универсальнее
Недостатки	Сложнее в проектировании	«Бутылочное горлышко»: команды и данные передаются по одной шине по очереди
Использование	Подавляющее большинство современных МК (ARM Cortex-M, AVR, PIC)	Некоторые МК (например, на базе ядра 8051), процессоры общего назначения (x86)

Современный гибридный подход: многие МК (например, с ядром ARM Cortex-M) используют гарвардскую архитектуру внутри ядра (раздельные кэши для команд и данных), но единое адресное пространство памяти (как у Фоннеймановской архитектуры) для удобства программирования.

Набор команд: RISC vs CISC.

Современный тренд: RISC-архитектуры абсолютно доминируют в мире МК из-за их эффективности, низкого энергопотребления и предсказуемости, что критически важно для встраиваемых систем.

Таблица 2.6

Еще одно ключевое отличие ядер МК

Параметр	RISC (Reduced Instruction Set Computer)	CISC (Complex Instruction Set Computer)
Философия	Небольшой набор простых команд, которые выполняются за один такт	Большой набор сложных команд, которые могут выполнять несколько операций и выполняются за несколько тактов
Плюсы	Высокая и предсказуемая производительность, проще конвейеризация, меньшее энергопотребление	Плотный код (программа занимает меньше места в памяти)
Минусы	Код менее плотный	Сложное устройство, variable время выполнения команд
Примеры ядер	ARM Cortex-M, AVR, RISC-V (подавляющее большинство современных МК)	8051, PIC18 (устаревающие архитектуры)

Особенности структуры ядра МК по сравнению с CPU ПК:

1. Тактовая частота: на порядки ниже (десятки-сотни МГц vs гигагерцы в ПК).

2. Энергопотребление: минимальное (милли- и микроватты в режиме сна).

3. Периферия: интегрирована прямо в кристалл и управляется через регистры.

4. Детерминизм: предсказуемое время выполнения операций, что критично для систем реального времени.

5. Отсутствие сложных подсистем: нет виртуальной памяти, сложного многозадачного ОС (часто используется RTOS) или многоуровневого кэша.

Понимание структуры ядра позволяет программисту не просто писать код, а понимать, как этот код выполняется на аппаратном уровне, что является ключом к созданию эффективных и надежных встраиваемых систем.

Система команд микроконтроллера (МК) — это набор всех команд (инструкций), которые его процессорное ядро может понять и выполнить. Это фундаментальный интерфейс между программистом (или компилятором) и аппаратным обеспечением.

В отличие от процессоров ПК, система команд МК существенно зависит от его аппаратной структуры (регистров, периферии).

Классификация команд по функциям.

Большинство систем команд МК можно разделить на несколько основных групп.

1. Команды пересылки данных (Data Transfer).

Самая часто используемая группа. Перемещают данные между регистрами, памятью и периферией:

- MOV (Move): копирует данные из одного регистра в другой;
- LD/LDR (Load): загружает данные из памяти в регистр;
- ST/STR (Store): сохраняет данные из регистра в память;
- IN (Input): читает данные из порта ввода-вывода в регистр;
- OUT (Output): записывает данные из регистра в порт ввода-вывода;
- PUSH/POP: работа со стеком (положить значение на стек, взять значение со стека).

2. Арифметические команды (Arithmetic).

Выполняют математические операции:

- ADD/ADC (Add / Add with Carry): сложение;
- SUB/SBC (Subtract / Subtract with Carry): вычитание;

• INC/DEC (Increment/Decrement): увеличить/уменьшить значение на 1;

• MUL (Multiply): умножение (есть не у всех 8-битных МК);

• DIV (Divide): деление (часто только у 32-битных МК).

3. Логические команды (Logical).

Выполняют побитовые операции. Крайне полезны для управления отдельными битами в регистрах:

• AND: логическое И. Используется для маскирования (обнуления) битов;

Пример: AND R1, R1, 0xFE — обнулить младший бит регистра R1;

• OR: логическое ИЛИ. Используется для установки битов в 1;

• XOR: исключающее ИЛИ. Используется для инверсии битов;

• NOT/COM (Complement): инверсия (логическое НЕ);

• SHR/SHL/ROR/ROL (Shift/Rotate): команды сдвига и циклического сдвига битов.

4. Команды перехода и управления программой (Control Transfer).

Изменяют порядок выполнения программы:

• JMP/BR (Jump/Branch): безусловный переход к другой метке/адресу;

• CALL/RCALL/BL (Call): вызов подпрограммы (функции);

• RET/RETI (Return / Return from Interrupt): возврат из подпрограммы или обработчика прерывания;

• условные переходы: переход выполняется, только если выполняется условие (проверяются флаги в регистре статуса):

– BREQ/BNE (Branch if Equal / Not Equal): переход, если равно / не равно;

– BRCS/BRCC (Branch if Carry Set / Clear): переход, если установлен/сброшен флаг переноса;

– BRLT/BRGE (Branch if Less Than / Greater or Equal): переход, если меньше, больше или равно.

5. Команды управления процессором (Processor Control):

• NOP (No Operation): пустая операция. Используется для создания небольших задержек или выравнивания по времени;

• CLI/SEI (Clear Interrupts / Set Interrupts): запретить/разрешить обработку прерываний;

• SLEEP/IDLE: переход в режим пониженного энергопотребления.

Ключевые особенности системы команд МК:

1. Работа с регистрами: подавляющее большинство команд оперирует с внутренними регистрами процессора, а не напрямую с памятью. Это связано с принципом RISC-архитектуры.

Пример: ADD R0, R1, R2 // Сложить содержимое регистров R1 и R2, результат поместить в R0.

2. Битовые операции (Bit-Banding): многие МК (особенно AVR, PIC) имеют мощные команды для работы с отдельными битами:

• SBI/CBI (Set Bit in I/O, Clear Bit in I/O): установить или сбросить конкретный бит в порту ввода-вывода;

• SBIS/SBIC (Skip if Bit Set, Skip if Bit Cleared): пропустить следующую команду, если бит установлен/сброшен. Это основа для проверки состояний кнопок, флагов.

3. Прямая работа с периферией: команды IN/OUT позволяют напрямую (одной инструкцией) обращаться к регистрам периферии (таймеров, АЦП, UART), что обеспечивает высокую скорость реакции.

4. Компактность кода: система команд сильно оптимизирована под типовые задачи МК (управление, проверка флагов, обработка прерываний). Одна команда часто занимает всего 2 или 4 байта в памяти программ.

Сравнение архитектур: AVR (RISC) vs 8051 (CISC) (табл. 2.7).

Уровни абстракции:

1. Ассемблер (Assembly): программист пишет код, используя мнемонические обозначения команд (MOV, ADD, JMP). Полный контроль над железом, но код сложен и непортабелен.

2. Язык С: программист пишет на языке высокого уровня. Компилятор (например, GCC, IAR, Keil) транслирует конструкции языка С в соответствующие команды ассемблера:

• операция a = b + c; может быть скомпилирована в последовательность: LD R1, b → LD R2, c → ADD R3, R1, R2 → ST a, R3;

• обращение к периферии происходит через обращение к адресам регистров специальных функций (SFR), которые объявлены как переменные.

Таблица 2.7

Сравнение архитектур МК

Параметр	**AVR (RISC-архитектура)**	**8051 (CISC-архитектура)**
Размер команды	В основном 16-бит (2 байта). Фиксированная длина	Разная длина (1–3 байта)
Регистры общего назначения (РОН)	Много (32 шт.). Все РОН равнозначны и быстро доступны	Мало (8 шт.). Регистры имеют специализацию (аккумулятор, регистры данных)
Обращение к памяти	Отдельные команды LD/ST. Доступ медленнее	Многие команды могут работать напрямую с памятью
Пример кода	• LDI R16, 0x55 // Загрузить константу; • OUT PORTB, R16 // Вывести в порт	• MOV A, #0x55 // Загрузить в аккумулятор; • MOV P1, A // Вывести из аккумулятора в порт
Эффективность	Высокая производительность, предсказуемое время выполнения	Менее эффективная конвейеризация из-за разной длины команд

Большинство современных проектов пишутся на языке С, как это делается в Arduino и STM32CubeIDE. Ассемблер используется редко — для критичных ко времени участков кода или для написания низкоуровневых библиотек.

Система команд МК — это минималистичный и эффективный набор инструкций:

- жестко завязанный на аппаратную архитектуру (регистры, память, периферию);
- оптимизированный для задач управления, работы с битами и энергоэффективности.

Понимание системы команд (даже если вы пишете на С) позволяет создавать более эффективный и оптимизированный код, так как вы начинаете понимать, как высокоуровневые конструкции преобразуются в машинные инструкции.

Память микроконтроллера (МК) — это иерархическая структура, от которой напрямую зависят его возможности, производительность и способ хранения программы и данных. В отличие от ПК вся память находится на одном кристалле с ядром процессора.

Классификация памяти МК.

Память в МК можно классифицировать по нескольким ключевым признакам:

1. По назначению и функциональности (табл. 2.8).

Таблица 2.8

Классификация памяти по назначению и функциональности

Тип памяти	Назначение	Аналог в ПК	Важные характеристики
Flash-память	Хранение программы (прошивки) и констант	Жесткий диск (SSD) с ОС и программами	Энергонезависимая. Перезаписываемая (~10k — 100k циклов). Относительно медленная

Тип памяти	Назначение	Аналог в ПК	Важные характеристики
ОЗУ (RAM)	Хранение временных данных: переменных, стека, буферов	Оперативная память (DRAM)	Энергозависимая (данные теряются при отключении питания). Быстрая. Ограниченный объем
EEPROM	Хранение данных, которые должны сохраняться после выключения, но могут меняться в работе (настройки, калибровочные коэффициенты, счетчики)	Маленький дополнительный SSD для пользовательских файлов	Энергонезависимая. Перезаписываемая (~100k — 1M циклов). Медленная при записи
Регистры	Сверхбыстрая память *внутри* ядра процессора для обработки данных	Кэш-память L1 процессора	Энергозависимая. Максимальная скорость. Очень маленький объем (несколько десятков байт)

2. По энергозависимости:

• энергозависимая (Volatile): данные теряются при отключении питания:

– ОЗУ (RAM);

– регистры процессора;

• энергонезависимая (Non-Volatile): данные сохраняются после отключения питания:

– Flash-память;

– EEPROM;

– ROM (Mask ROM — программируется на заводе, не перезаписывается).

3. По возможности записи:

• память только для чтения (Read-Only).

Mask ROM: однократная запись при производстве;

• память для чтения и многократной записи (Read-Write):

– Flash;

– EEPROM;

– RAM.

Детальное описание типов памяти:

1. Flash-память (память программ):

• технология: электрически стираемая и перепрограммируемая память (EEPROM), но стирается не побайтно, а блоками (страницами). Это удешевляет производство;

• особенности доступа: в большинстве МК организована по архитектуре фон Неймана (общее адресное пространство для команд и данных), но с гарвардским доступом (отдельные шины для выборки команд и доступа к данным). Это позволяет ядру параллельно выбирать следующую команду и работать с данными;

• важно: код из Flash не может быть напрямую изменен самой программой. Для самообновления (bootloader) требуется специальная процедура с копированием кода в RAM.

2. ОЗУ (оперативное запоминающее устройство — RAM):

• технология: статическая RAM (SRAM). Не требует регенерации, проще в управлении, но менее плотная и более дорогая, чем динамическая RAM (DRAM) в ПК;

• структура: обычно разделена на секции:

– .data: инициализированные переменные (при старте МК данные копируются из Flash в этот раздел);

– .bss: неинициализированные переменные (обнуляются при старте);

– Heap (куча): динамическая память, выделяемая функциями типа malloc(). Используется редко в МК из-за риска фрагментации и детерминизма;

– Stack (стек): память для хранения локальных переменных функций, адресов возврата и параметров. Растет «вниз».

3. EEPROM (Electrically Erasable Programmable Read-Only Memory):

• ключевое преимущество перед Flash: позволяет перезаписывать отдельные байты без необходимости стирания целого блока;

• недостатки: меньшая плотность, больше ячейка памяти, медленнее запись;

• типичное использование:

```
// Псевдокод для AVR
#include <avr/eeprom.h>
uint8_t settings = eeprom_read_byte((uint8_t*)0x10); // Прочитать байт по адресу 0x10
settings = 5;
eeprom_write_byte((uint8_t*)0x10, settings);  // Записать байт по адресу 0x10
```

4. Регистры специальных функций (SFR — Special Function Registers).

Это не отдельная память, а особый раздел в общем адресном пространстве:

• назначение: обеспечивают доступ к периферии МК. Каждый регистр управляет конкретным модулем (таймером, АЦП, UART и т.д.);

• доступ: обращение к ним выглядит как чтение/запись в память по определенному адресу:

– регистр управления (Control Register): настройка режима работы;

– регистр данных (Data Register): чтение/запись данных;

– регистр статуса (Status Register): проверка состояния (например, «данные получены»).

Организация памяти: модель памяти Harvard vs Princeton (Von Neumann).

Это фундаментальное понятие, определяющее, как процессор обращается к разным типам памяти (табл. 2.9).

Таблица 2.9

Обращение к различным архитектурам

Параметр	**Гарвардская архитектура**	**Фоннеймановская (принстонская) архитектура**
Суть	Раздельные шины и физическое пространство для памяти программ (Flash) и памяти данных (RAM)	Единое адресное пространство и шина для команд и данных
Преимущества	Выше производительность: возможность параллельной выборки команды и данных (за один такт). Более предсказуемое время выполнения	Проще для проектирования и программирования
Недостатки	Более сложная организация	«Бутылочное горлышко» на шине, так как команды и данные передаются по очереди
Использование	Подавляющее большинство современных МК (AVR, PIC, ARM Cortex-M)	Некоторые устаревшие МК (например, Intel 8051)

Современный гибрид: многие МК (например, на ARM Cortex-M) используют гарвардскую архитектуру на уровне ядра (раздельные шины для команд и данных), но единое линейное адресное пространство для удобства программирования. Это называется «модифицированная гарвардская архитектура».

Практические аспекты для программиста (табл. 2.10):

1. Ограниченный объем: главный лимит при разработке под МК. Необходимо оптимизировать код (по размеру Flash) и данные (по объему RAM).

2. Отсутствие виртуальной памяти: если программа пытается обратиться за пределы реальной памяти, возникает сброс (reset) или неопределенное поведение.

3. Важность стека: переполнение стека — одна из самых частых и трудноотлаживаемых ошибок, приводящая к краху системы.

4. Прямой доступ к памяти (DMA): в более продвинутых МК есть контроллер DMA, который позволяет периферии (например, UART, ADC) обмениваться данными с RAM напрямую, без участия ядра, что сильно разгружает процессор.

Таблица 2.10

Сводная таблица характеристик

Параметр	**Flash**	**RAM (SRAM)**	**EEPROM**
Назначение	Код программы, константы	Переменные, стек, буферы	Данные для долговременного хранения
Энергозависимость	Нет	Да	Нет
Скорость доступа	Средняя	Высокая	Низкая (особенно запись)

Параметр	Flash	RAM (SRAM)	EEPROM
Циклы перезаписи	~10 000–100 000	Не ограничено	~100 000–1 000 000
Типичный объем	От 1 КБ до 2 МБ+	От 128 Б до 512 КБ+	От 64 Б до 64 КБ+

Понимание организации памяти МК критически важно для создания эффективных, надежных и отлаживаемых встраиваемых систем. Оно определяет, как будет работать программа, где хранятся данные и как взаимодействовать с аппаратными ресурсами.

2.3. ПОРТЫ ВВОДА-ВЫВОДА

Порты ввода-вывода (I/O Ports) — это один из самых важных и фундаментальных интерфейсов микроконтроллера, его «органы чувств» и «руки» для взаимодействия с внешним миром.

Это физические выводы (пины) микросхемы, которые можно программно настроить либо на вход (чтобы «читать» внешний сигнал), либо на выход (чтобы «управлять» внешним устройством).

Назначение и функции.

Порты ввода-вывода позволяют МК:

- читать данные от внешних устройств: состояние кнопок, показания датчиков (например, цифровой выход датчика температуры), сигналы с других микросхем;
- передавать данные на внешние устройства: управление светодиодами, дисплеями, моторчиками, передача сигналов другим цифровым устройствам;
- реализовывать аппаратные интерфейсы: многие специализированные интерфейсы, такие как UART, I2C, SPI, реализуются на осно-

ве универсальных портов ввода-вывода (или имеют для этого выделенные пины).

Базовая структура одного вывода (пина).

Каждый физический вывод микроконтроллера имеет внутри связанную с ним схему управления. Упрощенно ее можно представить так:

- данные: цифровой сигнал (0 или 1, Low или High);
- управление: направление (Input/Output), подтяжка, режим.

Как это работает:

1. Данные от ядра процессора поступают по внутренней шине.
2. Логика управления на основе регистров определяет, что делать с этими данными.
3. Выходной каскад усиливает сигнал для управления внешней нагрузкой.
4. Входной буфер передает состояние внешнего сигнала ядру процессора.

Регистры управления портами.

Программист управляет портами через запись в специальные регистры в адресном пространстве памяти. Основных регистров обычно три:

1. Регистр направления данных (DDRx — Data Direction Register).

Определяет, является ли пин входом или выходом.

Бит = 0: соответствующий пин сконфигурирован как ВХОД.

Бит = 1: соответствующий пин сконфигурирован как ВЫХОД.

Пример для AVR: *DDRB = 0b00000001; // Сделать PB0 выходом, а остальные пины порта B — входами.*

2. Регистр порта (PORTx):

- для выхода: устанавливает уровень сигнала на пине.

Бит = 0: установить низкий логический уровень (0V, GND).

Бит = 1: установить высокий логический уровень (например, 3.3V или 5V).

Пример: *PORTB = 0b00000001; // Установить на PB0 высокий уровень (если он настроен как выход).*

• для входа: управляет внутренней подтяжкой.

Бит = 1: включить встроенный подтягивающий резистор (pull-up resistor). Это нужно, чтобы пин имел четкий логический уровень, когда на нем нет внешнего сигнала (например, кнопка не нажата). Предотвращает «плавание» уровня.

Бит = 0: отключить подтяжку.

Пример: *PORTB = 0b00000010; // Включить подтяжку на пине PB1 (если он настроен как вход).*

3. Регистр ввода (PINx — Port Input Pin Register).

Только для чтения. Содержит текущее логическое состояние пина, сконфигурированного как вход. Чтение этого регистра позволяет узнать, какой сигнал сейчас приходит на пин извне.

Пример для AVR: *if (PINB & (1 << PB3)) { ... } // Проверить, установлен ли высокий уровень на пине PB3.*

Дополнительные функции и настройки.

Современные МК часто имеют для портов прогрессивные функции:

• альтернативные функции (Alternate Function): большинство пинов могут работать не только как простой GPIO, но и как специализированный интерфейс.

Пример: Пин PB6 может быть:

– простым GPIO (по умолчанию);

– выводом таймера (ШИМ-сигнал);

– выводом аппаратного UART (приемник RX);

– выводом интерфейса I2C.

Выбор функции осуществляется через регистр альтернативных функций;

• типы выходных каскадов (Output Driver modes):

– Push-Pull (двухтактный): стандартный режим. Пин может четко выдавать и «0» (подключен к GND), и «1» (подключен к VCC);

– Open-Drain (открытый сток) / Open-Collector (открытый коллектор): пин может только «отключаться» (переходить в высокоимпедансное состояние — Z) или «притягивать к 0». Для получения «1»

требуется внешний подтягивающий резистор. Критически важен для шин, где несколько устройств могут быть ведущими (например, I2C);

• скорость нарастания сигнала (Slew Rate Control): позволяет ограничить скорость переключения выхода. Это снижает уровень электромагнитных помех (EMI).

Практические примеры кода (на псевдо-C для AVR).

Пример 1. Мигание светодиодом (пин как выход):

```
// Светодиод подключен к пину PB0 через резистор к GND
#include <avr/io.h>

int main(void) {
   DDRB |= (1 << DDB0);  // Сконфигурировать PB0 как выход (бит DDB0 в регистре DDRB = 1)

   while (1) {
      PORTB |= (1 << PORTB0);   // Зажечь светодиод (HIGH)
      _delay_ms(500);           // Ждем 500 мс
      PORTB &= ~(1 << PORTB0);  // Погасить светодиод (LOW)
      _delay_ms(500);           // Ждем 500 мс
      // Можно использовать XOR для переключения: PORTB ^= (1 << PORTB0);
   }
}
```

Пример 2. Опрос кнопки (пин как вход с подтяжкой):

```
// Кнопка подключена между пином PD2 и GND
#include <avr/io.h>

int main(void) {
   DDRD &= ~(1 << DDD2);   // Сконфигурировать PD2 как вход (бит DDD2 в регистре DDRD = 0)
   PORTD |= (1 << PORTD2); // Включить внутреннюю подтяжку на PD2 к VCC
```

```
    while (1) {
        if (!(PIND & (1 << PIND2))) { // Если на PD2 низкий уровень (кнопка нажата и замкнула пин на GND)
            // Выполнить действие (например, включить светодиод)
            // Кнопка нажата
        } else {
            // Кнопка отпущена
        }
    }
}
```

Порты ввода-вывода — это основа взаимодействия МК с окружающим миром. Понимание их устройства и управления через регистры (DDRx, PORTx, PINx) является первым и самым важным шагом в освоении программирования микроконтроллеров. Дальнейшее изучение периферии (таймеров, АЦП, UART) всегда опирается на эти базовые принципы.

2.4. ТАЙМЕРЫ, МОДУЛЬ ПРЕРЫВАНИЙ МК

Это аппаратные модули внутри МК, которые могут **считать** тактовые импульсы. В зависимости от источника импульсов они работают в одном из двух режимов:

1) как таймер: считают импульсы внутреннего тактового генератора (системной частоты). Используются для измерения промежутков времени;

2) как счетчик: считают импульсы от внешнего источника, подаваемого на специальный вывод МК. Используются для подсчета внешних событий (например, оборотов вала, количества импульсов).

Ключевые компоненты таймера:

1. Регистр счета (TCNT — Timer / Counter Register):

• это собственно «счетчик», значение которого увеличивается (или уменьшается) с каждым пришедшим импульсом;

• разрядность регистра определяет максимальное число, до которого можно досчитать (8 бит = 255, 16 бит = 65 535).

2. Регистры сравнения (OCR — Output Compare Register):

• в них программа записывает некое опорное значение;

• аппаратура таймера постоянно сравнивает значение регистра счета (TCNT) с этим опорным значением;

• при совпадении генерируется событие сравнения, которое можно использовать для генерации прерывания или управления выводом (см. ШИМ).

3. Регистры настройки (TCCR — Timer / Counter Control Register).

Главный регистр управления таймером. Им задаются:

• режим работы (обычный, CTC, ШИМ);

• источник тактирования (предделитель — prescaler): позволяет «замедлить» таймер, деля системную частоту (1, 8, 64, 256, 1024);

• действие по совпадению (например, сброс счетчика или изменение вывода).

Основные режимы работы таймера:

• Normal Mode (обычный режим): счетчик считает от 0 до максимума (например, 255 для 8-битного), переполняется и начинает сначала. По переполнению выставляется флаг;

• CTC Mode (Clear Timer on Compare): счетчик считает от 0 до значения в регистре сравнения (OCR), затем сбрасывается в 0. Идеален для генерации точных временных интервалов;

• Fast PWM Mode: счетчик считает от 0 до максимума и сбрасывается. Используется для генерации ШИМ-сигнала (широтно-импульсной модуляции) для управления яркостью светодиодов, скоростью моторов;

• Phase Correct PWM Mode: счетчик считает от 0 до максимума и обратно до 0. Более медленный, но симметричный ШИМ-сигнал, лучше подходящий для управления моторами.

Модуль прерываний (Interrupt Controller).

Прерывание (Interrupt) — это механизм, который позволяет МК немедленно отреагировать на важное событие, приостановив выполнение текущей программы.

Событие → Прерывание → Обработка → Возврат.

Типы прерываний:

1. Аппаратные прерывания (External Interrupts): вызываются изменением уровня напряжения на определенных выводах МК (INT0, INT1 и т.д.).

Пример: Нажатие кнопки, приход данных от внешнего датчика.

2. Внутренние прерывания (Internal Interrupts): генерируются внутренними модулями МК.

Примеры:

- прерывание по переполнению таймера: таймер досчитал до максимума;
- прерывание по совпадению таймера (Compare Match): значение счетчика совпало со значением в регистре сравнения;
- прерывание от АЦП: аналого-цифровое преобразование завершено;
- прерывание от UART: принят новый байт или готовность к передаче.

Механизм обработки прерывания:

1. Запрос на прерывание (IRQ): событие от таймера или другого источника устанавливает флаг прерывания.

2. Проверка маски: процессор проверяет, разрешено ли это прерывание глобально (флаг I в регистре статуса) и локально (бит разрешения в регистре управления прерываниями).

3. Сохранение контекста: если прерывание разрешено, процессор завершает текущую инструкцию, сохраняет адрес возврата и регистры в стек.

4. Переход к обработчику: процессор ищет в таблице векторов прерываний адрес программы-обработчика для данного события и переходит к нему.

5. Выполнение обработчика (ISR): выполняется код, написанный программистом для реакции на событие. Код должен быть коротким и быстрым.

6. Возврат (RETI): по команде RETI процессор восстанавливает контекст из стека и возвращается к выполнению прерванной программы.

Связь таймеров и прерываний (практический пример) (табл. 2.11).

Таблица 2.11

Сравнение реакции программы на наличие/отсутствие таймеров и прерываний

Без таймеров и прерываний	С таймерами и прерываниями
Программа работает последовательно, «залипая» в циклах delay()	Программа реагирует на события мгновенно и асинхронно
Невозможно одновременно выполнять задачу и отслеживать время	МК может выполнять несколько задач «параллельно» (на самом деле псевдопараллельно)
Процессор простаивает в циклах ожидания	Процессор полностью utilized, выполняя полезную работу в главном цикле
Реакция на события медленная (пока не дойдет до проверки условия в цикле)	Реакция на события детерминирована и быстра (не зависит от кода в главном цикле)

Самая частая связка — использование таймера для генерации точных временных интервалов с помощью прерываний.

Задача: мигать светодиодом раз в секунду, не используя функцию delay(), которая блокирует выполнение всего кода.

Реализация на AVR:

1. Настраиваем таймер в режиме CTC. Подбираем предделитель и значение регистра сравнения (OCR) так, чтобы прерывание возникало ровно раз в секунду.

2. Разрешаем прерывание по совпадению (Compare Match) для этого таймера.

3. Пишем обработчик прерывания (ISR): в этой функции мы будем переключать состояние светодиода.

4. Главный цикл main() при этом полностью свободен и может выполнять другую работу (опрашивать кнопки, вычислять данные):

```
#include <avr/io.h>
#include <avr/interrupt.h> // Важная библиотека для работы с прерываниями

// Обработчик прерывания по совпадению таймера 1
ISR(TIMER1_COMPA_vect) {
  PORTB ^= (1 << PORTB0); // Инвертировать состояние пина PB0 (к которому подключен светодиод)
}

int main(void) {
  // 1. Настройка пина со светодиодом как выхода
  DDRB |= (1 << DDB0);

  // 2. Настройка таймера 1 в режиме CTC
  TCCR1B |= (1 << WGM12); // Режим CTC (сброс по совпадению с OCR1A)
  OCR1A = 15624;        // Установить значение для сравнения.
                        // (16 000 000 Гц / 1024 предделитель) / 1 Гц = 15 625 (считаем от 0)
  TIMSK1 |= (1 << OCIE1A); // Разрешить прерывание по совпадению с OCR1A
```

```
    // 3. Установить предделитель 1024 и запустить таймер
    TCCR1B |= (1 << CS12) | (1 << CS10); // Предделитель = 1024

    // 4. Глобально разрешить прерывания
    sei();

    // 5. Бесконечный цикл может делать что угодно, светодиод будет мигать «сам»
    while (1) {
      // Здесь можно опрашивать кнопки, общаться по UART и т.д.
      // Работа не прерывается!
    }
}
```

Вывод: использование таймеров и прерываний — это признак профессионального подхода к программированию МК, позволяющий создавать эффективные, отзывчивые и сложные многозадачные системы.

2.5. КОНЕЧНЫЕ АВТОМАТЫ ТИПА UART, SPI, I2C, USB, CAN ДЛЯ РАЗГРУЗКИ ЦЕНТРАЛЬНОГО ПРОЦЕССОРА

Конечные автоматы (Finite State Machines, FSM) — это фундаментальная и чрезвычайно мощная математическая модель и метод программирования, идеально подходящий для микроконтроллеров. Они позволяют структурировать программу, сделать ее понятной, предсказуемой и легко модифицируемой.

Представьте себе умный светофор. Он не может быть одновременно красным, желтым и зеленым. Он находится в одном из конкретных состояний: «горит красный», «горит зеленый», «горит желтый». Переход между этими состояниями происходит по четким правилам (например, «через 60 секунд после красного перейти на зеленый»).

Конечный автомат — это и есть такая модель поведения системы, характеризуемая:

1) конечным набором состояний, в которых она может находиться;

2) переходами между этими состояниями;

3) событиями (условиями), которые инициируют переходы;

4) действиями, которые могут выполняться при входе в состояние, выходе из него или при переходе из одного состояния в другое.

Использование в программировании МК.

Без FSM код для сложного поведения быстро превращается в лапшу из if-else и switch-case, в которой трудно разобраться и которую легко сломать.

Проблемы «плоского» кода:

- обработка кнопок с антидребезгом и разными типами нажатий;
- управление сложными последовательностями (например, алгоритм работы стиральной машины);
- реализация протоколов связи (например, парсинг входящей команды по UART);
- управление двигателями с несколькими режимами работы.

Преимущества конечного автомата:

- наглядность: логику работы системы можно отобразить в виде понятной диаграммы;
- структурированность: код становится чистым, модульным и легким для понимания;
- предсказуемость: поведение системы четко определено для каждого состояния;
- тестируемость: легко проверить все возможные переходы между состояниями;
- надежность: исключаются конфликты состояний (например, мотор не получит команду «включиться» и «выключиться» одновременно).

Компоненты конечного автомата:

1. Состояния (States): это возможные режимы работы системы. Каждое состояние уникально.

Примеры для МК: *STATE_IDLE, STATE_RUNNING, STATE_ERROR, STATE_BUTTON_PRESSED.*

2. Переходы (Transitions): это «стрелки», ведущие из одного состояния в другое. Они срабатывают при выполнении определенного условия (события).

3. События (Events) / условия (Conditions): это триггеры для переходов. Они могут быть:

- внешними (нажатие кнопки, приход данных по UART, сигнал от таймера);
- внутренними (значение переменной достигло порога, завершилось измерение АЦП).

4. Действия (Actions): операции, выполняемые в момент:

- входа в состояние (Entry Action): например, при входе в STATE_RUNNING включить мотор и запустить таймер;
- выхода из состояния (Exit Action): например, при выходе из STATE_RUNNING выключить мотор;
- во время пребывания в состоянии (Do Action): непрерывно выполняемая работа, пока автомат находится в данном состоянии.

Реализация конечного автомата на языке С для МК.

Существует несколько способов реализации. Самый простой и популярный — это автомат с switch-case.

Шаг 1. Определяем состояния.

Создаем enum (перечисление) для всех возможных состояний:

```
typedef enum {
    STATE_IDLE,          // Ожидание
    STATE_BUTTON_PRESSED, // Кнопка нажата (обработка дребезга)
    STATE_RUNNING,       // Работа
    STATE_STOPPING       // Процесс остановки
} system_state_t;
```

Шаг 2. Создаем переменную для хранения текущего состояния:

system_state_t current_state = STATE_IDLE; *// Начальное состояние*

Шаг 3. Пишем функцию-диспетчер.

Это ядро автомата — бесконечный цикл (switch или if-else), который смотрит на current_state и выполняет код, соответствующий этому состоянию:

```
void system_fsm(void) {
   while(1) { // Главный цикл автомата
      switch (current_state) {
         case STATE_IDLE:
            // 1. Действия внутри состояния
            led_off();

            // 2. Проверка условий для перехода
            if (is_button_pressed()) {
               // 3. Действие перед переходом (опционально)
               debounce_timer_start();
               // 4. Совершить переход
               current_state = STATE_BUTTON_PRESSED;
            }
            break;

         case STATE_BUTTON_PRESSED:
            // Ждем конца таймера антидребезга
            if (debounce_timer_elapsed()) {
               if (is_button_still_pressed()) {
                  // Кнопка действительно нажата, переходим к работе
                  motor_start();
                  current_state = STATE_RUNNING;
               } else {
                  // Это был дребезг, возвращаемся в ожидание
                  current_state = STATE_IDLE;
```

```
            }
        }
        break;

    case STATE_RUNNING:
        // Постоянная работа в этом состоянии
        motor_work();

        // Проверяем условия выхода
        if (is_button_pressed() || is_error_detected()) {
            motor_stop();
            current_state = STATE_STOPPING;
        }
        break;

    case STATE_STOPPING:
        // Ждем полной остановки мотора
        if (motor_is_stopped()) {
            current_state = STATE_IDLE;
        }
        break;
}

// Небольшая задержка или передача управления другим задачам
_delay_ms(10);
}
}
```

Шаг 4. Вызываем диспетчер из main():

```
int main(void) {
    hardware_init(); // Инициализация периферии
    current_state = STATE_IDLE; // Установка начального состояния
```

```
    while(1) {
        system_fsm(); // Запуск конечного автомата
    }
}
```

Визуализация: диаграмма состояний.

Логику этого автомата можно визуализировать, что значительно упрощает проектирование и обсуждение. На рис. 2.3 представлена диаграмма для примера выше.



Рис. 2.3. Диаграмма состояний

Использование конечных автоматов — это признак зрелого подхода к программированию микроконтроллеров. Это лучший способ справиться со сложной, многошаговой логикой, которая должна реагировать на множество асинхронных событий.

Начинайте с рисования диаграммы состояний на бумаге или в программе.

Каждое состояние должно быть четко определено.

Программа становится набором простых кусков кода (кейсов внутри switch), за которые отвечает только одно состояние, а не гигантская запутанная функция.

Этот метод делает код не только работающим, но и читаемым и сопровождаемым, что критически важно в реальных проектах.

Конечные автоматы (FSM) для периферийных интерфейсов — это ключевой механизм, который позволяет разгрузить центральное ядро МК от рутинных задач обмена данными.

Здесь речь идет не о FSM, которые вы программируете в основном коде, а о встроенных аппаратных конечных автоматах (Hardware State Machines), которые реализованы внутри самих периферийных модулей (UART, SPI, I2C и т.д.).

Суть концепции: аппаратные FSM.

Представьте, что у вас есть помощник (периферийный модуль), который знает весь протокол обмена данными. Вы даете ему задание («отправь эти байты»), и он самостоятельно (без вашего участия) выполняет всю последовательность действий: выставляет нужные уровни сигналов, проверяет флаги, обрабатывает ответы.

Этот «помощник» и есть аппаратный конечный автомат. Его состояния — это этапы протокола (старт-бит, бит данных, стоп-бит и т.д.), а переходы между состояниями управляются тактовыми импульсами и условиями протокола.

Результат: ЦПУ лишь инициирует операцию и получает уведомление о ее завершении, оставаясь свободным для выполнения других задач.

Как это работает на примере основных интерфейсов:

1. UART (Universal Asynchronous Receiver-Transmitter).

Задача: преобразовать параллельный байт из ЦПУ в последовательность битов (и наоборот), добавляя стартовые, стоповые биты и проверяя четность.

Аппаратный FSM UART-передатчика (TX):

- Idle: ожидание. Линия в состоянии «1»;
- Start Bit: ЦПУ записывает байт в регистр данных (UDR). Автомат переходит в состояние «старт» и выставляет линию в «0» на 1 такт;
- Data Bits: последовательно, бит за битом, автомат выставляет на линию значение каждого бита байта, начиная с младшего;
- Parity Bit (опционально): вычисляет и выставляет бит четности;
- Stop Bit(s): возвращает линию в состояние «1» на 1 или 2 такта;
- Transfer Complete: устанавливает флаг UDRE (UART Data Register Empty) или TXC (Transfer Complete), генерируя прерывание. ЦПУ может загрузить следующий байт.

Аппаратный FSM UART-приемника (RX):

• работает параллельно и независимо от передатчика;

• постоянно находится в состоянии «ожидание старт-бита». Обнаружив спад линии с «1» на «0», он запускает свой FSM для выборки последующих битов данных, проверки четности и стоп-бита;

• при успешном приеме байта устанавливает флаг RXC (Receive Complete) и помещает данные в регистр UDR, генерируя прерывание.

Разгрузка ЦПУ: ЦПУ не должен вручную переключать ножку и выдерживать точные временные интервалы для каждого бита. Он лишь реагирует на прерывания и пересылает данные между RAM и UART.

2. SPI (Serial Peripheral Interface).

Задача: синхронный обмен данными по принципу «ведущий — ведомый».

Аппаратный FSM SPI-модуля:

• Idle: ожидание;

• Setup: ЦПУ записывает байт в регистр данных SPI (SPDR). Автомат начинает генерировать тактовые импульсы на линии SCK;

• Shift Out: сс каждым тактом на SCK автомат выставляет очередной бит на линии MOSI (Master Out Slave In);

• Shift In: одновременно (с каждым тактом на SCK) автомат считывает очередной бит с линии MISO (Master In Slave Out) и формирует из них байт;

• Transfer Complete: после передачи/приема 8 бит автомат устанавливает флаг SPIF и генерирует прерывание. Принятый байт находится в регистре SPDR.

Разгрузка ЦПУ: ЦПУ не генерирует тактовые импульсы программно и не сдвигает биты. Он лишь инициирует обмен и читает готовый результат.

3. I2C (TWI — Two-Wire Interface).

Задача: связь по двухпроводной шине с несколькими устройствами. Имеет самый сложный протокол, а значит, и аппаратный FSM.

Аппаратный FSM I2C-модуля (например, в AVR):

• автомат имеет десятки состояний, соответствующих каждому этапу протокола: Start condition, SLA+R/W, Address Acknowledge, Data transfer, Stop condition;

• состояние автомата можно прочитать из регистра статуса (TWSR);

• ЦПУ записывает в регистр управления (TWCR) команды, которые переводят автомат в следующее состояние (например, «сформировать условие START»);

• после выполнения каждой команды автомат останавливается и ждет указаний ЦПУ, устанавливая соответствующий флаг статуса.

Разгрузка ЦПУ: ЦПУ не работает с битовым уровнем. Он работает на уровне команд протокола («начать сеанс», «отправить адрес», «прочитать байт», «завершить сеанс»). Вся «грязная» работа по формированию сигналов на линиях SDA и SCL с соблюдением всех таймингов лежит на аппаратном FSM.

4. USB и CAN.

Эти интерфейсы настолько сложны, что их контроллеры часто представляют собой специализированные процессоры (SIE — Serial Interface Engine) со своим ядром и памятью.

USB: аппаратный FSM обрабатывает все этапы: reset, suspend, resume, распознавание пакетов (SYNC, PID, данные, CRC), обработку транзакций. ЦПУ работает только с конечными буферами данных.

CAN: аппаратный FSM отвечает за арбитраж (разрешение конфликтов на шине), бит-стаффинг, расчет и проверку CRC, формирование кадров ошибок. ЦПУ видит только принятые сообщения в почтовых ящиках (mailboxes).

Практическое использование: разгрузка ЦПУ.

Вот как выглядит «разгруженный» код общения по UART с использованием прерываний:

```
// Буферы и переменные для работы с UART через прерывания
#define BUF_SIZE 64
```

```
volatile uint8_t tx_buf[BUF_SIZE];
volatile uint8_t tx_head = 0, tx_tail = 0;

// Обработчик прерывания «Буфер передатчика пуст»
ISR(USART_UDRE_vect) {
  if (tx_head != tx_tail) { // Если в буфере есть данные для отправки
    UDR0 = tx_buf[tx_tail]; // Отправить следующий байт
    tx_tail = (tx_tail + 1) % BUF_SIZE; // Сдвинуть указатель
  } else {
    // Буфер пуст, отключить прерывание по готовности передатчика
    // чтобы оно не генерировалось вхолостую
    UCSR0B &= ~(1 << UDRIE0);
  }
}

// Функция для отправки строки (неблокирующая!)
void uart_send_string(const char *str) {
  // Копирование строки в буфер (с защитой от переполнения)
  // ...
  // Включить прерывание по готовности передатчика
  UCSR0B |= (1 << UDRIE0);
  // Автомат UART начнет самостоятельно забирать данные из буфера
  // и отправлять их, генерируя прерывания после каждого байта.
  // Функция завершается мгновенно, не дожидаясь окончания передачи.
}

int main(void) {
  uart_init();
```

```
    // Разрешить глобальные прерывания
    sei();

    uart_send_string("Hello World!\r\n"); // Отправить данные и сразу работать дальше

    while(1) {
      // ЦПУ свободен делать полезную работу здесь,
      // пока аппаратный модуль UART в фоне отправляет данные.
      process_data();
      check_buttons();
    }
}
```

Аппаратные конечные автоматы в периферийных модулях — это мощнейший инструмент для создания высокоэффективных систем на МК. Они:

1) кардинально разгружают ЦПУ: освобождают ядро от рутинных, требующих точного тайминга операций;

2) повышают надежность: работают строго по протоколу, не подвержены влиянию других участков кода;

3) упрощают программирование: разработчик работает на более высоком уровне абстракции (буферы, сообщения), а не на уровне битов и тактов;

4) обеспечивают параллелизм: позволяют нескольким процессам обмена данными идти одновременно (например, UART принимает данные, а SPI общается с датчиком, в то время как ЦПУ что-то вычисляет).

Понимание этого принципа — ключ к переходу от простых «показывающих» проектов к сложным реальным устройствам.

2.6. МИНИМИЗАЦИЯ ЭНЕРГОПОТРЕБЛЕНИЯ В СИСТЕМАХ С МК. ТАКТОВЫЕ ГЕНЕРАТОРЫ МК

Минимизация энергопотребления — это критически важная задача для любых устройств с автономным питанием (от батареек или аккумуляторов). Современные МК предоставляют мощный арсенал средств для ее решения.

Основной принцип реализует концепцию «спи больше, работай быстрее».

Главное правило:

$$\text{Потребляемая мощность} \approx \text{Тактовая частота} \times (\text{Напряжение питания})^2.$$

Из этой формулы следуют два основных пути экономии:

1) снижать тактовую частоту (и напряжение, если возможно) в периоды простоя;

2) полностью отключать неиспользуемые модули и периферию.

Современные МК не работают постоянно на полной мощности. Их работа — это цикл: короткие всплески активности для выполнения задачи и длительные периоды глубокого сна.

Иерархия энергосбережения: от программных приемов до аппаратного выбора.

Меры по снижению энергопотребления можно выстроить в иерархию — от самых простых к самым сложным и эффективным.

Уровень 1. Программные оптимизации (что делать коду):

• оптимизация алгоритмов: самый эффективный способ — выполнить работу быстрее и уйти в сон. Используйте эффективные алгоритмы и избегайте «тяжелых» операций (деление, вычисление с плавающей точкой на слабых ядрах);

• работа с периферией по прерываниям: не опрашивайте кнопку или датчик в цикле (while). Настройте прерывание, которое разбудит МК только по событию (нажатие, приход данных);

• «ленивая» инициализация: включайте периферию (например, ADC, I2C) только непосредственно перед использованием и сразу же отключайте после;

• управление выводами (GPIO): неочевидный, но важный пункт. Не оставляйте выводы в состоянии «высокоимпеданс» (высокого сопротивления), если к ним ничего не подключено. Это может привести к утечкам тока. Настройте их на выход и установите в логический «0» или «1» либо включите внутреннюю подтяжку.

Уровень 2. Аппаратные настройки МК (как настроить железо).

Это самый широкий и эффективный пласт методов:

• тактирование:

– снижение частоты ядра: если задача не требует высокой производительности (например, опрос датчика раз в минуту), уменьшите частоту системной шины;

– выбор источника тактирования: используйте внутренний RC-генератор (HSI) вместо внешнего кварца (HSE). Он менее точен, но потребляет меньше энергии;

– отключение тактирования неиспользуемых модулей (Clock Gating): у каждого современного МК в регистрах есть биты, позволяющие отключать тактовый сигнал для конкретного периферийного модуля (UART, SPI, Timer). Нет тактирования — нет динамических потерь;

• управление питанием (режимы сна) (табл. 2.12): ключевая функция! Все МК имеют несколько предопределенных режимов энергосбережения. Чем «глубже» сон, тем меньше потребление, но и тем сложнее «проснуться».

Алгоритм работы: выполнить задачу → перевести ненужную периферию в низковольтный режим → перевести ядро в режим Stop → ждать прерывания от таймера или внешнего датчика → проснуться, выполнить задачу, повторить;

• настройка периферии на низкое потребление:

– АЦП (ADC): включайте только на время измерения;

– аналоговые компараторы: отключайте, когда не нужны;

– снижение скорости интерфейсов: работайте на пониженной скорости I2C/SPI, если это позволяет периферия.

Таблица 2.12

Сравнение режимов потребления

Режим	Потребление	Что отключено	Как проснуться
Sleep/Idle	Снижено	Ядро CPU (остановлено). Периферия работает	Любое прерывание от работающей периферии
Stop/ Standby	Сильно снижено (мкА)	Ядро CPU, большая часть периферии, тактирование системы. Состояние ОЗУ сохраняется	Внешнее прерывание (EXTI), некоторые таймеры, RTC
Shutdown/ Standby	Минимально (нА)	Почти все. Часто отключается даже ОЗУ	Только внешнее воздействие, сброс или сигнал от RTC (часов реального времени)

Уровень 3. Аппаратный дизайн системы (что делать с платой):

• выбор МК: с самого начала выбирайте МК из линейки Ultra-Low-Power (ULP). Например, STM32Lxx (ST), MSP430 (TI), nRF52x (Nordic). Они спроектированы для наноамперного потребления в режиме сна;

• стабилизатор питания (LDO vs DC DC): для систем с переменной нагрузкой используйте DC DC преобразователь (Buck converter). Он эффективнее линейного стабилизатора (LDO), особенно когда МК активен;

• отключение питания внешних компонентов: используйте MOSFET-ключи, чтобы полностью отключать питание датчиков, дисплеев и другой периферии, когда они не используются;

• правильная разводка и пассивные компоненты: убедитесь, что нет утечек по дорожкам, используйте качественные конденсаторы.

Рассмотрим на практическом примере датчик температуры на батарейке.

Задача: раз в 10 минут измерять температуру и передавать по радиоканалу.

Стратегия минимизации потребления:

1. Аппарат: выбрать МК с ULP-ядром (например, STM32L0) и низкопотребляющим радиомодулем (например, LoRa).

2. Режим работы:

• 99,9 % времени: МК находится в режиме Stop (потребление ~1 мкА). Его будит только таймер (RTC);

• 0,1 % времени: по прерыванию от таймера МК просыпается;

• включает питание и тактирование для датчика температуры и АЦП;

• выполняет измерение (работает на пониженной частоте);

• включает радиомодуль и передает данные на высокой скорости (чтобы быстрее закончить);

• отключает всю периферию и тактирование;

• возвращается в режим Stop.

3. Итог: среднее потребление системы будет определяться не мощным радиопередатчиком, а мизерным током сна, что позволит батарейке работать годы.

4. Сводка методов представлена в табл. 2.13.

Минимизация энергопотребления — это комплексный подход, требующий внимания на всех этапах:

1) выборе компонентов (ULP-МК);

2) проектировании схемы (управление питанием периферии);

3) написании firmware (активное использование режимов сна, прерываний, отключение неиспользуемых модулей).

Грамотная стратегия «спи больше, работай быстрее» — ключ к созданию автономных устройств, работающих от одного элемента питания годами.

Таблица 2.13

Методы минимизации энергопотребления

Метод	Пример	Эффективность	Сложность
Выбор ULP-МК	STM32L, MSP430	Очень высокая	Низкая
Режимы сна	Stop, Standby	Очень высокая	Средняя
Тактирование	Снижение частоты, HSI	Высокая	Низкая
Управление периферией	Clock Gating, отключение	Высокая	Средняя
Программная оптимизация	Прерывания вместо циклов	Средняя	Низкая
Аппаратный дизайн	DC DC преобразователь, ключи	Высокая	Высокая

Тактовые генераторы МК — это «сердце» микроконтроллера, которое задает ритм работы всей системы на микросхеме. Без тактового сигнала не будет выполняться ни одна команда, не будет работать ни один таймер или интерфейс связи.

Зачем нужен тактовый сигнал.

Представьте себе дирижера в оркестре. Он взмахивает палочкой, задавая темп, и все музыканты играют синхронно. Тактовый генератор — это и есть тот самый дирижер для внутренних блоков МК.

Каждый «такт» (импульс) — это шаг, на котором процессор выполняет элементарную операцию (например, выборку команды из памяти). Чем чаще такты (выше частота), тем быстрее работает МК, но и тем больше он потребляет энергии.

Классификация тактовых генераторов.

Тактовые генераторы в МК можно классифицировать по нескольким признакам:

1. По расположению:

• внутренние генераторы: расположены на кристалле самого МК.

Плюсы: дешевы (не требуют внешних компонентов), запускаются быстро, потребляют мало.

Минусы: низкая точность и стабильность (сильно зависят от температуры и напряжения питания);

• внешние генераторы: расположены вне кристалла МК. МК подключается к ним через выводы.

Плюсы: высокая точность и стабильность.

Минусы: требуют дополнительных компонентов (кварцевый резонатор и керамические конденсаторы), дороже, занимают место на плате, медленнее запускаются.

2. По точности и стабильности.

• низкоточные (RC-генераторы): внутренние генераторы на резисторах и конденсаторах. Их частота «плывет». Точность обычно ±1–2 %;

• высокоточные (кварцевые генераторы): используют пьезоэффект кварцевого кристалла. Очень стабильны и точны. Точность может достигать ±10 ppm (частей на миллион), т.е. ±0,001 %;

3. По частоте.

• низкочастотные (LSE — Low Speed External): обычно 32,768 кГц. Нужны для часов реального времени (RTC), так как эта частота легко делится до 1 Гц (1 импульс в секунду);

• высокочастотные (HSE — High Speed External): от 4 МГц до десятков МГц. Это основные генераторы для работы ядра и периферии;

• высокоскоростные внутренние (HSI — High Speed Internal): обычно на фиксированную частоту (например, 8, 16, 64 МГц).

Основные типы тактовых генераторов в МК.

В табл. 2.14 представлены конкретные типы генераторов, которые можно встретить в datasheet (технической документации) любого современного МК.

Таблица 2.14

Типы генераторов

Аббревиатура	**HSI**	**HSE**	**LSI**	**LSE**
Название	High Speed Internal	High Speed External	Low Speed Internal	Low Speed External
Тип	Внутренний RC	Внешний кварц	Внутренний RC	Внешний кварц
Частота	Фиксированная (напр., 8, 16, 64 МГц)	Высокая (напр., 4–25 МГц)	Низкая (напр., 32–128 кГц)	32,768 кГц
Плюсы	Дешево, быстро запускается	Очень точный и стабильный	Дешево, работает в режиме сна	Точный, малопотребляющий
Минусы	Неточный (~1 %)	Дороже, медленный запуск	Очень неточный (~5 %)	Доп. компоненты
Основное применение	Работа при старте, резервный генератор, когда точность не важна	Основная работа ядра и периферии, USB, Ethernet	Сторожевой таймер (IWDG), часы RTC в простых задачах	Часы реального времени (RTC) для точного времени

Схемы подключения внешних генераторов:

1. Кварцевый резонатор (Crystal Oscillator): наиболее распространенный вариант. Требует подключения кварца и двух нагрузочных конденсаторов (рис. 2.4).

Расчет конденсаторов (C1, C2): их номинал критически важен и обычно указан в datasheet на кварц. Типовые значения — 10–22 пФ.

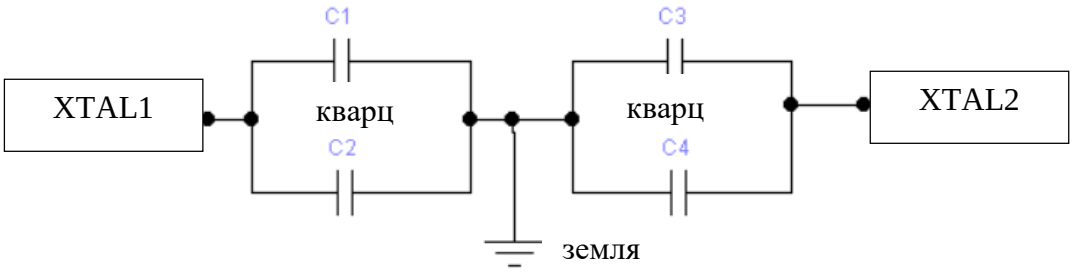


Рис. 2.4. Схема подключения кварцевого резонатора

2. Внешний тактовый генератор (External Clock Generator): готовый генератор, который выдает готовый прямоугольный сигнал. МК в этом случае работает только как приемник (рис. 2.5).

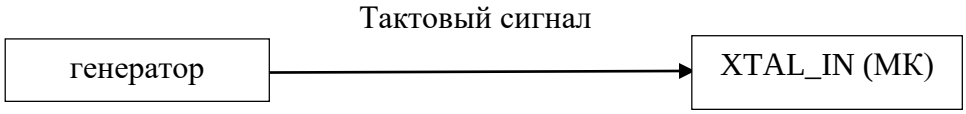


Рис. 2.5. Схема подключения внешнего тактового генератора

Плюсы: максимальная простота для разработчика, высочайшая стабильность.

Минусы: самая высокая стоимость и большее энергопотребление.

Система тактирования (Clock Tree): как это работает внутри.

Это одна из самых сложных, но важных тем. МК не обязан работать от одного генератора. Внутри него существует система тактирования — целое «дерево» мультиплексоров и делителей, которое позволяет гибко распределять тактовые сигналы.

Ключевые элементы системы:

1. Источники (Multiplexer): выбирают, от какого генератора (HSI, HSE, LSI, LSE) будет работать система.

2. Делители частоты (Prescaler): делят частоту источника, чтобы получить нужную частоту для разных частей МК. Например, ядро может работать на 64 МГц, а периферийная шина APB — на 32 МГц.

3. PLL (Phase-Locked Loop — фазовая автоподстройка частоты): очень важный блок! Позволяет умножить частоту входящего сигнала. Например, внешний кварц на 8 МГц можно подать на PLL и умножить в 9 раз, чтобы получить 72 МГц для ядра.

Типичный сценарий старта МК:

1. Сразу после подачи питания запускается HSI (он быстрый и не требует внешних компонентов).

2. МК выполняет начальную инициализацию на HSI.

3. Программа включает HSE и ждет его стабилизации.

4. Программа переключает системную шину на тактирование от HSE (часто через PLL для умножения частоты).

5. HSI можно отключить для экономии энергии.

Управление тактированием для энергосбережения.

Это ключевой навык для программирования автономных устройств:

• снижение частоты: если задача не требует высокой производительности (опрос датчика), частоту системной шины можно снизить через делители;

• отключение тактирования неиспользуемых модулей (Clock Gating): у каждого периферийного модуля (UART, SPI, Timer, ADC) есть свой бит в регистре управления тактированием (RCC). Если модуль не используется, его тактирование нужно отключить — это исключит динамическое энергопотребление модуля;

• выбор источника в режиме сна: в режимах энергосбережения (Sleep, Stop) можно оставить включенным только LSI или LSE для работы сторожевого таймера или RTC, а HSI и HSE отключить.

Понимание системы тактирования — признак перехода от новичка к продвинутому разработчику на МК. Оно позволяет:

• настроить МК на оптимальную производительность (используя PLL и внешний кварц);

• кардинально снизить энергопотребление (отключая ненужные генераторы и периферию);

• обеспечить точную работу интерфейсов, требующих стабильной частоты (USB, UART);

• реализовать часы реального времени с точностью до секунд в сутки.

2.7. АППАРАТНЫЕ СРЕДСТВА ОБЕСПЕЧЕНИЯ НАДЕЖНОЙ РАБОТЫ МК

Аппаратные средства обеспечения надежной работы МК — это встроенные механизмы, которые защищают микроконтроллер от сбоев, внешних помех и программных ошибок, превращая его из «хрупкой» микросхемы в robust-компонент (надежный) для промышленных и критичных применений.

Эти средства можно разделить на несколько ключевых категорий:

1. Защита питания.

Нестабильное питание — одна из самых частых причин сбоев:

• схема ПОС (Power-On Reset): гарантирует, что МК начнет выполнение программы только тогда, когда напряжение питания достигнет безопасного и стабильного уровня. Без POR МК мог бы начать работать при недостаточном напряжении, что привело бы к неопределенному поведению;

• схема ПНК (Power-Down Reset / Brown-Out Detector — BOD): контролирует напряжение питания *во время работы*. Если напряжение проседает (например, из-за скачка нагрузки или садящейся батареи), BOD принудительно сбрасывает МК, предотвращая corruption данных в памяти и выполнение кода с ошибками;

• программируемый регулятор напряжения (PVD): более advanced версия BOD, позволяющая программно задать порог напряжения, при котором будет сгенерировано прерывание (а не сброс). Это дает возможность программе успеть корректно завершить работу (сохранить данные в EEPROM, безопасно остановить двигатель).

2. Защита от зависаний (Watchdog Timers — WDT).

Это один из важнейших механизмов надежности. WDT — это счетчик, который должен периодически сбрасываться программой.

Принцип работы: программа должна «пинать собаку» (watchdog), записывая в специальный регистр до того, как счетчик досчитает до нуля.

Что происходит при сбое: если программа ушла в бесконечный цикл или зависла и перестала сбрасывать WDT, счетчик обнуляется и аппаратура выполняет принудительный сброс МК.

Виды:

• IWDT (Independent WatchDog Timer): работает от своего собственного низкочастотного генератора (LSI). Независим от основного тактирования. Не может быть отключен программно. Максимально надежен;

• WWDT (Window WatchDog Timer): более строгий. Требует сброса не просто до истечения времени, а в строго определенном временном «окне». Защищает от сбоев, когда программа сбрасывает WDT *слишком часто*.

3. Защита памяти и целостности кода:

• контроль четности / ECC (Error-Correcting Code): в продвинутых МК для памяти (RAM и Flash) добавляются дополнительные биты. При чтении проверяется, не испортились ли данные из-за радиации или электромагнитных помех. ECC может не только обнаружить, но и исправить одиночные ошибки;

• защита от считывания Flash-памяти: предотвращает копирование и reverse engineering прошивки;

• Write Protection: блокировка от случайной записи в критичные области памяти (например, bootloader) или в configuration-регистры.

4. Защита периферии и вводов-выводов:

• защита от залипания (Clock Security System — CSS): следит за работой внешнего кварцевого генератора (HSE). Если генератор вы-

ходит из строя, система автоматически переключается на резервный внутренний генератор (HSI) и генерирует прерывание;

• защита выводов (I/O Protection):

– встроенные ограничительные диоды: защита выводов от статического электричества (ESD);

– программируемые подтяжки: внутренние резисторы (pull-up/pull-down), которые можно включить программно, чтобы избежать «висящих в воздухе» входов, чувствительных к помехам;

– функция «аналоговый выключатель»: возможность программно отключить аналоговую часть пина (например, вход АЦП), когда она не используется, чтобы уменьшить потребление и чувствительность к помехам.

5. Контроль и мониторинг работы.

• датчик температуры: внутренний sensor, который позволяет программе отслеживать перегрев кристалла и предпринимать действия (снизить частоту, включить вентилятор, выдать alarm);

• контроль тактовой частоты: система может проверять, что ядро и шины работают на заявленных частотах.

6. Аппаратная поддержка отладки (табл. 2.15).

Хотя это и не прямая «защита», такие интерфейсы, как JTAG и SWD (Serial Wire Debug), позволяют глубоко анализировать работу МК, находить сложные ошибки (например, переполнение стека) и программировать его in-circuit, что косвенно повышает надежность конечного продукта.

Практический пример.

Сценарий: промышленный контроллер двигателя:

1. Включение: POR ensures корректный запуск при подаче питания.

2. Работа: IWDT постоянно сбрасывается главным циклом программы. BOD следит за напряжением в сети.

3. Сбой: возникает мощная электромагнитная помеха от двигателя:

• вариант А: помеха вызывает скачок питания → BOD обнаруживает просадку напряжения и выполняет сброс;

• вариант Б: помеха вызывает сбой в программе → программа перестает сбрасывать IWDT → IWDT выполняет сброс;

• вариант В: помеха повреждает данные в ОЗУ → ECC обнаруживает и исправляет ошибку, генерируя прерывание. Программа-обработчик принимает решение (например, переинициализировать модуль).

4. Восстановление: после любого сброса МК запускается заново с первого шага. Система возвращается в рабочее состояние без вмешательства человека.

Таблица 2.15

Сводная таблица аппаратных средств надежности

Средство	Назначение	Реакция на сбой
POR/BOD (ПОС/ПНК)	Контроль напряжения питания	Сброс МК
Watchdog Timer (WDT)	Контроль зависания программы	Сброс МК
Clock Security System (CSS)	Контроль исправности генератора	Переключение на резервный генератор + прерывание
ECC / Контроль четности	Обнаружение/исправление ошибок в памяти	Прерывание (ошибка может быть исправлена аппаратно)
Защита выводов (ESD)	Защита от статического электричества	Предотвращение физического повреждения вывода
Датчик температуры	Контроль перегрева кристалла	Прерывание

Современные МК — это не просто «голые» процессоры. Это сложные системы, нагруженные аппаратными механизмами самозащиты и самодиагностики. Грамотное использование этих средств (особенно WDT и BOD) является обязательным требованием для создания надежных промышленных, автомобильных и медицинских устройств, которые должны годами работать без сбоев в неидеальных условиях.

2.8. ДОПОЛНИТЕЛЬНЫЕ МОДУЛИ МК: ПОСЛЕДОВАТЕЛЬНОГО ВВОДА-ВЫВОДА, АНАЛОГОВОГО ВВОДА-ВЫВОДА

Дополнительные модули — это то, что превращает микроконтроллер из просто процессора в универсальный инструмент для взаимодействия с внешним миром. Модули последовательного и аналогового ввода-вывода являются самыми востребованными из них.

Модули последовательного ввода-вывода.

Эти модули отвечают за обмен цифровыми данными с другими устройствами (другими МК, датчиками, компьютерами, экранами) по проводам или без них. Данные передаются последовательно, бит за битом:

А. Асинхронные интерфейсы (без общего тактового сигнала), характеристика которых представлена в табл. 2.16.

Таблица 2.16

Асинхронный интерфейс

Интерфейс	**UART (USART)**	**1-Wire**
Суть	Универсальный асинхронный приемопередатчик. Самый простой и распространенный последовательный интерфейс	Однопроводной интерфейс для общения с датчиками

Интерфейс	UART (USART)	1-Wire
Кол-во проводов	2 (TX, RX) + (иногда GND)	1 (Data + GND)
Скорость	Настраиваемая (бодрейт). Стандартные: 9 600, 115 200 и др.	Низкая (~15 кбит/с)
Применение	Связь с компьютером через USB UART преобразователь. Модемы, GPS-модуль. Обмен между двумя МК	Датчики температуры (DS18B20). Электронные ключи (iButton)
Плюсы	Простота, широкое распространение	Крайне малое число проводов, дешевизна
Минусы	Нет механизма проверки доставки (кроме проверки четности)	Низкая скорость, сложный протокол

Б. Синхронные интерфейсы (с общим тактовым сигналом) (табл. 1.17).

Таблица 2.17

Синхронный интефейс

Интерфейс	I²C (TWI)	SPI
Суть	Inter-Integrated Circuit. Двухпроводная шина для подключения множества устройств	Serial Peripheral Interface. Высокоскоростной интерфейс «точка — точка» или «ведущий — ведомые»
Кол-во проводов	2: SDA (данные), SCL (такты)	4: MISO, MOSI, SCK, SS (на каждое устройство свой SS)

Интерфейс	I²C (TWI)	SPI
Топология	Шина. Много устройств на две линии. Адресация по 7/10 бит	«Звезда». Каждое устройство имеет отдельную линию выбора (SS)
Скорость	• Standard: 100 кбит/с; • Fast: 400 кбит/с; • High-speed: 3,4 Мбит/с	Высокая (десятки Мбит/с)
Применение	Датчики (температура, давление, гироскопы). EEPROM-память. Цифровые потенциометры	SD-карты, LCD-дисплеи. FLASH-память. ADC/DAC-преобразователи. RF-модули
Плюсы	Малое число проводов, поддержка множества устройств	Самая высокая скорость среди «проводных» последовательных интерфейсов
Минусы	Средняя скорость, чувствительность к помехам на длинных проводах.	Большое число проводов (4 + n * SS)

Модули аналогового ввода-вывода.

Эти модули работают с непрерывными аналоговыми сигналами (напряжение, ток), преобразуя их в цифровые коды (и наоборот):

А. Модули ввода (аналого-цифровые преобразователи — АЦП).

Измеряет уровень аналогового напряжения на своем входе и преобразует его в цифровое число.

Ключевые параметры:

• разрешение: количество бит в результате (например, 8, 10, 12, 16 бит). Определяет точность. 12-битный АЦП может выдать $2^{12} = 4\,096$ различных значений;

• частота дискретизации: как быстро АЦП может делать преобразования (например, 100 kSPS — тысяч samples per second);

• количество каналов: один АЦП может иметь multiple входных каналов и multiplexer для поочередного их опроса.

Применение:

• считывание показаний с аналоговых датчиков (термисторы, фоторезисторы, потенциометры);

• измерение напряжения аккумулятора;

• сбор любых данных из аналогового мира.

Б. Модули вывода (цифроаналоговые преобразователи — ЦАП).

Выполняет обратную операцию — получает цифровой код и выдает соответствующее ему аналоговое напряжение.

Ключевые параметры: аналогичны АЦП: разрешение, скорость обновления.

Применение:

• генерация звука (цифровой синтезатор, проигрывание WAV-файлов);

• управление скоростью моторов или яркостью светодиодов через аналоговый сигнал (а не ШИМ);

• задание опорных напряжений для других схем.

В. Компараторы напряжения.

Простой, но очень полезный аналоговый модуль. Сравнивает два напряжения (входное и опорное) и цифровым выходом показывает, какое из них больше.

Применение:

• детектор перехода через ноль (zero-crossing detection);

• определение превышения порога (например, перезаряд аккумулятора);

• простые АЦП при малом количестве ресурсов.

Взгляд изнутри.

Производитель интегрирует эти модули прямо в кристалл МК. В datasheet это выглядит как блок-схема.

ЦП (CPU) общается с этими модулями через системную шину путем чтения и записи в их регистры управления:

• чтобы начать преобразование АЦП, программа пишет в регистр управления АЦП;

• чтобы проверить, готов ли результат, она читает регистр статуса АЦП;

• чтобы отправить данные по UART, она записывает байт в регистр данных UART.

Модули последовательного и аналогового ввода-вывода — это «органы чувств» и «средства общения» микроконтроллера:

• без последовательных интерфейсов МК был бы изолированным островком, неспособным общаться с другими цифровыми устройствами;

• без аналоговых модулей МК был бы «слеп» и «нем» по отношению к аналоговому миру, который его окружает.

Именно комбинация мощного ядра и богатого набора этих периферийных модулей делает современные микроконтроллеры такими универсальными и позволяет им решать огромный спектр задач — от простой мигалки до сложной системы управления беспилотником.

2.9. АППАРАТНЫЕ И ПРОГРАММНЫЕ СРЕДСТВА ДЛЯ РАЗРАБОТКИ ПРИЛОЖЕНИЙ НА БАЗЕ МК

Аппаратные и программные средства для разработки под МК — это весь комплекс инструментов, который превращает идею в работающее устройство. Это как мастерская для инженера: есть и станки (аппаратура), и чертежи (ПО).

Аппаратные средства (Hardware Tools).

Это физические устройства, которые подключаются к компьютеру и самому микроконтроллеру:

1. Платы для разработки и прототипирования:

• отладочные (Evaluation Boards / EVK): платы от производителя МК (ST, Microchip, TI и др.). На них установлен конкретный МК, распаяна вся обвязка (кварц, стабилизатор), есть отладочный про-

грамматор и богатая периферия (кнопки, светодиоды, датчики). Идеальны для начала изучения конкретного МК.

Пример: STM32 Nucleo, ESP32 DevKit, Arduino Nano;

• макетные (Breadboard): платы с отверстиями для быстрой сборки прототипов без пайки. Позволяют быстро подключать к МК внешние компоненты (датчики, кнопки, резисторы);

• платы расширения (Shields): специализированные платы, которые устанавливаются поверх отладочной платы (например, Arduino Shield) и добавляют ей функциональность (мотор-драйвер, Ethernet-модуль, LCD-дисплей).

2. Инструменты для программирования и отладки:

• программатор (Programmer): устройство, которое «заливает» скомпилированную прошивку из компьютера во Flash-память МК;

• отладчик (Debugger): устройство, которое позволяет не только загрузить код, но и отлаживать его:

– останавливать выполнение в точках останова (Breakpoints);

– пошагово выполнять программу (Step Into, Step Over);

– просматривать и изменять значения переменных и регистров в реальном времени;

– Often-программатор и отладчик объединены в одном устройстве (Debug Probe);

• популярные интерфейсы и инструменты:

– ST-LINK/V2/V3: для отладки и программирования МК STM32;

– J-Link: универсальный отладчик от SEGGER, поддерживает множество архитектур (ARM, RISC-V);

– DAPLink (CMSIS-DAP): открытый стандарт отладчика, часто используется на платах Nucleo;

– USB UART преобразователь: нужен для обмена данными с МК через последовательный порт (например, CH340, FT232RL).

3. Контрольно-измерительные приборы:

• мультиметр: для измерения напряжения, тока, проверки целостности цепей;

• осциллограф: для визуализации и анализа аналоговых сигналов (например, ШИМ, работа АЦП);

• логический анализатор: для анализа цифровых сигналов и протоколов (I2C, SPI, UART). Показывает временные диаграммы и может декодировать протоколы, показывая, какие байты передавались.

Программные средства (Software Tools).

Это программы, которые работают на компьютере разработчика для написания, компиляции и отладки кода:

1. Интегрированные среды разработки (IDE).

Главный программный комплекс, который объединяет в себе все инструменты:

• Keil MDK (μVision): мощная коммерческая IDE для ARM-архитектуры. Отраслевой стандарт для профессиональной разработки;

• IAR Embedded Workbench: другая популярная коммерческая IDE, известная своими оптимизирующими компиляторами;

• STM32CubeIDE: бесплатная IDE от STMicroelectronics на базе Eclipse. Включает в себя все необходимые инструменты для работы с МК STM32 (компилятор, отладчик, конфигуратор);

• PlatformIO: современная кроссплатформенная IDE, плагин для VS Code. Поддерживает огромное количество платформ (Arduino, ESP32, STM32, Raspberry Pi Pico) и фреймворков. Очень популярна за свою гибкость;

• Arduino IDE: максимально упрощенная среда для начинающих и быстрого прототипирования на платах Arduino и ESP32.

2. Системы автоматизации сборки и компиляторы:

• компилятор: программа, переводящая исходный код (C/C++) в машинный код для МК:

– GCC (GNU Compiler Collection): бесплатный и мощный компилятор. Лежит в основе многих IDE (STM32CubeIDE, PlatformIO);

– ARM Compiler (AC6): проприетарный компилятор с высокой степенью оптимизации, используется в Keil;

• Make/CMake: инструменты для автоматизации процесса сборки проекта.

3. Вспомогательные утилиты и конфигураторы.

• STM32CubeMX / MPLAB Code Configurator: визуальные конфигураторы от ST и Microchip. Позволяют мышкой настроить всю периферию МК (тактовые сигналы, пины, интерфейсы), а затем автоматически сгенерировать готовый код инициализации на С. Экономят огромное количество времени;

• прошивальщики (Flash Tools): утилиты для загрузки прошивки в МК (например, STM32CubeProgrammer);

• терминалы (Serial Monitor): программы для обмена данными с МК через UART (PuTTY, Tera Term, монитор порта в Arduino IDE).

Типовой процесс разработки (как это все используется вместе) (табл. 2.18):

1. Идея и выбор МК: выбираем подходящий микроконтроллер по цене, производительности и периферии.

2. Схема и прототип: рисуем принципиальную схему и собираем прототип на макетной плате.

3. Настройка конфигурации: в STM32CubeMX настраиваем пины, частоту, периферию. Генерируем проект для IDE.

4. Написание кода: в IDE (Keil, STM32CubeIDE, PlatformIO) пишем логику работы программы на C/C++.

5. Сборка (Compile): нажимаем кнопку «Build». Компилятор (GCC) преобразует код в бинарный файл (*.bin, *.hex).

6. Прошивка и отладка: подключаем отладчик (ST-LINK) к плате. Через IDE загружаем прошивку в МК и начинаем отладку: ставим точки останова, смотрим переменные.

7. Анализ проблем: если что-то работает не так, подключаем логический анализатор или осциллограф, чтобы посмотреть, что происходит на линиях связи (I2C, SPI) или с аналоговыми сигналами.

8. Создание финального устройства: после успешного тестирования прототипа создаем печатную плату (ПП) и переносим на нее нашу схему.

Таблица 2.18

Сводная таблица инструментов

Категория	Аппаратные средства	Программные средства
Разработка/ прототип	Отладочные платы, макетные платы	IDE (STM32CubeIDE, PlatformIO)
Программирование	Программатор (ST-LINK, J-Link)	Прошивальщик (STM32CubeProgrammer)
Отладка	Отладчик (тот же ST-LINK, J-Link)	Отладочный интерфейс в IDE
Анализ сигналов	Осциллограф, логический анализатор, мультиметр	Виртуальные инструменты в IDE (логирование)
Конфигурация	–	Визуальные конфигураторы (STM32CubeMX)

Современная разработка под МК — это высокоуровневый процесс, сильно упрощенный благодаря мощным инструментам. Аппаратные средства дают нам возможность «пощупать» железо, а программные — эффективно им управлять.

Правильный подбор этих инструментов и умение ими пользоваться — ключевой навык инженера по встроенным системам, позволяющий быстро и эффективно создавать сложные и надежные устройства.

2.10. ФУНКЦИОНАЛЬНЫЕ БЛОКИ МИКРОКОНТРОЛЛЕРА. КОНФИГУРИРОВАНИЕ МК

Функциональные блоки микроконтроллера — это основные структурные компоненты, интегрированные на едином кристалле, которые превращают его из простого процессора в законченную вычислительную систему (System-on-a-Chip, SoC).

Знание этих блоков — основа для понимания того, как программировать и применять МК (рис. 2.6).

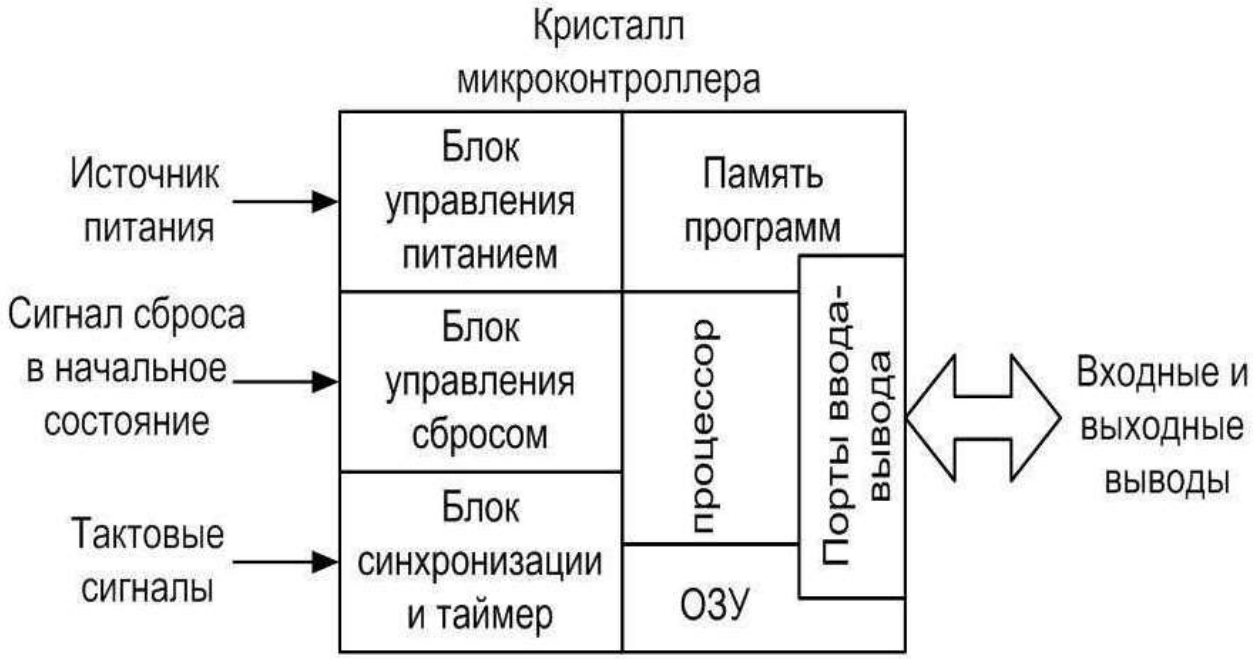


Рис. 2.6. Ключевые функциональные блоки
современного микроконтроллера

Детальное описание каждого блока:

1. Вычислительное ядро (центральный процессор, CPU).

Функция: «мозг» системы. Выполняет команды программы, находящейся в памяти.

Состав:

• АЛУ (арифметико-логическое устройство): выполняет математические и логические операции;

• регистры общего назначения: сверхбыстрая память внутри процессора для работы с данными;

• устройство управления: декодирует команды и управляет работой всех блоков МК.

2. Память (Memory).

Состоит из нескольких типов, каждый для своих задач:

• ПЗУ (Flash-память): энергонезависимая память. Здесь хранится прошивка (ваша программа). Сохраняется после выключения питания;

• ОЗУ (RAM): энергозависимая память. Здесь хранятся временные данные: переменные, стек. Быстрая, но стирается при отключении питания;

• ЭСППЗУ (EEPROM): энергонезависимая память. Здесь хранятся данные, которые могут меняться, но должны сохраняться после выключения (настройки, калибровочные константы).

3. Системные модули:

• тактовый генератор (Clock Generator): задает темп работы всей системы. Часто имеет несколько источников:

– HSI: внутренний высокочастотный RC-генератор (неточный, но быстрый);

– HSE: внешний высокочастотный кварцевый резонатор (точный);

– LSI: внутренний низкочастотный RC-генератор (для сторожевого таймера);

– LSE: внешний низкочастотный кварц на 32.768 кГц (для часов реального времени RTC);

• схемы сброса (Reset Circuits):

– POR (Power-On Reset): гарантирует корректный запуск при подаче питания;

– BOD (Brown-Out Detector): следит за напряжением питания и сбрасывает МК, если оно падает ниже нормы, предотвращая сбои;

• сторожевой таймер (Watchdog Timer — WDT): специальный таймер, который должен периодически сбрасываться программой.

Если программа зависла, WDT инициирует перезагрузку МК. Критически важен для надежности;

• контроллер прерываний (NVIC): управляет внешними и внутренними событиями (прерываниями), которые требуют немедленной реакции процессора.

4. Периферийные модули (модули ввода-вывода).

Это «органы чувств» и «инструменты» МК для взаимодействия с внешним миром:

• порты ввода-вывода (GPIO): универсальные выводы, которые можно настроить на вход (читать состояние кнопки) или выход (подать напряжение на светодиод);

• таймеры/счетчики (Timers/Counters):

– счетчик импульсов: подсчет внешних событий (оборотов двигателя);

– таймер: формирование точных временных интервалов;

– генерация ШИМ-сигнала (PWM): управление яркостью светодиодов, скоростью моторов;

– захват (Input Capture): фиксация времени наступления внешнего события;

• интерфейсы связи:

– UART (USART): асинхронный последовательный порт. Общение с компьютером, GPS-модулями;

– I²C (TWI): двухпроводной интерфейс для связи со множеством датчиков;

– SPI: высокоскоростной интерфейс для связи с дисплеями, картами памяти;

– USB: для подключения к компьютеру;

– CAN: промышленный интерфейс для автомобилей;

• аналоговые модули:

– АЦП (ADC): преобразует аналоговое напряжение (с датчика температуры) в цифровой код;

– ЦАП (DAC): выполняет обратную операцию — преобразует цифровой код в аналоговое напряжение (для генерации звука);

– компаратор: сравнивает два напряжения и выдает цифровой сигнал о том, какое из них больше.

Принцип взаимодействия блоков.

Все эти блоки соединены системной шиной. Программист взаимодействует с периферийными модулями не напрямую, а через чтение и запись в их регистры — специальные ячейки памяти, связанные с каждым модулем.

Пример работы АЦП:

1. Программа записывает в регистр управления АЦП команду на запуск преобразования на определенном канале.

2. АЦП выполняет преобразование.

3. Программа читает регистр статуса АЦП, чтобы проверить, завершено ли преобразование.

4. Если завершено, программа читает результат из регистра данных АЦП.

Микроконтроллер — это не просто процессор, а целый компьютер на одном кристалле. Его функциональность определяется набором периферийных модулей.

Понимание назначения каждого блока позволяет:

- осознанно выбирать МК для задачи (не переплачивать за ненужную периферию);
- эффективно программировать его, напрямую работая с регистрами периферии;
- предвидеть конфликты ресурсов и избегать их (когда два модуля требуют один и тот же вывод или шину);
- глубоко отлаживать систему, понимая, на каком уровне произошел сбой.

Конфигурирование МК — это процесс настройки его внутренних ресурсов (периферии, тактирования, выводов) перед началом или в процессе выполнения основной программы. Это фундаментальный этап разработки встроенной системы.

Без правильной конфигурации МК либо не будет работать вообще, либо будет работать некорректно.

Практически каждый функциональный блок МК требует начальной настройки:

1) системы тактирования (Clock Tree): источников clock (HSI, HSE, PLL), делителей частоты для ядра и шин (AHB, APB1, APB2);

2) портов ввода-вывода (GPIO): направления (вход-выход), типа выхода (push-pull, open-drain), скорости, подтяжек (pull-up/pull-down);

3) периферийных модулей: режимов работы таймеров, настроек UART (скорость, биты данных, стоп-биты), настроек АЦП (разрешение, канал) и т.д.;

4) прерываний (Interrupts): разрешения прерываний от конкретных источников, установки их приоритетов;

5) контроллера питания: настройки режимов энергосбережения.

Уровни конфигурирования (от низкоуровневого к высокоуровневому).

Уровень 1. Прямая работа с регистрами (Register-Level).

Это самый фундаментальный и гибкий способ. Программист напрямую обращается к регистрам специальных функций (SFR — Special Function Registers), которые являются «органами управления» модулей МК.

Пример программы:

```
// Пример для STM32: Включение тактирования порта A и настройка вывода PA5 на выход
RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;      // Тактирование для GPIOA
GPIOA->MODER &= ~GPIO_MODER_MODER5_Msk;   // Очистка битов режима для пина 5
GPIOA->MODER |= (0x1 << GPIO_MODER_MODER5_Pos); // Установка режима 'Output'
GPIOA->OTYPER &= ~GPIO_OTYPER_OT5_Msk;    // Режим выхода: Push-Pull
GPIOA->OSPEEDR |= (0x3 << GPIO_OSPEEDR_OSPEED5_Pos); // Высокая скорость
```

Плюсы: полный контроль, минимальный размер кода, высокая производительность.

Минусы: требует глубокого изучения даташита (1 000+ страниц), легко ошибиться, код плохо читаем.

Уровень 2. Использование стандартных периферийных библиотек (HAL/LL).

Производители создают библиотеки функций-оберток для упрощения работы с регистрами:

• HAL (Hardware Abstraction Layer): высокоуровневая библиотека. Функции носят общий характер (HAL_UART_Transmit(), HAL_ADC_Start()).

Плюсы: быстрая разработка, код легко переносим между разными МК одного производителя.

Минусы: большой размер кода, не всегда оптимальная производительность;

• LL (Low Layer): низкоуровневая библиотека. Ближе к регистрам, но более читаема и безопасна.

Плюсы: больший контроль и эффективность, чем у HAL, но проще, чем прямой доступ.

Минус: Менее переносима.

Пример программы:

// Тот же пример: настройка вывода PA5 на выход с помощью HAL

GPIO_InitTypeDef GPIO_InitStruct = {0};

__HAL_RCC_GPIOA_CLK_ENABLE(); *// Включить тактирование GPIOA*

GPIO_InitStruct.Pin = GPIO_PIN_5; *// Выбираем пин 5*

GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP; *// Режим: выход Push-Pull*

GPIO_InitStruct.Pull = GPIO_NOPULL; *// Подтяжка: нет*

GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_HIGH; *// Скорость: высокая*

HAL_GPIO_Init(GPIOA, &GPIO_InitStruct); *// Применить настройку*

Уровень 3. Использование визуальных конфигураторов и генераторов кода (CubeMX, MPLAB).

Современный стандарт для быстрого старта. Это программы с графическим интерфейсом, где можно мышкой настроить пины, периферию и тактирование.

Алгоритм работы:

1. Выбираем модель МК.

2. В графическом интерфейсе «расставляете» назначения пинов (например, какой пин будет UART_TX, какой I2C_SDA).

3. Настраиваете дерево тактирования, двигая ползунки.

4. Настраиваете периферию в диалоговых окнах.

5. Нажимаете «Generate Code». Программа генерирует полностью готовый проект для IDE (Keil, IAR, Eclipse) со всей инициализацией на С.

Плюсы: невероятная скорость настройки, минимум ошибок, не нужно помнить все регистры.

Минус: требует понимания, что именно генерируется, чтобы при необходимости исправить код вручную.

Примеры: STM32CubeMX (для STM32), MPLAB Code Configurator (для PIC), Simplicity Configurator (для EFM32), STM32CubeIDE включает в себя этот функционал.

Типовой алгоритм конфигурирования в коде.

Любая программа для МК начинается с функции инициализации (часто main() выглядит так):

```
int main(void) {
  // 1. Конфигурация тактирования (Настройка HSI/HSE, PLL, делителей)
   SystemClock_Config();

  // 2. Включение тактирования необходимых периферийных модулей
```

```
    __HAL_RCC_GPIOA_CLK_ENABLE();
    __HAL_RCC_USART2_CLK_ENABLE();

    // 3. Конфигурация выводов GPIO
    MX_GPIO_Init(); // Эта функция часто генерируется CubeMX

    // 4. Конфигурация периферийных модулей
    MX_USART2_UART_Init(); // Настройка скорости, битов и т.д.

    // 5. Настройка прерываний (если нужны)
    HAL_NVIC_SetPriority(USART2_IRQn, 0, 0);
    HAL_NVIC_EnableIRQ(USART2_IRQn);

    // 6. Бесконечный рабочий цикл
    while (1) {
      // Здесь основная логика программы
    }
}
```

Полезные инструменты и ресурсы:

1. Даташит (Datasheet): документ с описанием электрических характеристик, выводов корпуса.

2. Reference Manual (RM): самый важный документ! Содержит подробное описание всех регистров и работы всех модулей МК. Это ваша «Библия».

3. Application Notes (AN): примеры применения от производителя по конкретным задачам (например, «как реализовать USB HID на STM32»).

4. Примеры кода (Examples): готовые проекты от производителя и сообщества.

Конфигурирование — это процесс «оживления» железа. От простого включения тактирования до сложной настройки продвинутых режимов работы таймеров — все это конфигурирование.

Новичкам лучше начинать с визуальных конфигураторов (CubeMX), чтобы быстро получить результат и избежать ошибок.

Для глубокого понимания и создания оптимизированного кода необходимо учиться работать с регистрами и читать Reference Manual.

В реальных проектах часто используется комбинация: базовая инициализация генерируется в CubeMX, а затем точечно дописывается и оптимизируется вручную под конкретные задачи.

КОНТРОЛЬНЫЕ ВОПРОСЫ ПО ТЕМЕ «ВСТРАИВАЕМЫЕ СИСТЕМЫ НА ОСНОВЕ МИКРОКОНТРОЛЛЕРОВ»

Теоретические основы:

1. Дайте определение встраиваемой системе. Чем она отличается от персонального компьютера?

2. Что такое микроконтроллер (МК)? Перечислите его основные компоненты, интегрированные на кристалле.

3. В чем ключевое отличие микроконтроллера от микропроцессора (CPU)?

4. Опишите архитектуру фон Неймана и гарвардскую архитектуру. Какая из них чаще применяется в МК и почему?

5. Что такое система команд (ISA)? В чем разница между архитектурами RISC и CISC? Приведите примеры каждого типа.

Аппаратное обеспечение и архитектура МК:

6. Что такое система тактирования МК? Для чего нужны основные источники тактовых сигналов (HSI, HSE, LSI, LSE)?

7. Объясните назначение и принцип работы сторожевого таймера (Watchdog Timer).

8. Что такое регистры специальных функций (SFR) и для чего они используются?

9. Опишите назначение и принципы работы основных модулей памяти МК (Flash, RAM, EEPROM).

10. Каковы функции и особенности работы портов ввода-вывода (GPIO)? Что означают настройки input pull-up, output open-drain?

11. Для чего нужны схемы сброса (POR, BOD) в МК?

Периферийные модули и интерфейсы:

12. Перечислите основные последовательные интерфейсы связи в МК. Кратко охарактеризуйте принципы работы UART, I2C и SPI.

13. В чем ключевое отличие асинхронного (UART) и синхронного (SPI, I2C) обмена данными?

14. Для чего предназначены таймеры в МК? Назовите основные режимы их работы (счетчик, генерация ШИМ, захват, сравнение).

15. Что такое АЦП и ЦАП? Где они применяются во встраиваемых системах?

16. Что такое прерывания (interrupts) в МК? Опишите алгоритм обработки прерывания.

Программное обеспечение и разработка:

17. Что такое кросс-компиляция и почему она используется при разработке для МК?

18. Опишите типовую структуру программы (прошивки) для МК. Что обычно происходит в функции main()?

19. Что такое бесконечный цикл (super loop) и почему он используется в программах для МК?

20. Какие средства и инструменты (аппаратные и программные) используются для отладки программ на МК?

21. Объясните, что такое энергосберегающие режимы работы МК (Sleep, Stop, Standby). Какой принцип лежит в основе минимизации энергопотребления?

Проектирование и практические аспекты:

22. Какие основные этапы разработки встраиваемой системы вы можете выделить?

23. По каким критериям выбирают микроконтроллер для конкретной задачи?

24. Что такое реальное время (real-time) в контексте встраиваемых систем? Чем система жесткого реального времени (hard real-time) отличается от системы мягкого реального времени (soft real-time)?

25. Что такое RTOS (ОСРВ)? Зачем она нужна и в каких случаях ее применение оправдано?

26. Назовите основные аппаратные и программные методы обеспечения надежности и отказоустойчивости встраиваемых систем.

Вопросы на примере конкретных платформ:

27. Назовите популярные семейства микроконтроллеров и их производителей (например, STM32, AVR, PIC, ESP32). В чем их сильные стороны?

28. Что такое Arduino? В чем разница между разработкой под «голый» МК и с использованием платформы Arduino?

29. Для каких задач лучше подходят 8-битные МК (например, AVR), а для каких — 32-битные (например, ARM Cortex-M)?

30. Что такое визуальные конфигураторы (STM32CubeMX, MPLAB Code Configurator)? Как они упрощают процесс разработки?

Ключевые темы для подготовки:

• отличие МК от микропроцессора: МК — это законченная система на кристалле (CPU, память, периферия), а микропроцессору требуется внешняя обвязка;

• иерархия памяти: понимание, для чего нужны Flash, RAM и EEPROM;

• принципы работы периферии: как работают таймеры (счетчик, ШИМ), АЦП, последовательные интерфейсы (UART, I2C, SPI);

• прерывания: механизм обработки, векторы прерываний, приоритеты;

• система тактирования: источники clock, PLL, делители частоты;

• энергопотребление: режимы сна, методы снижения потребления;

• инструменты: назначение программаторов, отладчиков, IDE;

• надежность: роль сторожевого таймера (WDT) и детектора питания (BOD).

ЛАБОРАТОРНЫЕ РАБОТЫ

ЛАБОРАТОРНАЯ РАБОТА № 1
СТРУКТУРА ПРОЦЕССОРНОГО ЯДРА МК.
СИСТЕМА КОМАНД. ОРГАНИЗАЦИЯ ПАМЯТИ

Цель: изучить архитектурные особенности микроконтроллера, получить практические навыки анализа системы команд и работы с различными типами памяти на примере конкретного МК (например, AVR ATmega328P или ARM Cortex-M).

Теоретическая часть (для подготовки и отчета):

1. Структура процессорного ядра МК:

• центральный процессор (ЦП): состоит из АЛУ (арифметико-логическое устройство) и устройства управления;

• регистровый файл: набор сверхбыстрой памяти внутри процессора:

– регистры общего назначения (РОН): используются для временного хранения данных и результатов операций (например, 32 регистра R0-R31 в AVR);

– специальные регистры:

○ регистр состояния (SREG — Status Register): содержит флаги (биты), которые устанавливаются после выполнения арифметических или логических операций (Zero, Carry, Overflow и др.);

○ счетчик команд (Program Counter — PC): хранит адрес следующей команды в памяти программ;

○ указатель стека (Stack Pointer — SP): хранит адрес вершины стека в ОЗУ;

• системная шина: соединяет ядро с памятью и периферией (шина адреса, шина данных, шина управления).

2. Система команд МК (На примере AVR).

Типы команд:

- команды пересылки данных: MOV, LDI, LDS, STS;
- арифметические и логические команды: ADD, SUB, AND, OR, INC, DEC;
- команды условных и безусловных переходов: RJMP, RCALL, BRNE, BREQ;
- команды работы с битами: SBI, CBI, SBIC, SBIS.

Способы адресации:

- непосредственная: операнд содержится в коде команды (LDI R16, 0xFF);
- прямая: адрес операнда содержится в коде команды (STS 0x0060, R16);
- регистровая: операнды находятся в регистрах (ADD R16, R17);
- косвенная: адрес операнда находится в регистре-указателе (LD R16, X+).

3. Организация памяти МК (гарвардская архитектура):

- память программ (Flash-память): энергонезависимая. Хранит код программы и константы. Адресуется 16-разрядными адресами (до 64К слов);
- ОЗУ (RAM): энергозависимая. Хранит переменные, стек, буферы. Адресуется байтно;
- ЭСППЗУ (EEPROM): энергонезависимая. Хранит данные, которые должны сохраняться после отключения питания (настройки, калибровки). Адресуется байтно;
- пространство ввода-вывода (I/O Space): содержит адреса регистров периферии (например, PORTB, DDRB) и системных регистров (SREG).

Практическая часть (задания для выполнения).

Оборудование: ПК, IDE (Atmel Studio, Microchip Studio, PlatformIO), программатор / отладочная плата (например, Arduino Uno как программатор AVR), документация (даташит ATmega328P).

Задание 1. Анализ системы команд и регистрового файла:

1. Откройте даташит на МК (ATmega328P). Найдите раздел «Instruction Set Summary».

2. Изучите группы команд. Для трех команд из каждой группы напишите:

- мнемонику (название);
- синтаксис;
- краткое описание операции;
- как она влияет на флаги регистра статуса (SREG).

3. Найдите раздел «Register Summary». Выпишите адреса в пространстве I/O для следующих регистров:

- SREG (Status Register);
- SPL (Stack Pointer Low);
- DDRB (Data Direction Register B);
- PORTB (Data Register B).

Задание 2. Написание и анализ простейшей программы на ассемблере.

Напишите программу на ассемблере, которая выполняет следующие действия:

1) загружает в регистр R16 число 150;

2) загружает в регистр R17 число 170;

3) складывает содержимое R16 и R17;

4) результат (сумму) записывает в ячейку ОЗУ с адресом 0x0100;

5) результат также записывает в старший байт регистра Z (R31).

Вопросы для анализа.

Какое значение будет в ячейке 0x0100 после выполнения? Какие флаги в SREG установятся?

Какой командой можно загрузить адрес 0x0100 в регистр Z (Z = R30:R31)?

Пример кода:

```
asm
.include "m328pdef.inc" ; Подключение описания МК
.cseg                   ; Начало сегмента кода (Flash)
.org 0x0000             ; Установка адреса начала кода

   ldi r16, 150         ; Непосредственная загрузка в R16
   ldi r17, 170         ; Непосредственная загрузка в R17
   add r16, r17         ; Сложение R16 = R16 + R17
   sts 0x0100, r16      ; Сохранение результата в ОЗУ по адресу 0x0100
   mov r31, r16         ; Копирование результата в R31 (старший байт Z)
```

Задание 3. Работа с памятью (моделирование в симуляторе):

1. Загрузите написанную программу в симулятор (например, в Atmel Studio).

2. Запустите пошаговое выполнение (F10).

3. После каждого шага наблюдайте за изменением значений в регистрах R16, R17, регистре статуса (SREG) и ячейке ОЗУ с адресом 0x0100.

4. Отчетный вопрос: какое шестнадцатеричное значение будет записано в ячейку ОЗУ? Объясните, почему установились определенные флаги в SREG (например, флаг переноса C).

Задание 4. Сравнительный анализ (для продвинутого уровня):

• для AVR: объясните разницу между командами STS 0x0100, R16 и ST Z, R16. В каком случае какая команда эффективнее?

• если используется ARM Cortex-M: объясните модель памяти и систему команд Thumb-2. Чем она принципиально отличается от AVR?

Содержание отчета:

1. Титульный лист.

2. Цель работы.

3. Краткие теоретические сведения (по пунктам 1–3 теоретической части).

4. Выполненные задания с ответами на все вопросы.

5. Листинг программы из задания 2 с комментариями.

6. Скриншоты из симулятора, показывающие состояние регистров и памяти после выполнения программы.

7. Выводы: что изучили, какие навыки приобрели, с какими трудностями столкнулись.

ЛАБОРАТОРНАЯ РАБОТА № 2
ПОРТЫ ВВОДА-ВЫВОДА, ТАЙМЕРЫ, МОДУЛЬ ПРЕРЫВАНИЙ МК

Цель: получить практические навыки работы с основными периферийными модулями микроконтроллера, такими как настройка портов ввода-вывода, использование таймеров и организация обработки прерываний.

Оборудование: ПК, IDE (Arduino, PlatformIO, Keil, STM32CubeIDE), отладочная плата (Arduino Uno, STM32 Nucleo, etc.), светодиод, кнопка, резистор 1–10 кОм.

Теоретическая часть (кратко):

1. Порты ввода-вывода (GPIO).

Назначение: основной способ взаимодействия с внешним миром.

Регистры управления (на примере AVR):

- DDRx (Data Direction Register): определяет направление пина (1 — выход, 0 — вход);
- PORTx (Data Register): для выхода: устанавливает уровень (1 — высокий, 0 — низкий); для входа: управляет подтягивающим резистором (1 — включить, 0 — выключить);
- PINx (Port Input Register): чтение текущего логического уровня на пине.

2. Таймеры-счетчики.

Назначение: подсчет временных интервалов, генерация сигналов, измерение длительности импульсов.

Основные режимы:

• Normal Mode: простой счет от 0 до максимума (переполнения) и сброс;

• CTC (Clear Timer on Compare Match): сброс таймера при достижении значения сравнения. Идеален для генерации точных интервалов;

• Fast PWM: генерация ШИМ-сигнала для управления яркостью светодиода, скоростью мотора.

Ключевые регистры: TCNTx (счетчик), OCRxA/B (регистр сравнения), TCCRxA/B (регистры управления таймером).

3. Модуль прерываний.

Назначение: реагирование на асинхронные события без постоянного опроса (освобождает ресурсы ЦП).

Типы прерываний:

• внешние (External Interrupt): вызываются изменением уровня на определенном пине;

• внутренние (Timer Interrupt): вызываются по событию таймера (переполнение, совпадение).

Вектор прерывания: адрес в памяти, с которого начинается выполнение программы-обработчика прерывания.

Приоритет: определяет, какое прерывание будет обработано первым, если они произошли одновременно.

Практическая часть (задания для выполнения).

Задание 1. Управление светодиодом с помощью кнопки (GPIO).

Цель: настроить пин на вход (кнопка) и на выход (светодиод). Реализовать инвертирование состояния светодиода при каждом нажатии кнопки.

Схема:

• светодиод: анод → резистор 220 Ом → PIN 13, катод → GND;

• кнопка: один вывод → PIN 2, другой вывод → GND. *Использовать внутреннюю подтяжку к питанию.*

Алгоритм (псевдокод):

1. Настроить пин светодиода как выход.

2. Настроить пин кнопки как вход с включенной внутренней подтяжкой (PORTx = 1 для входа).

3. В бесконечном цикле:

• считать состояние пина кнопки;

• если состояние изменилось с высокого на низкое (нажатие обнаружено), инвертировать состояние светодиода;

• сделать небольшую задержку для защиты от дребезга контактов.

Задание: напишите код, реализующий эту логику. Объясните, зачем нужна подтяжка и борьба с дребезгом.

Задание 2. Мигание светодиодом с помощью таймера (Timer CTC Mode + Interrupt).

Цель: отказаться от функции delay(). Реализовать мигание светодиодом с периодом 1 секунда, используя прерывание по совпадению таймера.

Алгоритм:

1. Настроить пин светодиода как выход.

2. Сконфигурировать таймер (например, Timer1) в режиме **CTC**.

3. Рассчитать значение регистра сравнения (OCR1A) для заданного интервала (1 сек) с учетом частоты МК и предделителя таймера.

4. Разрешить прерывание по совпадению с OCR1A.

5. Написать обработчик прерывания (ISR), в котором инвертируется состояние светодиода.

6. В функции main() запустить таймер и включить глобальное разрешение прерываний. Основной цикл должен быть пустым.

Задание: напишите код, реализующий мигание через прерывания таймера. Приложите расчет значения для OCR1A.

Задание 3. Измерение длительности нажатия кнопки (Timer Input Capture Mode)

Цель: использовать таймер для измерения времени в микросекундах, пока кнопка нажата.

Алгоритм:

1. Настроить пин кнопки как вход. Настроить внешнее прерывание на этом пине на любое изменение.

2. Настроить таймер (например, Timer1) в режиме нормального счета с максимальным предделителем для максимального времени счета.

3. В обработчике внешнего прерывания:

- если кнопка нажата (переход из 1 в 0) — зафиксировать значение счетчика таймера (TCNT1) в переменную start_time;
- если кнопка отпущена (переход из 0 в 1) — зафиксировать значение счетчика таймера в переменную end_time;
- вычислить длительность: duration = end_time — start_time;
- пересчитать длительность в микросекунды, учитывая частоту таймера.

Задание: напишите код, который измеряет длительность нажатия кнопки и выводит результат в последовательный порт (UART). Постройте схему временной диаграммы работы.

Содержание отчета:

1. Титульный лист.
2. Цель работы.
3. Краткие теоретические сведения по трем модулям.
4. Описание заданий и алгоритмов решения.
5. Принципиальные схемы подключения кнопки и светодиода.
6. Листинги программ с подробными комментариями:

- код для задания 1 (GPIO);
- код для задания 2 (Timer + Interrupt);
- код для задания 3 (Input Capture).

7. Расчеты для настройки таймера (предделитель, значение сравнения).

8. Скриншоты работы программы (например, вывод в Serial Monitor длительности нажатия).

9. Выводы:

- каковы преимущества использования таймеров и прерываний перед функцией delay()?
- с какими трудностями столкнулись и как их решили?
- где могут применяться изученные режимы таймеров на практике?

ГЛАВА 3. СТРУКТУРА ПРОГРАММЫ И ОСНОВНЫЕ КОНСТРУКЦИИ ЯЗЫКА СИ

3.1. ВВОДНЫЕ ПОНЯТИЯ ЯЗЫКА СИ. СТРУКТУРА ПРОГРАММЫ НА СИ

Язык С является прародителем многих современных языков (C++, C#, Java, Python) и до сих пор широко используется в системном программировании, разработке под микроконтроллеры и создании высокопроизводительных приложений.

Ключевые концепции языка Си.

Особенности и философия языка Си:

- компилируемый язык: исходный код (текст программы) преобразуется компилятором в машинный код, понятный процессору. Это обеспечивает высокую производительность;
- статическая типизация: тип каждой переменной (целое число, символ и т.д.) должен быть объявлен заранее и не может меняться во время выполнения;
- язык среднего уровня: С сочетает в себе возможности низкоуровневого программирования (прямая работа с памятью, адресами) с конструкциями высокоуровневых языков (функции, циклы, условия). Это его главная сила;
- минимализм и эффективность: в языке относительно мало ключевых слов. Он не только предоставляет программисту большую свободу и контроль, но и требует большей внимательности (например, нет встроенной проверки выхода за границы массива).

Базовая структура программы.

Любая программа на С начинается с функции *main*. Это точка входа, с которой начинает выполняться код:

```
// Однострочный комментарий
```

```
/* Многострочный
   комментарий */

// Подключение заголовочного файла для стандартных функций ввода/вывода
#include <stdio.h>

// Объявление функции main. `int` означает, что функция возвращает целое число.
int main(void) { // Начало тела функции

  // Тело функции: здесь пишутся инструкции
  printf("Hello, World!\n"); // Вывод текста на экран

  return 0; // Возврат кода успешного завершения операционной системе
} // Конец тела функции
```

• #include <stdio.h> — директива препроцессора. Она «подключает» библиотеку функций для ввода и вывода данных (например, printf);

• int main(void) — объявление главной функции. Программа всегда начинает работу с нее;

• { } — фигурные скобки обозначают начало и конец блока кода (тела функции);

• printf(...) — вызов функции для вывода текста на экран;

• return 0; — инструкция возврата из функции. 0 обычно означает, что программа завершилась без ошибок;

• ; — точка с запятой обязательна после каждой инструкции. Она отмечает конец выражения.

Переменные и типы данных.

Переменная — это именованная область памяти для хранения данных (табл. 3.1). Перед использованием переменную необходимо

объявить, указав ее тип. Объявление переменной обеспечивает ее хранение в определенном разделе памяти и быстрый доступ к ней.

Таблица 3.1

Основные типы данных

Тип данных	Размер (обычно)	Что хранит	Пример
int	4 байта	Целые числа	int count = 10;
float	4 байта	Числа с плавающей точкой	float price = 12.99;
double	8 байт	Числа с плавающей точкой (двойной точности)	double pi = 3.14159;
char	1 байт	Символ	char grade = 'A';

Пример объявления и использования переменных:

```
int main(void) {
  int age;        // Объявление переменной 'age' типа int
  age = 25;       // Инициализация (присвоение значения)

  float height = 1.75f; // Объявление и инициализация в одной строке

  char initial = 'R';   // Символы заключаются в одинарные кавычки

  return 0;
}
```

Основные операторы:

• арифметические: + (сложение), – (вычитание), * (умножение), / (деление), % (остаток от деления).

int result = 5 + 3 * 2; // result будет 11, а не 16! Важен приоритет операций;

• операторы присваивания: = (присвоить), +=, -=, *=, /=.

age += 1; // То же самое, что age = age + 1;

• операторы сравнения: == (равно), != (не равно), > (больше), < (меньше), >= (больше или равно), <= (меньше или равно).

Возвращают 1 (истина) или 0 (ложь);

• логические операторы: && (И), || (ИЛИ), ! (НЕ).

Используются для объединения условий.

Базовые структуры управления, логические блоки.

Условный оператор if-else — позволяет выполнять код только при выполнении определенного условия:

```
int number = 10;

if (number > 0) {
  printf("Число положительное\n");
} else if (number < 0) {
  printf("Число отрицательное\n");
} else {
  printf("Число равно нулю\n");
}
```

Цикл for — позволяет повторять выполнение блока кода заданное количество раз:

```
// for (инициализация; условие; инкремент)
for (int i = 0; i < 5; i++) {
  printf("i = %d\n", i); // %d будет заменено на значение переменной i
}
// На экране: i = 0, i = 1, i = 2, i = 3, i = 4
```

Цикл while — выполняет блок кода, пока условие истинно:

```
int counter = 3;

while (counter > 0) {
  printf("Обратный отсчет: %d\n", counter);
```

```
  counter--; // Уменьшаем counter на 1
}
```

Функции. Функции позволяют разбивать программу на отдельные логические блоки, чтобы избежать повторения кода, сделать его более читаемым и оптимальным:

// Объявление функции (прототип). Говорит компилятору о ее существовании.

```
int add_numbers(int a, int b);

int main(void) {
  int result = add_numbers(5, 3); // Вызов функции
  printf("Сумма: %d\n", result);
  return 0;
}

// Определение функции (реализация).
// Функция принимает два целых числа и возвращает их сумму.
int add_numbers(int a, int b) {
  int sum = a + b;
  return sum; // Возврат результата
}
```

Базовая памятка начинающему программисту:

1. Всегда начинайте с функции main().
2. Подключайте необходимые заголовочные файлы (#include...).
3. Объявляйте переменные, которые вам нужны.
4. Пишите логику программы, используя операторы, условия и циклы.
5. Для повторяющихся действий создавайте функции.
6. Компилируйте и запускайте свою программу.

Программа на Си — это всегда набор функций во главе с обязательной функцией *main()*. Код выполняется последовательно внутри этих функций, а директивы *#include* подключают необходимые инструменты для работы.

3.2. ТИПЫ ДАННЫХ В СИ. ПЕРЕМЕННЫЕ В СИ. КОНСТАНТЫ В СИ

Типы данных, переменные и константы — это фундаментальные кирпичики, из которых строится любая программа на Си. Рассмотрим их подробнее:

1. Переменные (Variables).

Переменная — это именованная область памяти, предназначенная для хранения данных, которые могут изменяться во время выполнения программы.

Объявление переменной: вы сообщаете компилятору о существовании переменной, указывая ее тип и имя:

```
int age;        // Объявлена переменная 'age' типа int
float price;    // Объявлена переменная 'price' типа float
char letter;    // Объявлена переменная 'letter' типа char
```

Инициализация переменной: присваивание переменной начального значения:

```
int age = 25;               // Объявление с инициализацией
float price = 99.99f;       // Суффикс 'f' для float
char letter = 'A';          // Символы в одинарных кавычках
int count;                  // Объявление
count = 10;                 // Инициализация позже
```

Правила именования:

- могут содержать буквы, цифры и знак подчеркивания _;
- не могут начинаться с цифры;
- регистрозависимы (myVar и myvar — разные переменные);
- нельзя использовать ключевые слова языка (int, return, if и т.д.).

2. Типы данных (Data Types).

Тип данных определяет:

- какой вид данных может храниться в переменной;
- сколько памяти она занимает;
- какие операции с ней можно выполнять.

Основные встроенные (primitive) типы данных в Си приведены в табл. 3.1.

Модификаторы типов (Type Modifiers).

Их можно добавлять к int и char для изменения их диапазона и размера:

• short / long: изменяют диапазон значений:

```
short int smallNumber;    // Обычно 2 байта
long int bigNumber;       // Обычно 4 или 8 байт
long long int veryBigNumber; // Обычно 8 байт
```

• signed / unsigned: убирают возможность хранить отрицательные числа, тем самым удваивая положительный диапазон. По умолчанию все типы signed:

```
unsigned int positiveOnly; // Только от 0 и выше
signed char normalChar;    // От -128 до 127 (по умолчанию)
unsigned char byte;        // От 0 до 255
```

Проверить размер типа можно с помощью оператора sizeof:

```
printf("Size of int: %zu bytes\n", sizeof(int));
printf("Size of double: %zu bytes\n", sizeof(double));
```

3. Константы (Constants) (табл. 3.2, 3.3).

Константа — это именованная область памяти, значение которой не может быть изменено после ее инициализации.

Способы создания констант:

• ключевое слово const.

Самый современный и рекомендуемый способ:

```
const float PI = 3.14159f;
const int MAX_USERS = 100;
// PI = 5.0; // ОШИБКА! Нельзя изменить константу.
```

• директива препроцессора #define.

Старый, но до сих пор широко используемый способ. Препроцессор просто заменяет все вхождения IDENTIFIER (идентификаторов, объявленных переменных) на значения в коде до компиляции:

#define PI 3.14159 *// Не ставится точка с запятой и не используется равно!*

#define MAX_USERS 100

#define APP_NAME "MyApp"

• перечисление enum.

Идеально подходит для создания констант, представляющих набор связанных целочисленных значений:

// Создает константы: JAN=0, FEB=1, ..., DEC=11

enum months { JAN, FEB, MAR, APR, MAY, JUN, JUL, AUG, SEP, OCT, NOV, DEC };

// Можно явно указать значения

enum error_codes { SUCCESS = 0, FILE_NOT_FOUND = 1, PERMISSION_DENIED = 2 };

Таблица 3.2

Сравнение const и #define

Критерий	**const**	**#define**
Обработка	Компилятор	Препроцессор
Память	Выделяется память, как для переменной	Память не выделяется, просто подстановка текста
Отладка	Видно в отладчике как переменную	Не видно в отладчике
Область видимости	Подчиняется правилам области видимости переменных	Действует от точки объявления до конца файла
Тип данных	Имеет конкретный тип	Не имеет типа, просто текст

Таблица 3.3

Сводная таблица основных типов и их размеров

Тип данных	Обычный размер	Диапазон значений (примерный)	Спецификатор для printf
char	1 байт	от –128 до 127	%c (символ), %d (число)
unsigned char	1 байт	от 0 до 255	%u, %d
int	4 байта	от –2^31 до 2^31-1	%d
unsigned int	4 байта	от 0 до 2^32-1	%u
short int	2 байта	от –32,768 до 32,767	%hd
long int	4 или 8 байт	Зависит от системы	%ld
float	4 байта	~6–7 значащих цифр	%f
double	8 байт	~15 значащих цифр	%lf
long double	10+ байт	>15 значащих цифр	%Lf

Пример программы, объединяющей все концепции:

```
#include <stdio.h>

// Константы через #define
#define TAX_RATE 0.05
#define PRODUCT_NAME "Coffee"

// Константа через const
```

```
const int MAX_QUANTITY = 100;

int main(void) {
    // Объявление и инициализация переменных
    int quantity = 2;
    double price_per_unit = 2.50;

    // Расчеты
    double subtotal = price_per_unit * quantity;
    double tax = subtotal * TAX_RATE;
    double total = subtotal + tax;

    // Вывод информации
    printf("Product: %s\n", PRODUCT_NAME);
    printf("Quantity: %d (max: %d)\n", quantity, MAX_QUANTITY);
    printf("Price per unit: $%.2f\n", price_per_unit);
    printf("Subtotal: $%.2f\n", subtotal);
    printf("Tax (%.0f%%): $%.2f\n", (TAX_RATE * 100), tax);
    printf("Total: $%.2f\n", total);

    return 0;
}
```

Вывод на экран:

```
text
Product: Coffee
Quantity: 2 (max: 100)
Price per unit: $2.50
Subtotal: $5.00
Tax (5%): $0.25
Total: $5.25
```

Правильный выбор типов данных, осмысленные имена переменных и использование констант вместо «магических чисел» в ко-

де — это основа написания эффективных, надежных и легкочитаемых программ на Си.

3.3. АРИФМЕТИЧЕСКИЕ И ЛОГИЧЕСКИЕ ОПЕРАТОРЫ ЯЗЫКА СИ

Арифметические и логические операторы в языке С — это инструмент для выполнения вычислений и принятия решений в программе.

Арифметические операторы (Arithmetic Operators) (табл. 3.4).

Используются для выполнения математических операций над числовыми значениями (переменными и константами).

Таблица 3.4

Арифметические операторы

Оператор	Название	Пример	Результат (при a = 10, b = 3)
+	Сложение	c = a + b;	c = 13
–	Вычитание	c = a – b;	c = 7
*	Умножение	c = a * b;	c = 30
/	Деление	c = a / b;	c = 3 (для целых чисел!)
%	Деление по модулю (остаток)	c = a % b;	c = 1 (остаток от деления 10 на 3)

Особенности деления (/):

• если оба операнда целые, выполняется целочисленное деление, результат округляется в сторону нуля:

int a = 10, b = 3;

float result = a / b; *// result = 3.0, а не 3.333!*

• чтобы получить дробный результат, хотя бы один операнд должен быть дробным (приведенным к float или double):

float result = (float)a / b; *// result = 3.333...*

float result = 10.0 / 3; *// result = 3.333...*

Операторы присваивания (Assignment Operators).

Используются для присваивания значений переменным. Базовый оператор — «=».

Присваивание — это помещение в ячейку памяти с заданным именем конкретного значения, числа, слова, знака. Например, «a = 67» означает, что в ячейку памяти с именем «a» помещается численное значение — 67. Если следующим действием будет команда «a = 1 270», то значение 67 заменится значением 1 270.

Составные операторы присваивания (Compound Assignment) (табл. 3.5).

Комбинируют арифметическую операцию и присваивание. Очень часто используются для краткости.

Таблица 3.5

Комбинированные операторы присваивания

Оператор	Пример	Эквивалентная запись
+=	a += 5;	a = a + 5;
-=	a -= 3;	a = a — 3;
*=	a *= 2;	a = a * 2;
/=	a /= 4;	a = a / 4;
%=	a %= 2;	a = a % 2;

Плюсы составных операторов:

1. Краткость: код становится компактнее.

2. Удобство: не нужно дублировать имя переменной.

3. Производительность: (на низком уровне) могут компилироваться в более эффективный код.

Операторы инкремента и декремента (Increment/Decrement) (табл. 3.6).

Команда *Increment* увеличивает значение переменной на 1, *Decrement* уменьшает на 1. Имеют две формы: префиксную и постфиксную.

Таблица 3.6

Форма записи Increment/Decrement

Оператор	Название	Пример	Эффект
++	Инкремент (увеличение на 1)	a++; или ++a;	a = a + 1;
--	Декремент (уменьшение на 1)	a--; или --a;	a = a — 1;

Важно! Ключевая разница между a++ и ++a состоит:

• в постфиксной форме (a++) сначала используется текущее значение переменной, а только потом оно увеличивается:

```
int a = 5;
int b = a++; // Шаг 1: b = 5; Шаг 2: a становится 6.
// Результат: b = 5, a = 6
```

• префиксной форме (++a) сначала значение переменной увеличивается, а затем используется:

```
int a = 5;
int b = ++a; // Шаг 1: a становится 6; Шаг 2: b = 6.
// Результат: b = 6, a = 6
```

Операторы сравнения (Comparison Operators) (табл. 3.7).

Используются для сравнения двух значений. Всегда возвращают целое число: 1 (если условие истинно) или 0 (если ложно). Эти операторы — основа для условий в if-else и циклов.

Важно! Не путайте оператор сравнения == (равно) с оператором присваивания = (присвоить). Эта ошибка очень распространена, и ее трудно найти:

if (a == 5) { ... } *// Проверяем, равно ли а пяти*

if (a = 5) { ... } *// ОШИБКА! Присваиваем а значение 5. Условие всегда истинно!*

Таблица 3.7

Операторы сравнения

Оператор	Проверка	Пример	Результат (при a = 5, b = 3)
==	Равно	a == b	0 (ложь)
!=	Не равно	a != b	1 (истина)
>	Больше	a > b	1 (истина)
<	Меньше	a < b	0 (ложь)
>=	Больше или равно	a >= 5	1 (истина)
<=	Меньше или равно	a <= 3	0 (ложь)

Логические операторы (Logical Operators) (табл. 3.8, 3.9).

Используются для объединения или инвертирования условий. Работают с логическими значениями (истина/ложь; или любыми числами, где 0 — это ложь, а любое не-zero — истина).

Таблица 3.8

Логические операторы

Оператор	Название	Пример	Пояснение
&&	Логическое И	(a > 0) && (a < 10)	Истинно, только если оба условия истинны
||	Логическое ИЛИ	(a == 0) || (b == 0)	Истинно, если хотя бы одно из условий истинно
!	Логическое НЕ	!(a == 5)	Инвертирует значение. !0 → 1, !1 → 0

Таблица 3.9

Таблицы истинности

a	b	a && b	a || b	!a
0	0	0	0	1
0	1	0	1	1
1	0	0	1	0
1	1	1	1	0

Приоритет операторов (Operator Precedence).

Операторы выполняются не слева направо, а в соответствии с их приоритетом (старшинством). Как в математике: умножение выполняется перед сложением.

Пример:

int result = 5 + 3 * 2; *// result = 11, а не 16!*

Сводка приоритета (от высшего к низшему):

1. () (скобки) — наивысший приоритет
2. ++, -- (постфиксные), !
3. *, /, %
4. +, -
5. >, >=, <, <=
6. ==, !=
7. &&
8. ||
9. =, +=, -=, *=, /=, %= — наинизший приоритет

Важно! Если сомневаетесь в порядке выполнения, используйте скобки (). Это делает код понятнее и избавляет от ошибок:

```
// Неочевидный код
if (a > 5 && b < 10 || c == 0) {...}
// Понятный код со скобками (даже если они не всегда нужны)
if ( (a > 5) && (b < 10) || (c == 0) ) {...}
```

3.4. ОПЕРАТОРЫ ВЕТВЛЕНИЯ В СИ

Операторы ветвления (табл. 3.10) — это конструкции, которые позволяют программе принимать решения и выполнять различные участки кода в зависимости от выполнения/невыполнения определенных условий.

Таблица 3.10

Сводная таблица операторов ветвления

Оператор	Когда использовать	Плюсы	Минусы
if	Простая проверка одного условия	Простота	_

Оператор	Когда использовать	Плюсы	Минусы
if-else	Выбор из двух вариантов	Простота, понятность	–
else if	Множественный выбор по разным условиям	Гибкость	Может стать громоздким
? :	Короткое условие при присваивании	Краткость	Снижает читаемость сложных условий
switch-case	Множественный выбор по **одному** выражению	Читаемость, скорость	Работает только с целыми типами

Они являются основой для создания логики любой программы. Существует три основных оператора ветвления: ***if-else***, ***else-if*** и ***switch-case***.

Оператор *if* (одиночное ветвление).

Самая простая форма. Выполняет блок кода, только если условие истинно (не равно нулю).

Синтаксис:

```
if (условие) {
   // Тело оператора if: код, выполняемый, если условие истинно
}
```

Пример:

```
int number = 10;

if (number > 0) {
   printf("Число положительное.\n"); // Этот код выполнится
}
////////////////////////////////////////////////////////////////////////////////////////////
int number = 10;
```

```
if (number < 0) {
   printf("Число отрицательное.\n"); // Этот код не выполнится
}
```

Оператор** if-else **(двойное ветвление).

Расширяет оператор if, добавляя блок кода, который выполняется, если условие ложно.

Синтаксис:

```
if (условие) {
   // Выполняется, если условие ИСТИННО
} else {
   // Выполняется, если условие ЛОЖНО
}
```

Пример:

```
int age = 16;

if (age >= 18) {
   printf("Доступ разрешен.\n");
} else {
   printf("Доступ запрещен. Вам нет 18.\n"); // Выполнится этот код
}
```

Цепочка условий** else-if **(множественное ветвление).

Позволяет проверить несколько взаимосвязанных условий последовательно.

Синтаксис:

```
if (условие1) {
   // Выполняется, если условие1 ИСТИННО
} else if (условие2) {
   // Выполняется, если условие1 ЛОЖНО, но условие2 ИСТИННО
} else if (условие3) {
   // Выполняется, если условие1 и условие2 ЛОЖНЫ, но условие3 ИСТИННО
```

```
} else {
    // Выполняется, если ВСЕ вышеперечисленные условия ЛОЖНЫ
}
```

Важно! Как только компилятор находит истинное условие, выполняется соответствующий блок кода, а *все последующие* else if и else игнорируются.

Пример:

```
int score = 85;

if (score >= 90) {
    printf("Оценка: A\n");
} else if (score >= 80) { // Проверяется, только если score < 90
    printf("Оценка: B\n"); // Выполнится этот код
} else if (score >= 70) {  // Проверяется, только если score < 80
    printf("Оценка: C\n");
} else {
    printf("Оценка: F\n");
}
```

Тернарный условный оператор ? : (Ternary Operator).

Краткая форма записи простого условия if-else. Возвращает значение, поэтому его часто используют при присваивании.

Синтаксис:

условие → ? → выражение_если_истина → выражение_если_ложь

Пример:

```
c
int a = 5, b = 10;
int max;

// Классический способ с if-else
if (a > b) {
    max = a;
```

```
} else {
    max = b;
}
// То же самое через тернарный оператор
max = (a > b) ? a : b; // max станет равным 10

// Другой пример: вывод в зависимости от условия
printf("Число %d является %s.\n", a, (a % 2 == 0) ? "четным": "нечетным");
```

Оператор выбора switch-case.

Идеален для ситуаций, когда нужно сравнить одну переменную со множеством конкретных значений. Читается лучше, чем длинная цепочка else if.

Синтаксис:

```
switch (выражение) {
    case константа1:
        // код, если выражение == константа1
        break; // Обязательно!
    case константа2:
        // код, если выражение == константа2
        break;
    // ...
    default:
        // код, если ни один case не подошел (аналог else)
        break;
}
```

Особенности:

- выражение должно быть целочисленного типа (int, char, enum);
- break необходим для выхода из switch. Если его забыть, выполнение «провалится» в следующий case;
- default-секция необязательна, но highly рекомендуется для обработки непредвиденных значений.

Пример без break (проваливание):

```
char grade = 'B';

switch (grade) {
    case 'A':
        printf("Отлично!\n");
        break;
    case 'B': // Сюда попадет программа
    case 'C':
        printf("Хорошо.\n"); // Выполнится этот код...
        break;              // ...и здесь break остановит switch
    case 'D':
        printf("Нужно подтянуть.\n");
        break;
    default:
        printf("Неизвестная оценка.\n");
}
// Вывод: "Хорошо."
```

Пример с проваливанием (часто используется для группировки):

```
int day = 3; // 1-Пн, 2-Вт, ..., 7-Вс

switch (day) {
    case 1:
    case 2:
    case 3:
    case 4:
    case 5:
        printf("Рабочий день\n");
        break;
    case 6:
    case 7:
```

```
        printf("Выходной день\n");
        break;
    default:
        printf("Ошибка: в неделе только 7 дней!\n");
}
// Вывод: "Рабочий день"
```

Операторы ветвления — это механизм принятия решений в программе. Выбор оператора зависит от конкретной задачи:

- для 1–2 условий используйте if или if-else;
- для нескольких разнородных условий используйте цепочку else if;
- для выбора из многих вариантов по значению одной переменной всегда используйте switch-case;
- для краткой записи простого присваивания по условию используйте тернарный оператор ? :.

3.5. ЦИКЛИЧЕСКИЕ КОНСТРУКЦИИ В СИ

Циклические конструкции в Си (табл. 3.11) необходимы для многократного выполнения одного и того же блока кода.

Таблица 3.11

Сравнительная таблица циклов

Цикл	Когда использовать	Преимущества	Недостатки
for	Известно кол-во итераций	Компактность, все управление в одной строке	Менее читаем при сложных условиях

Цикл	Когда использовать	Преимущества	Недостатки
while	Кол-во итераций неизвестно	Высокая читаемость для условий, не связанных со счетчиком	Можно забыть обновить условие (риск бесконечного цикла)
do...while	Минимум 1 итерация + проверка после	Гарантия, по меньшей мере одного выполнения	Используется реже, можно не заметить условие в конце

В языке Си существует три основных типа циклов:

1. Цикл *while* (цикл с предусловием).

Синтаксис:

```
while (условие) {
   // Тело цикла (блок кода, который повторяется)
}
```

Алгоритм работы:

1. Проверяется условие в скобках.

2. Если условие истинно (true, не равно нулю), выполняется тело цикла.

3. После выполнения тела цикла управление снова переходит к проверке условия.

4. Если условие ложно (false, равно нулю), выполнение цикла прекращается и программа переходит к следующей за циклом инструкции.

Используется, когда количество повторений заранее неизвестно и зависит от условия, которое вычисляется во время выполнения.

Пример. Чтение данных до тех пор, пока не будет введен 0:

```
int number;
printf("Введите числа (0 для выхода):\n");
scanf("%d", &number);
```

```
while (number != 0) {
    printf("Вы ввели: %d\n", number);
    scanf("%d", &number); // Обновление условия внутри цикла
}
```

2. Цикл *do...while* (цикл с постусловием).

Синтаксис:

```
do {
    // Тело цикла
} while (условие);
```

Алгоритм работы:

1. Сначала выполняется тело цикла.
2. После этого проверяется условие.
3. Если условие истинно, выполнение цикла повторяется.
4. Если условие ложно, цикл завершается.

Ключевая особенность: тело цикла do...while выполнится как минимум один раз, даже если условие изначально ложно.

Используется, когда действие нужно выполнить хотя бы один раз, а затем проверить условие для повторения (например, вывод меню).

Пример. Запрос пароля (как минимум одна попытка):

```
int password;
do {
    printf("Введите пароль (1234): ");
    scanf("%d", &password);
} while (password != 1234);
printf("Доступ разрешен!\n");
```

3. Цикл *for* (цикл со счетчиком).

Синтаксис:

```
for (инициализация; условие; изменение) {
    // Тело цикла
}
```

Инициализация выполняется один раз перед началом цикла. Обычно здесь объявляется и инициализируется счетчик.

Условие: условие продолжения цикла. Проверяется перед каждой итерацией.

Изменение: выполняется после каждой итерации цикла. Обычно здесь происходит увеличение/уменьшение счетчика.

Алгоритм работы:

1. Выполняется инициализация.
2. Проверяется условие. Если ложно → выход из цикла.
3. Выполняется тело цикла.
4. Выполняется изменение.
5. Переход к шагу 2.

Используется, когда количество повторений известно заранее или определяется на этапе инициализации.

Примеры:

a) классический счетчик:

```
// Вывод чисел от 0 до 9
for (int i = 0; i < 10; i++) {
    printf("%d\n", i);
}
```

b) изменение счетчика с другим шагом:

```
// Вывод четных чисел от 10 до 0
for (int i = 10; i >= 0; i -= 2) {
    printf("%d\n", i);
}
```

c) возможны сложные конструкции:

```
// Два счетчика в инициализации и изменении
for (int i = 0, j = 100; i < 10; i++, j -= 10) {
    printf("i = %d, j = %d\n", i, j);
}
```

4. Операторы управления циклами.

break; — **немедленно завершает выполнение всего цикла** и передает управление следующей за циклом инструкции.

```
for (int i = 0; i < 10; i++) {
    if (i == 5) {
        break; // Цикл прервется, когда i станет равным 5
    }
    printf("%d\n", i); // Выведет 0, 1, 2, 3, 4
}
```

continue; — **прерывает текущую итерацию цикла** и переходит к следующей (к проверке условия в while/for или к изменению счетчика в for):

```
for (int i = 0; i < 10; i++) {
    if (i % 2 == 0) {
        continue; // Перейти к следующей итерации, пропустив печать
    }
    printf("%d\n", i); // Выведет только нечетные числа: 1, 3, 5, 7, 9
}
```

5. Бесконечные циклы.

Иногда нужны циклы, которые работают вечно (например, основной цикл в прошивке МК).

Способы создания:

```
// 1. Использование for
for (;;) { // Все выражения можно опустить
    // Бесконечный цикл
}
// 2. Использование while
while (1) { // Условие всегда истинно
    // Бесконечный цикл
}
// 3. Использование do...while
do {
    // Бесконечный цикл
} while (1);
```

Выход из бесконечного цикла обычно осуществляется с помощью оператора break по какому-либо условию.

Выбор цикла зависит от конкретной задачи. Часто одну и ту же задачу можно решить с помощью любого из них, но правильный выбор делает код более читаемым и логичным.

3.6. УКАЗАТЕЛИ И АДРЕСА ПЕРЕМЕННЫХ В СИ

Адрес переменной.

Каждая переменная хранится в памяти компьютера по определенному адресу. Адрес — это просто число, обозначающее конкретную «ячейку» в оперативной памяти.

Например, работая с таблицами EXCEL, мы при написании формул используем адреса ячеек, в которых записаны данные, а не непосредственно числовые значения.

С помощью оператора взятия адреса & (амперсанд) этот адрес (число) можно определить:

```
int number = 42; // объявили целочисленную переменную
printf("Значение переменной: %d\n", number); // Вывод: Значение переменной: 42
printf("Адрес переменной: %p\n", (void*)&number); // Вывод: Адрес переменной: 0x7ffd42a1a23c (значение будет разным)
```

Спецификатор %p используется для вывода адреса (указателя).

Приведение (void*) является хорошим тоном, так как %p ожидает указатель на void.

Указатель (табл. 3.12).

Указатель (pointer) — это особый тип переменной, которая хранит не данные, а адрес другой переменной (или области памяти).

Это позволяет:

• обращаться к произвольным участкам памяти — это полезно при работе с массивами, структурами и динамической памятью;

• быстро передавать данные — вместо копирования больших объектов можно передавать только их адрес, который всегда занимает фиксированный объем (4 или 8 байт);

• изменять данные в функциях — в Си при передаче аргументов в функции создается копия данных, но указатели позволяют работать с данными напрямую в памяти.

Таблица 3.12

Ключевые операции с указателями

Операция	Синтаксис	Описание
Взятие адреса	&variable	Возвращает адрес переменной
Разыменование	*pointer	Получает значение по адресу в указателе
Присваивание	ptr1 = ptr2	Копирует адрес из одного указателя в другой
Сравнение	ptr1 == ptr2	Проверяет, указывают ли указатели на один адрес
Арифметика	ptr + n, ptr – n	Сдвигает указатель на n * sizeof(type) байт

Объявление указателя:

*<тип_данных> *<имя_указателя>;*

<тип_данных> — это тип данных переменной, на которую указатель будет ссылаться (например, int, float, char).

* (звездочка) — это синтаксис, который сообщает компилятору, что мы объявляем указатель.

Примеры объявления:

int *ptr; *// Указатель на переменную типа int*

float *fptr; *// Указатель на переменную типа float*

char *cptr; *// Указатель на переменную типа char*

Инициализация указателей.

Для того чтобы связать указатель с переменной, надо присвоить ему адрес этой переменной с помощью оператора &:

int number = 42;

int *ptr; *// Объявили указатель (пока он ни на что не указывает!)*

ptr = &number; *// Инициализировали указатель, присвоив ему адрес переменной number*

// Можно совместить объявление и инициализацию:

int number = 42;

int *ptr = &number; *// Объявление и инициализация в одной строке*

Важно! Неинициализированный указатель хранит «мусор», и его использование ведет к неопределенному поведению (undefined behavior) и сбоям. Всегда инициализируйте указатели.

Разыменование указателя.

Это операция, обратная получению адреса. Позволяет получить или изменить значение переменной, на которую указывает указатель. Для этого используется оператор * (звездочка):

int number = 42;

int *ptr = &number; *// ptr хранит адрес number*

printf("Адрес в ptr: %p\n", (void*)ptr); *// Выведет адрес number*

printf("Значение через ptr: %d\n", *ptr); *// Выведет 42 (РАЗЫМЕНОВАНИЕ)*

*ptr = 100; *// Меняем значение переменной number через указатель*

printf("Новое значение number: %d\n", number); *// Выведет 100*

ptr — это адрес, по которому хранятся данные.

*ptr — это сами данные, лежащие по этому адресу.

Рассмотрим аналогию:

• переменная number — это дом (в нем живет значение 42);

• адрес &number — это почтовый адрес этого дома (например, ул. Лунного Света, д. 0x7ffd42a1a23c);

• указатель ptr — это бумажка, на которой записан этот почтовый адрес;

• разыменование *ptr — это процесс: прийти по адресу, написанному на бумажке, и взаимодействовать с домом (посмотреть, кто живет, или поселить кого-то нового).

Указатель и массивы.

Имя массива является указателем на его первый элемент. Это одна из самых важных связей в Си:

```
int arr[5] = {10, 20, 30, 40, 50};
int *ptr = arr; // Эквивалентно int *ptr = &arr[0];

printf("Первый элемент: %d\n", arr[0]); // 10
printf("Через указатель: %d\n", *ptr);  // 10

// Арифметика указателей позволяет легко перемещаться по массиву
printf("Второй элемент: %d\n", *(ptr + 1)); // 20
// (ptr + 1) сдвигает указатель на sizeof(int) байт вперед
```

Указатель на указатель.

Указатель может хранить адрес другого указателя, как в квесте:

```
int value = 42;
int *ptr = &value;   // ptr хранит адрес value
int **pptr = &ptr;   // pptr хранит адрес ptr

printf("Значение value: %d\n", value);    // 42
```

```
printf("Через ptr: %d\n", *ptr);          // 42
printf("Через pptr: %d\n", **pptr);       // 42 (двойное разыменование)
```

NULL-указатель.

Специальное значение, которое означает, что указатель ни на что не указывает:

```
int *ptr = NULL; // Корректная инициализация «пустого» указателя

// Проверка на NULL перед использованием — признак хорошего тона
if (ptr != NULL) {
    printf("%d\n", *ptr);
} else {
    printf("Указатель не инициализирован!\n");
}
```

Попытка разыменовать NULL-указатель приводит к краху программы (сегфолту).

Итак, указатели и адреса необходимы:

1) для работы с массивами и строками;

2) передачи аргументов в функции по ссылке (чтобы функция могла изменить исходную переменную);

3) динамического выделение памяти (malloc, calloc, free);

4) эффективной работы со структурами данных (связные списки, деревья, графы);

5) работы с аппаратным обеспечением (в embedded-сис-темах), где регистры устройств отображаются на конкретные адреса памяти.

Понимание указателей — ключ к полноценному использованию всех возможностей языка Си.

3.7. РАБОТА С ФУНКЦИЯМИ В СИ. ОСОБЕННОСТИ ПЕРЕДАЧИ ДАННЫХ ПРИ ОБРАЩЕНИИ К ФУНКЦИИ В СИ

Объявление и определение функции.

Объявление (прототип) сообщает компилятору о существовании функции — ее имени, возвращаемом типе и типах параметров. Делается до main():

```
// Прототип функции
int sum(int a, int b); // <-- Точка с запятой обязательна
```

Определение: реализация функции — код, который выполняется при ее вызове:

```
// Определение функции
int sum(int a, int b) { // <-- Фигурные скобки
    int result = a + b;
    return result; // Возврат значения
}
```

Вызов: использование функции в коде:

```
int main() {
    int x = 5, y = 3;
    int s = sum(x, y); // Вызов функции
    printf("Sum = %d\n", s); // Вывод: Sum = 8
    return 0;
}
```

Особенности передачи данных в функции (табл. 3.13).

В языке Си все аргументы в функцию передаются по значению (by default). Это фундаментальная концепция.

Передача по значению (Pass by Value).

При вызове функции создаются копии значений аргументов. Функция работает с этими копиями. Оригинальные переменные не могут быть изменены внутри функции:

```
void increment(int num) {
    num = num + 1; // Меняется копия!
```

```
    printf("Inside function: %d\n", num); // 6
}

int main() {
    int a = 5;
    increment(a); // Передается копия значения переменной 'a'
    printf("In main: %d\n", a); // 5 — оригинал не изменился!
    return 0;
}
```

Таблица 3.13

Сводная таблица: передача данных в функции

Способ передачи	**Синтаксис**	**Что передается**	**Можно ли изменить оригинал**	**Применение**
По значению	func(int var)	Копия значения переменной	Нет	Передача простых данных (int, float, char), которые не должны меняться
По указателю	func(int *ptr)	Копия адреса переменной	Да (через разыменование *ptr)	Изменение оригинальных переменных, передача массивов и строк

Способ передачи	Синтаксис	Что передается	Можно ли изменить оригинал	Применение
По константному указателю	func(const int *ptr)	Копия адреса	Нет (значение read-only)	Эффективная передача больших структур (массивов) без возможности их изменения

Физически происходит следующее:

1. В *main* есть переменная *a* со значением 5.

2. При вызове *increment(a)* создается копия этого значения (5) и помещается в новую переменную *num* (параметр функции).

3. Функция *increment* работает только с этой копией (*num*).

4. После завершения функции копия уничтожается. Оригинальная переменная *a* осталась неизменной.

Плюсы: простота, защита исходных данных от случайного изменения.

Минусы: невозможность изменить исходные переменные, дополнительное время на создание копий (для больших структур).

Имитация передачи по ссылке (Pass by Reference).

Для того чтобы изменить оригинальные переменные, нужно передать в функцию не значения, а их адреса (указатели на оригинальные переменные). Это и есть имитация передачи по ссылке:

```
// Функция принимает не int, а указатели на int (int *)
void real_increment(int *num_ptr) { // Указатель как параметр
    *num_ptr = *num_ptr + 1; // РАЗЫМЕНОВАНИЕ: изменение значения по адресу
    printf("Inside function: %d\n", *num_ptr); // 6
```

```
}

int main() {
    int a = 5;
    real_increment(&a); // Передается АДРЕС переменной 'a' ( &a )
    printf("In main: %d\n", a); // 6 — оригинал изменен!
    return 0;
}
```

Физически происходит следующее:

1. В *main* есть переменная *a* со значением 5.

2. При вызове *real_increment(&a)* создается копия, но копируется не значение 5, а адрес переменной *a*.

3. Функция *real_increment* получает этот адрес и с помощью операции разыменования * напрямую изменяет значение, лежащее по этому адресу (т.е. оригинальную переменную *a*).

Передача массивов в функции.

При передаче массива в функцию на самом деле передается указатель на его первый элемент. Массив не копируется целиком (это эффективно):

```
// Функция для печати массива
// Аргумент 'arr' — это указатель на int (int *)
void printArray(int arr[], int size) { // Запись int arr[] эквивалентна int *arr
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]); // Работа с массивом через индексы
        // Это работает, так как arr[i] == *(arr + i)
    }
}

int main() {
    int myArray[5] = {1, 2, 3, 4, 5};
    printArray(myArray, 5); // Передается имя массива (адрес 1-го элемента) и размер
```

```
    return 0;
}
```

Размер массива теряется при передаче, поэтому его всегда нужно передавать отдельным параметром (size).

Так можно изменить массив внутри функции:

```
void modifyArray(int *arr, int size) {
    for (int i = 0; i < size; i++) {
        arr[i] *= 2; // Изменяет оригинальный массив!
    }
}
```

Возврат значений из функций.

Функция может вернуть только одно значение с помощью оператора *return*. Тип возвращаемого значения должен совпадать с объявленным.

Возврат простого типа:

```
int square(int x) {
    return x · x; // Возврат квадрата числа
}
```

Возврат указателя:

```
int* findMiddle(int *arr, int size) {
    return &arr[size/2]; // Возврат указателя на элемент массива
    // ВНИМАНИЕ: Нельзя возвращать указатель на локальную переменную!
}
```

Пример с const:

```
// Функция может читать данные по адресу, но не может их менять
void print_and_dont_change(const int *arr, int size) {
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
        // arr[i] = 0; // Эта строка вызовет ОШИБКУ компиляции!
    }
}
```

Рекомендации:

1. По умолчанию — передача по значению. Используйте для простых типов, когда изменение оригинала не требуется.

2. Чтобы изменить переменную, передавайте указатель на нее. Это основа работы с функциями, модифицирующими данные.

3. Массивы всегда передаются по указателю. Размер массива теряется — передавайте его отдельным параметром.

4. Для защиты данных используйте const. Если функция не должна менять массив или переменную, передавайте через const pointer.

5. Избегайте возврата указателей на локальные переменные. Они уничтожаются при выходе из функции, и указатель будет вести в «никуда».

3.8. СТРУКТУРЫ В СИ

Структуры в Си — это составной тип данных, позволяющий объединять под одним именем переменные разных типов. Они являются прообразом классов в объектно ориентированных языках (без методов).

Объявление структуры (создание шаблона).

Ключевое слово: *struct*.

Синтаксис:

```
struct <имя_тега> {
   <тип_1> <поле_1>;
   <тип_2> <поле_2>;
   // ...
   <тип_N> <поле_N>;
};
```

Пример. Создадим структуру для представления точки на плоскости и студента:

```
c
// Объявление структуры Point
```

```
struct Point {
    int x; // Поле x
    int y; // Поле y
};

// Объявление структуры Student
struct Student {
    char name[50]; // Поле name (массив символов)
    int student_id; // Поле student_id
    float gpa;      // Поле gpa
}; // <- ВНИМАНИЕ: точка с запятой после фигурной скобки ОБЯЗАТЕЛЬНА!
```

Определение переменных структуры.

Объявление структуры не выделяет память. Память выделяется при создании переменной этого типа.

Способы определения переменных:

```
// Способ 1: Отдельно от объявления
struct Point {
    int x;
    int y;
};

int main() {
    struct Point p1; // Определена переменная p1 типа struct Point
    struct Point p2; // Определена переменная p2 типа struct Point
    return 0;
}

// Способ 2: Сразу при объявлении структуры
struct Student {
    char name[50];
    int id;
```

```
} student1, student2; // Переменные student1 и student2 объявлены сразу

// Способ 3: Использование typedef (самый удобный и распространенный)
// Создает синоним для 'struct Point' с именем 'Point'
typedef struct {
    int x;
    int y;
} Point; // Теперь можно использовать 'Point' вместо 'struct Point'

int main() {
    Point p1; // Так короче и удобнее
    Point p2;
    return 0;
}
```

Доступ к полям структуры.

Для доступа к полям структуры используется оператор точки:

```
typedef struct {
    int x;
    int y;
} Point;

int main() {
    Point p1; // Переменная p1 не инициализирована, в ее полях «мусор»

    // Инициализация полей по отдельности
    p1.x = 10; // Обращение к полю x структуры p1
    p1.y = 20; // Обращение к полю y структуры p1

    printf("Coordinates: (%d, %d)\n", p1.x, p1.y); // Чтение полей
```

```
    // Возможна инициализация при объявлении (аналогично массивам)
    Point p2 = {5, 15}; // p2.x = 5, p2.y = 15
    Point p3 = { .y = 25, .x = 35 }; // Designated initializers (C99) — можно в любом порядке

    return 0;
}
```

Указатели на структуры и оператор →.

Часто структуры передаются в функции по указателю (чтобы избежать копирования всего содержимого). Для доступа к полям через указатель используется оператор → (стрелка):

```
typedef struct {
    int x;
    int y;
} Point;

// Функция, принимающая указатель на структуру
void printPoint(const Point *p) { // Указатель на константную структуру
    // Доступ через разыменование и оператор точки
    printf("Coordinates (dot): (%d, %d)\n", (*p).x, (*p).y);

    // Более удобный и частый способ — оператор стрелка ->
    printf("Coordinates (arrow): (%d, %d)\n", p→x, p→y);
}

void setPointZero(Point *p) { // Функция меняет оригинальную структуру
    p→x = 0; // Изменение поля через указатель
    p→y = 0;
}
```

```
int main() {
    Point my_point = {100, 200};
    printPoint(&my_point); // Передаем АДРЕС структуры
    setPointZero(&my_point);
    printPoint(&my_point); // Теперь (0, 0)
    return 0;
}
```

Размер структуры и выравнивание (Alignment).

Размер структуры не всегда равен сумме размеров ее полей. Это связано с выравниванием данных (alignment) для повышения производительности процессора:

```
struct Example {
    char a;    // 1 байт
            // Компилятор добавит 3 байта «пустоты» (padding) здесь...
    int b;     // 4 байта (должен быть выровнен по границе 4 байт)
    short c;   // 2 байта
            // ... и maybe 2 байта здесь, чтобы общий размер был кратен 4
};

int main() {
    printf("Size of char: %zu\n", sizeof(char));   // 1
    printf("Size of int: %zu\n", sizeof(int));     // 4
    printf("Size of short: %zu\n", sizeof(short)); // 2
    printf("Size of struct: %zu\n", sizeof(struct Example)); // Может быть 12, а не 1+4+2=7
    return 0;
}
```

Практический вывод: не стоит рассчитывать размер структуры «вручную». Всегда используйте оператор sizeof.

Структуры как элементы других структур (вложение).

Структуры могут содержать в себе другие структуры:

```
typedef struct {
    int x;
    int y;
} Point;

typedef struct {
    Point top_left;     // Поле типа Point
    Point bottom_right; // Еще одно поле типа Point
    char label[20];
} Rectangle;

int main() {
    Rectangle rect = {{0, 0}, {10, 20}, "My Rectangle"};

    // Доступ к вложенным полям
    printf("Top left X: %d\n", rect.top_left.x); // 0
    printf("Bottom right Y: %d\n", rect.bottom_right.y); // 20
    printf("Label: %s\n", rect.label); // My Rectangle

    return 0;
}
```

Массивы структур.

Структуры можно объединять в массивы:

```
#define NUM_STUDENTS 3

typedef struct {
    char name[50];
    int id;
} Student;

int main() {
```

```
    // Массив из 3 структур Student
    Student group[NUM_STUDENTS] = {
        {"Alice", 12345},
        {"Bob", 12346},
        {"Charlie", 12347}
    };

    // Доступ к элементу массива и его полю
    printf("Second student's name: %s\n", group[1].name); // Bob
    group[0].id = 99999; // Изменение поля первого студента

    // Перебор массива структур
    for (int i = 0; i < NUM_STUDENTS; i++) {
        printf("Student %d: %s (ID: %d)\n", i+1, group[i].name,
group[i].id);
    }

    return 0;
}
```

Резюме:

- структуры (struct) группируют данные разных типов;
- доступ к полям: через оператор «.» для переменных, и → для указателей;
- передача в функции: чаще всего передаются по указателю (для экономии памяти и возможности изменения);
- размер: определяется оператором sizeof, учитывает выравнивание;
- typedef — лучший друг для создания удобных псевдонимов.

3.9. МАССИВЫ И СТРОКИ В СИ

Массив — это упорядоченный набор элементов одного типа, хранящийся в непрерывной области памяти.

Объявление:

<тип_элементов> <имя_массива>[<размер>]; // *Размер должен быть целочисленной константой.*

int numbers[5]; *// Массив из 5 целых чисел (значения не определены)*

float temperatures[10]; *// Массив из 10 дробных чисел*

Инициализация:

// Способ 1: При объявлении с помощью фигурных скобок {}

int numbers[5] = {10, 20, 30, 40, 50}; *// Полная инициализация*

int partial[5] = {10, 20}; *// Остальные элементы будут 0: {10, 20, 0, 0, 0}*

int auto_size[] = {1, 2, 3}; *// Размер вычисляется автоматически (здесь 3)*

// Способ 2: Последующее присваивание (по элементу)

numbers[0] = 100;

numbers[1] = 200;

Доступ к элементам.

Доступ осуществляется по индексу (номеру элемента). Индексация начинается с «0!»:

int arr[3] = {100, 200, 300};

printf("First element: %d\n", arr[0]); *// 100*

printf("Second element: %d\n", arr[1]); *// 200*

arr[2] = 999; *// Изменение третьего элемента*

// Теперь массив: {100, 200, 999}

Важная особенность: отсутствие проверки границ.

Си не проверяет, выходит ли индекс за пределы массива, как, например, это делает Python. Это ответственность программиста:

int arr[3] = {1, 2, 3};

printf("%d\n", arr[5]); *// Неопределенное поведение! (Может вывести мусор, а может crash)*

Последствие: выход за границы массива — частая причина ошибок (багов) и уязвимостей безопасности.

Связь массивов и указателей.

Имя массива является указателем на его первый элемент:

```
int arr[3] = {1, 2, 3};
int *ptr = arr; // Эквивалентно int *ptr = &arr[0];

printf("First element: %d\n", *ptr); // 1 (разыменование указателя)
printf("Second element: %d\n", *(ptr + 1)); // 2 (арифметика указателей)
// *(ptr + i) эквивалентно arr[i]
```

Строки в Си (табл. 3.14).

В Си нет отдельного типа «строка». Строка — это массив символов (char), последний элемент которого равен специальному символу — терминатору '\0' (нуль-терминатор, код 0).

Таблица 3.14

Основные отличия и связь

Параметр	**Массив символов (строка)**	**Обычный массив (например, int)**
Тип элементов	char	Любой (int, float, etc.)
Критерий конца	Специальный символ '\0'	Размер массива (известный программисту)
Обработка	Часто обрабатывается как единое целое функциями (strlen, strcpy)	Обрабатывается поэлементно (через циклы)
Ввод-вывод	Специальные форматы %s, функции puts, fgets	Поэлементный ввод-вывод (циклы)

Объявление и инициализация.

Способ 1. Как массив символов (с явным указанием размера):

char name[20]; *// Массив для строки макс. длиной 19 символов + '\0'*

Способ 2. С инициализацией (компилятор сам добавит '\0'):

c

char greeting[] = "Hello"; *// Массив greeting[] = {'H','e','l','l','o','\0'}*

// Размер массива автоматически становится 6

Способ 3. Как указатель на строковый литерал (только для чтения):

char *message = "Hello World"; *// message указывает на область в памяти, где хранится "Hello World"*

// Попытка изменить: message[0] = 'h'; // ОШИБКА! Неопределенное поведение.

Ввод и вывод строк.

Вывод:

char str[] = "Hello";

printf("%s\n", str); *// Вывод строки до '\0'*

puts(str); *// Альтернатива, автоматически добавляет перевод строки*

Ввод (осторожно!):

char name[50];

printf("Enter your name: ");

scanf("%49s", name); *// Читает одно слово до пробела. 49 — чтобы не переполнить массив*

// Безопаснее: fgets(name, sizeof(name), stdin); // Читает всю строку с пробелами

Базовые функции для работы со строками (<string.h>**)**

Для работы со строками используются функции из стандартной библиотеки. Подключаем заголовок: #include <string.h>:

• strlen(str) — возвращает длину строки (без учета '\0'):

c

char s[] = "Hi";

int len = strlen(s); *// len = 2*

• strcpy(dest, src) — копирует строку src в dest:

c

```
char src[] = "Source";
char dest[20];
strcpy(dest, src); // dest теперь содержит "Source"
// Безопасная версия: strncpy(dest, src, sizeof(dest) — 1);
```

• strcat(dest, src) — добавляет (конкатенирует) строку src к концу строки dest:

c

```
char str1[20] = "Hello";
char str2[] = " World!";
strcat(str1, str2); // str1 теперь содержит "Hello World!"
// Безопасная версия: strncat(str1, str2, sizeof(str1) — strlen(str1) — 1);
```

• strcmp(str1, str2) — сравнивает две строки лексикографически:

– возвращает **0**, если строки равны;

– возвращает **< 0**, если str1 меньше str2;

– возвращает **> 0**, если str1 больше str2:

c

```
if (strcmp("abc", "abc") == 0) {
    printf("Strings are equal\n");
}
if (strcmp("abc", "abd") < 0) {
    printf("abc is less than abd\n");
}
```

Типичные ошибки.

Переполнение буфера:

```
char buf[5];
strcpy(buf, "Hello World"); // КАТАСТРОФА! Строка не помещается.
```

Решение: использовать безопасные функции (strncpy, snprintf) и проверять длину.

Забыть нуль-терминатор:

char s[3] = {'H', 'i'}; *// Массив { 'H', 'i', <мусор> }. Это НЕ строка!*

printf("%s\n", s); *// Будет выводить 'Hi' и мусор, пока не найдет случайный '\0'*

Решение: всегда явно добавлять '\0' или использовать инициализацию через двойные кавычки.

Сравнение строк через ==:

char s1[] = "hello";

char s2[] = "hello";

if (s1 == s2) { ... } *// Сравниваются АДРЕСА массивов, а не их содержимое! Всегда ложно.*

Решение: использовать strcmp().

Резюме:

- массивы — основа для хранения наборов данных;
- строки — это массивы символов, заканчивающиеся нуль-терминатором '\0';
- указатели и массивы тесно связаны. Имя массива — указатель на его начало;
- работа со строками ведется с помощью функций из <string.h>;
- внимание к границам массивов и размеру строковых буферов — залог безопасности и стабильности программы.

3.10. СТАНДАРТНЫЕ ФУНКЦИИ ВВОДА-ВЫВОДА В СИ

Стандартные функции ввода-вывода (табл. 3.15) в Си — это основа взаимодействия программы с пользователем и внешним миром. Они объявлены в заголовочном файле stdio.h (Standard Input/Output).

Таблица 3.15

Основные спецификаторы

Спецификатор	Тип данных	Пример
%d или %i	int (целые числа)	printf("%d", 42);
%f / %lf	float/double (дробные числа)	printf("%.2f", 3.14159);
%c	char (один символ)	printf("%c", 'A');
%s	char* (строка)	printf("%s", "Hello");
%p	void* (указатель)	printf("%p", &x);
%x / %X	int (шестнадцатеричный)	printf("%X", 255); // FF
%%	Символ %	printf("100%%");

Базовые функции вывода (Output):

• printf() — форматированный вывод.

Самая распространенная функция для вывода данных на стандартное устройство вывода (обычно консоль).

Синтаксис:

int printf(const char *format, ...);

– *format* — строка формата, содержащая текст и спецификаторы преобразования (%d, %f и т.д.);

– ... — переменное количество аргументов, значения которых подставляются вместо спецификаторов.

Пример:

int age = 25;

float height = 1.75f;

printf("Мне %d лет, мой рост — %.2f метра.\n", age, height);

// Вывод: Мне 25 лет, мой рост — 1.75 метра.

• puts() **— вывод строки.**

Выводит строку и автоматически добавляет символ перевода строки (\n).

Синтаксис:

int puts(const char *str);

Пример:

c

puts("Hello,"); *// Выведет "Hello," и переведет строку*

puts("world!"); *// Выведет "world!" и переведет строку*

Плюсы: проще и быстрее printf для вывода простых строк.

Минус: не позволяет форматировать вывод;

• putchar() — вывод одного символа

Выводит один символ на стандартный вывод.

Синтаксис:

int putchar(int c);

Пример:

putchar('A'); *// Выведет символ 'A'*

putchar('\n'); *// Выведет символ новой строки*

putchar(65); *// Также выведет 'A' (ASCII-код)*

Базовые функции ввода (Input) (табл. 3.16):

• scanf() **—** форматированный ввод.

Читает данные со стандартного ввода (обычно клавиатура) в соответствии с форматом.

Синтаксис:

int scanf(const char *format, ...);

– format — строка формата со спецификаторами;

– ... — адреса переменных, куда будут помещены считанные данные (**обязательно передавать адреса!**).

Пример:

int num;

float f;

char str[100];

printf("Введите целое число: ");
scanf("%d", &num); // & — *оператор взятия адреса*

printf("Введите дробное число: ");
scanf("%f", &f);

printf("Введите строку (без пробелов): ");
scanf("%s", str); // & *не нужен, т.к. str — массив (указатель)*
// ВНИМАНИЕ: %s читает до первого пробела / табуляции / новой строки.

Важно! scanf — опасная функция. Она не проверяет границы буфера:

char buf[5];
scanf("%s", buf); *// Если ввести "HelloWorld", произойдет переполнение буфера!*

Решение. Использовать модификатор ширины:

scanf("%4s", buf); *// Считает не более 4 символов (+ '\0' добавится пятым)*

• fgets() — безопасное чтение строки.

Читает строку из потока (включая пробелы) до символа новой строки или конца файла. Предпочтительный способ для чтения строк.

Синтаксис:

char *fgets(char *str, int n, FILE *stream);

– str — указатель на буфер (массив), куда будет помещена строка;

– n — максимальное количество символов для чтения (включая terminating '\0');

– stream — указатель на поток. Для stdin используется stdin.

Пример:

char name[50];
printf("Введите ваше имя: ");
fgets(name, sizeof(name), stdin); *// Безопасное чтение до 49 символов*

// fgets сохраняет символ '\n' в конце строки. Часто его удаляют:

name[strcspn(name, "\n")] = '\0'; *// Находим '\n' и заменяем на '\0'*

• getchar() — чтение одного символа.

Читает один символ из стандартного потока ввода.

Синтаксис:

int getchar(void);

Пример (очистка буфера ввода):

c

int c;

while ((c = getchar()) != '\n' && c != EOF); *// Читает все символы до новой строки*

Работа с файлами.

Те же функции имеют аналоги для работы с файлами. Для этого нужно объявить указатель на файл (FILE*).

Открытие файла:

FILE *fopen(const char *filename, const char *mode);

mode: "r" (чтение), "w" (запись), "a" (дозапись), "r+" (чтение и запись).

Пример записи в файл:

```
FILE *file = fopen("data.txt", "w");
if (file != NULL) {
   fprintf(file, "Hello, File!\n"); // Аналог printf для файлов
   fputs("Another line\n", file);   // Аналог puts для файлов
   fclose(file); // Закрытие файла обязательно!
}
```

Пример чтения из файла:

```
FILE *file = fopen("data.txt", "r");
char buffer[100];
if (file != NULL) {
   while (fgets(buffer, sizeof(buffer), file) != NULL) { // Чтение по-
строчно
        printf("%s", buffer);
```

```
    }
    fclose(file);
}
```

Таблица 3.16

Сравнительная таблица ввода

Функция	**Преимущества**	**Недостатки**	**Рекомендация**
scanf()	Гибкий форматированный ввод	Опасна из-за переполнения буфера	Использовать с указанием ширины (%9s)
fgets()	Безопасное чтение строк с пробелами	Сохраняет символ \n в строке	Основной способ для чтения строк
getchar()	Простое чтение одного символа	Не для сложного ввода	Для очистки буфера или меню

КОНТРОЛЬНЫЕ ВОПРОСЫ ПО ТЕМЕ «СТРУКТУРА ПРОГРАММЫ И ОСНОВНЫЕ КОНСТРУКЦИИ ЯЗЫКА СИ»

Базовые понятия и структура программы:

1. Из каких основных частей состоит минимальная программа на Си? Приведите пример.

2. Что такое директива препроцессора #include? Для чего она нужна и как работает?

3. Объясните назначение функции main(). Почему она является обязательной? Какие варианты объявления main вы знаете?

4. Что такое комментарии? Какие два способа записи комментариев существуют в Си? Для чего они используются?

Переменные, типы данных и объявления:

5. Что такое переменная? Из каких частей состоит объявление переменной?

6. Перечислите основные базовые типы данных в Си (int, float, char и т.д.). Какой размер в байтах занимает каждый из них?

7. Что такое квалификаторы const и volatile? Где и для чего они применяются?

8. Чем отличается инициализация переменной от присваивания?

9. Что такое область видимости (scope) переменной? Какие виды областей видимости вы знаете?

Операторы:

10. Какие группы операторов существуют в Си (арифметические, отношения, логические и т.д.)?

11. Чем отличается оператор = от оператора ==?

12. Что такое операция инкремента (++) и декремента (--)? В чем разница между префиксной и постфиксной формой?

13. Для чего используются операторы +=, -=, *= и другие составные операторы присваивания?

14. Как работает оператор «запятая» (,) и где он применяется?

Управляющие конструкции:

15. Опишите работу условного оператора if-else. Как работает конструкция else if?

16. Для чего используется оператор многозначного выбора switch? Чем он принципиально отличается от if-else? Почему в case-ветках часто используют break?

17. Перечислите все три вида циклов в Си (while, do-while, for) и опишите их синтаксис.

18. В каких случаях используется цикл while, а в каких — for?

19. В чем уникальность цикла do-while?

20. Для чего используются операторы break и continue? В чем их основное различие?

Функции:

21. Что такое функция? Из каких основных частей состоит объявление (прототип) функции?

22. Объясните разницу между объявлением (прототипом) и определением функции.

23. Что такое формальные и фактические параметры функции?

24. Как в Си передаются аргументы в функцию по умолчанию? (По значению или по ссылке?)

25. Как можно реализовать передачу параметров по ссылке в Си? Какой механизм для этого используется?

26. Что такое возвращаемое значение функции? Какой тип возвращает функция, если он не указан? Можно ли объявить функцию, которая ничего не возвращает?

27. Что такое рекурсия? Какое обязательное условие должно быть у рекурсивной функции?

Препроцессор:

28. Как работает директива #define? Приведите пример объявления простой константы и макроса с параметрами.

29. Чем макрос с параметрами отличается от функции? Назовите его преимущества и недостатки.

30. Для чего используются условные директивы компиляции #ifdef, #ifndef, #endif?

Практические задания на понимание кода:

31. **Что выведет на экран следующая программа?**

```
#include <stdio.h>

int main() {
    int x = 5;
    if (x = 10) {
        printf("x is 10\n");
    } else {
        printf("x is not 10\n");
    }
    return 0;
}
```

Сколько раз выполнится цикл?

c

```
int i = 0;
while (i < 5) {
    printf("%d ", i);
    i++;
}
```

32. **Найдите ошибки в коде:**

c

```
#include <stdio.h>

int main() {
    int a = 10
    int b = 20;
    printf("Sum: %d\n, a + b);
    return 0;
}
```

33. **Что делает эта функция?**

c

```
int mystery(int n) {
    if (n == 0) {
        return 1;
    } else {
        return n * mystery(n — 1);
    }
}
```

Вопросы на знание стандартных функций:

34. Для чего нужна стандартная библиотека Си? Как подключить ее функции?

35. Какие функции из stdio.h используются для ввода и вывода данных?

36. Какие функции из math.h вы знаете (sqrt, pow, sin и т.д.)?

ЛАБОРАТОРНЫЕ ЗАНЯТИЯ

ЛАБОРАТОРНОЕ ЗАНЯТИЕ № 1
ВЫПОЛНЕНИЕ ЛОГИЧЕСКИХ И АРИФМЕТИЧЕСКИХ КОМАНД МИКРОПРОЦЕССОРА

Работа может быть адаптирована для выполнения на реальном процессоре (например, на Arduino/AVR) или в симуляторе (например, для x86 или учебных архитектур).

Цель: изучить систему команд микропроцессора, освоить выполнение арифметических и логических операций на языке ассемблера. Исследовать влияние операций на флаги состояния процессора.

Теоретическая часть:

1. Арифметические команды:

- ADD — сложение;
- SUB — вычитание;
- INC/DEC — инкремент/декремент;
- MUL/DIV — умножение/деление (для некоторых архитектур);
- NEG — изменение знака (дополнение до двух).

2. Логические команды:

- AND — логическое И (побитовое);
- OR — логическое ИЛИ (побитовое);
- XOR — исключающее ИЛИ (побитовое);
- NOT — инверсия (побитовое);
- TEST — логическое сравнение (аналогично AND, но без сохранения результата, только флаги).

3. Команды сдвига и циклического сдвига:

- SHL/SHR — логический сдвиг влево/вправо;
- SAL/SAR — арифметический сдвиг влево/вправо (SAR сохраняет знак);
- ROL/ROR — циклический сдвиг влево/вправо;

• RCL/RCR — циклический сдвиг через флаг переноса CF.

4. Флаги состояния процессора (на примере x86):

• CF (Carry Flag) — флаг переноса. Устанавливается в 1, если при операции возник перенос из старшего бита или заем;

• ZF (Zero Flag) — флаг нуля. Устанавливается в 1, если результат операции равен нулю;

• SF (Sign Flag) — флаг знака. Равен значению старшего бита результата (1 для отрицательных чисел в дополнительном коде);

• OF (Overflow Flag) — флаг переполнения. Устанавливается в 1, если результат операции не поместился в разрядную сетку и исказил знак.

Практическая часть.

Задание 1. Арифметические операции:

1. Загрузите в регистры AX и BX значения 0x7FF0 и 0x10.

2. Выполните команду ADD AX, BX.

3. Запишите результат и определите состояние флагов CF, ZF, SF, OF. Объясните полученные значения флагов.

4. Повторите пункты 1–3 для команды SUB AX, BX с теми же исходными значениями.

Задание 2. Логические операции:

1. Загрузите в регистры AL и BL значения 0b10101010 и 0b00001111.

2. Последовательно выполните команды AND AL, BL, OR AL, BL, XOR AL, BL, NOT AL (восстанавливая исходное значение AL перед каждой новой операцией).

3. Для каждой операции запишите результат в двоичном и шестнадцатеричном виде.

4. Объясните, какая операция может использоваться для обнуления отдельных битов, а какая — для их инверсии.

Задание 3. Операции сдвига:

1. Загрузите в регистр AX значение 0x00A5 (что соответствует 0b0000000010100101).

2. Выполните команду SHL AX, 1. Запишите результат и состояние флага CF.

3. Выполните команду SHR AX, 2. Запишите результат и состояние флага CF.

4. Объясните, как с помощью операций сдвига можно реализовать умножение и деление на степени числа 2.

Задание 4. Комбинированная задача.

Напишите программу на ассемблере, которая:

1) вводит два однобайтовых числа (или берет их из заранее заданных ячеек памяти);

2) вычисляет выражение F = (A AND B) XOR (NOT C), где C = A – B;

3) сохраняет результат F в память или выводит его;

4) по ходу вычисления анализирует и записывает состояния флагов после ключевых операций.

Содержание отчета:

1. Титульный лист (название работы, ФИО, группа).

2. Цель работы.

3. Краткие теоретические сведения об арифметических и логических командах, флагах состояния.

4. Листинг программ для каждого задания с комментариями.

5. Результаты выполнения программ:

• таблицы с исходными данными, полученными результатами и состояниями флагов;

• объяснение полученных результатов и значений флагов.

6. Выводы по работе: что изучили, какие команды освоили, как влияют операции на флаги.

Пример кода (для x86 в emu8080 или подобном симуляторе):

asm

```
; Задание 1: Арифметические операции
mov ax, 7FF0h ; Загружаем значение в AX
mov bx, 0010h ; Загружаем значение в BX
add ax, bx    ; Складываем AX и BX, результат в AX
```

; Здесь состояние флагов: CF=0, ZF=0, SF=0, OF=1 (переполнение)

; Результат: AX = 8000h

; Задание 2: Логические операции

mov al, 0AAh *; 0b10101010*

mov bl, 0Fh *; 0b00001111*

and al, bl *; AL = 0Ah (0b00001010)*

or al, bl *; AL = 0Fh (0b00001111)*

xor al, bl *; AL = 00h (0b00000000)*

not al *; AL = FFh (0b11111111)*

Контрольные вопросы:

1. Какая арифметическая команда не изменяет свой первый операнд?

2. Как с помощью логической операции проверить, установлен ли конкретный бит в числе?

3. Чем отличается логический сдвиг от арифметического?

4. В каком случае устанавливается флаг переполнения (OF)?

5. Для чего используется команда TEST? Приведите пример.

Варианты заданий.

Варианты могут различаться начальными значениями регистров или самими выражениями для вычисления.

Пример варианта:

- задание 1. AX = 0x8A00, BX = 0x7600. Выполнить ADD и SUB;
- задание 2. AL = 0x55, BL = 0xF0. Выполнить логические операции;
- задание 3. Исходное значение AX = 0x00F1. Выполнить SHL AX, 2 и SAR AX, 1;
- задание 4. Вычислить F = (A OR B) + (A XOR B).

ЛАБОРАТОРНОЕ ЗАНЯТИЕ № 2
ВЫПОЛНЕНИЕ ЦИКЛИЧЕСКИХ КОНСТРУКЦИЙ И ОПЕРАТОРОВ ВЕТВЛЕНИЯ НА ЯЗЫКЕ СИ

Цель: изучить и практически освоить использование операторов ветвления (if-else, switch-case) и циклов (while, do-while, for) для управления потоком выполнения программы. Научиться анализировать алгоритмы и предсказывать их поведение.

Теоретическая часть:

1. Операторы ветвления:

• if / else if / else — условное выполнение блока кода в зависимости от истинности условия;

• switch/case/default — многозначное ветвление, выбор варианта выполнения на основе значения выражения.

2. Циклические конструкции:

• while — цикл с предусловием. Выполняется, пока условие истинно;

• do-while — цикл с постусловием. Тело выполняется по меньшей мере один раз, затем проверяется условие;

• for — цикл со счетчиком. Содержит инициализацию, условие и изменение счетчика в одной строке.

3. Вспомогательные операторы:

• break — немедленный выход из цикла или оператора switch;

• continue — переход к следующей итерации цикла с пропуском оставшегося кода тела.

Практическая часть.

Задание 1. Оператор ветвления if-else.

Напишите программу, которая:

1. Запрашивает у пользователя два целых числа.

2. С помощью оператора if-else определяет и выводит результат:

• какое из чисел больше;

• являются ли числа равными;

• являются ли числа четными или нечетными.

Пример работы программы:

text

Введите первое число: 8

Введите второе число: 5

Первое число (8) больше второго (5).

Числа не равны.

Первое число — четное. Второе число — нечетное.

Задание 2. Оператор многозначного выбора switch-case.

Напишите программу-калькулятор, которая:

1) запрашивает у пользователя два числа и символ операции (+, –, *, /);

2) используя оператор switch-case, выполняет соответствующую арифметическую операцию;

3) выводит результат (учтите возможность деления на ноль).

Пример работы программы:

text

Введите первое число: 10

Введите операцию (+, –, *, /): *

Введите второе число: 5

Результат: 10 * 5 = 50

Задание 3. Цикл while.

Напишите программу, которая:

1) запрашивает у пользователя положительное целое число N;

2) используя цикл while, вычисляет сумму всех целых чисел от 1 до N включительно;

3) выводит полученную сумму.

Пример работы программы:

text

Введите положительное число N: 100

Сумма чисел от 1 до 100 = 5 050

Задание 4. Цикл for.

Напишите программу, которая:

1) используя цикл for, выводит на экран таблицу умножения для числа, введенного пользователем (например, таблицу умножения на 7);

2) выводит результат, оформленный в виде:

text

7 × 1 = 7

7 × 2 = 14

...

7 × 10 = 70

Задание 5. Цикл do-while **и оператор** continue.

Напишите программу, которая:

1) используя цикл do-while, запрашивает у пользователя числа до тех пор, пока не будет введен ноль;

2) для каждого введенного числа проверяет, является ли оно отрицательным. Если да, использовать continue, чтобы перейти к следующей итерации, пропуская подсчет;

3) подсчитывает сумму только положительных чисел;

4) после ввода нуля выводит полученную сумму.

Пример работы программы:

text

Введите число (0 для выхода): 5

Введите число (0 для выхода): –3

Введите число (0 для выхода): 10

Введите число (0 для выхода): 0

Сумма положительных чисел: 15

Содержание отчета:

1. Титульный лист (название работы, ФИО, группа, дата).

2. Цель работы.

3. Краткие теоретические сведения об операторах ветвления и циклах.

4. Листинги программ для каждого задания с подробными комментариями.

5. Результаты выполнения программ:

• примеры запуска с разными входными данными (скриншоты или копии вывода консоли);

• для заданий 3–5 рекомендуется привести трассировку цикла (значения переменных на каждой итерации) для небольшого N.

6. Выводы по работе: что получилось, с какими трудностями столкнулись, как усвоен материал.

Примеры кода.

Для задания 1 (if-else**):**

```
#include <stdio.h>

int main() {
    int a, b;
    printf("Введите первое число: ");
    scanf("%d", &a);
    printf("Введите второе число: ");
    scanf("%d", &b);

    // Сравнение двух чисел
    if (a > b) {
        printf("Первое число (%d) больше второго (%d).\n", a, b);
    } else if (a < b) {
        printf("Второе число (%d) больше первого (%d).\n", b, a);
    } else {
        printf("Числа равны.\n");
    }

    // Проверка на четность
    if (a % 2 == 0) {
        printf("Первое число — четное.\n");
    } else {
        printf("Первое число — нечетное.\n");
    }
```

```
    if (b % 2 == 0) {
        printf("Второе число — четное.\n");
    } else {
        printf("Второе число — нечетное.\n");
    }

    return 0;
}
```

Для задания 3 (while**):**

```
#include <stdio.h>

int main() {
    int n, i = 1, sum = 0;
    printf("Введите положительное число N: ");
    scanf("%d", &n);

    if (n <= 0) {
        printf("Ошибка: требуется положительное число.\n");
        return 1;
    }

    while (i <= n) {
        sum += i;
        i++;
    }
    printf("Сумма чисел от 1 до %d = %d\n", n, sum);
    return 0;
}
```

Контрольные вопросы:

1. В чем основное различие между циклом while и do-while?
2. Как оператор break работает внутри вложенных циклов?
3. Всегда ли можно заменить цикл for на цикл while и наоборот?

4. В каких ситуациях предпочтительнее использовать оператор switch вместо if-else?

5. Для чего используется оператор continue? Приведите пример.

Критерии оценки:

• *отлично:* все программы работают корректно, отчет оформлен аккуратно, приведены примеры работы, даны полные ответы на контрольные вопросы;

• *хорошо:* программы работают, но есть незначительные недочеты в оформлении или выводе результатов;

• *удовлетворительно:* программы в основном работают, но есть ошибки в логике или оформлении.

ЛАБОРАТОРНОЕ ЗАНЯТИЕ № 3
РАБОТА С ЦИФРОВЫМИ ПОРТАМИ ВВОДА-ВЫВОДА МИКРОКОНТРОЛЛЕРА

Цель: изучить структуру порта ввода-вывода и принципы управления портами ввода-вывода общего назначения (GPIO). Освоить программирование регистров управления портами для работы на вход и выход. Практически реализовать чтение цифровых сигналов и управление внешними устройствами.

Теоретическая часть:

1. Структура порта ввода-вывода.

Цифровой порт микроконтроллера обычно состоит из нескольких одноразрядных портов, объединенных в 8-разрядные (реже в 16- или 32-разрядные) регистры. Ключевые регистры для управления:

• DDRx (Data Direction Register) — регистр направления данных. Определяет, работает ли вывод порта на вход или выход:

– 0 — вывод сконфигурирован как вход;

– 1 — вывод сконфигурирован как выход;

• PORTx (Data Register) — регистр данных:

– в режиме выхода: определяет уровень сигнала на выводе:

○ 0 — низкий уровень (0 В);

○ 1 — высокий уровень (Vcc, например, 5 В или 3.3 В);

– в режиме входа: управляет подключением подтягивающего резистора:

○ 0 — подтягивающий резистор отключен (высокоимпедансное состояние);

○ 1 — подтягивающий резистор подключен к Vcc;

• PINx (Port Input Pins Register) — регистр для чтения текущего логического состояния выводов порта (актуально, когда вывод сконфигурирован как вход).

2. Битовая арифметика

Для управления отдельными выводами порта используются побитовые операции:

• установка бита (bit set): REG |= (1 << n);

• сброс бита (bit clear): REG &= ~(1 << n);

• переключение бита (bit toggle): REG ^= (1 << n);

• проверка бита (bit check): if (REG & (1 << n)) {...}.

Практическая часть (на примере AVR ATmega328P).

Задание 1. Мигающий светодиод (основы работы на выход):

1. Подключите светодиод через токоограничивающий резистор (220–330 Ом) к выводу PD0 (Digital Pin 0) микроконтроллера.

2. Напишите программу, которая циклически включает и выключает светодиод с задержкой примерно 500 мс.

3. Алгоритм:

• сконфигурировать вывод PD0 на выход (записать 1 в соответствующий бит регистра DDRD);

• в бесконечном цикле:

– установить высокий уровень на PD0 (записать 1 в PORTD);

– вызвать функцию задержки;

– установить низкий уровень на PD0 (записать 0 в PORTD);

– вызвать функцию задержки.

Пример кода:

```
#include <avr/io.h>
#include <util/delay.h>

int main(void) {
   DDRD |= (1 << DDD0);   // Настроить PD0 на выход

   while (1) {
      PORTD |= (1 << PORTD0);  // Включить светодиод (HIGH)
      _delay_ms(500);
      PORTD &= ~(1 << PORTD0); // Выключить светодиод (LOW)
      _delay_ms(500);
   }
   return 0;
}
```

Задание 2. Управление светодиодом кнопкой (основы работы на вход):

1. Подключите тактовую кнопку между выводом **PD2** и землей (GND). Подключите подтягивающий резистор к Vcc (внутренний или внешний на 10 кОм).

2. Подключите светодиод к выводу PD0.

3. Напишите программу, которая включает светодиод только тогда, когда кнопка нажата.

4. Алгоритм:

- сконфигурировать PD0 на выход;
- сконфигурировать PD2 на вход. Активировать внутренний подтягивающий резистор для PD2;
- в бесконечном цикле постоянно опрашивать состояние кнопки (читать бит PIND2);
- если кнопка нажата (логический 0 на входе, так как кнопка подключена к GND), установить высокий уровень на светодиоде;

• если кнопка отпущена (логическая 1 благодаря подтяжке), установить низкий уровень на светодиоде.

Пример кода:

```
#include <avr/io.h>

int main(void) {
    DDRD |= (1 << DDD0);    // PD0 — выход
    DDRD &= ~(1 << DDD2);   // PD2 — вход
    PORTD |= (1 << PORTD2); // Включить подтяжку для PD2

    while (1) {
        if (!(PIND & (1 << PIND2))) { // Если кнопка нажата (логический 0)
            PORTD |= (1 << PORTD0);   // Включить светодиод
        } else {
            PORTD &= ~(1 << PORTD0);  // Выключить светодиод
        }
    }
    return 0;
}
```

Задание 3. «Бегущий огонь» (работа с несколькими выводами):

1. Подключите 4 светодиода к выводам **PD0, PD1, PD2, PD3**.

2. Напишите программу, реализующую эффект «бегущего огня» (Knight Rider) с задержкой между переключениями.

3. Алгоритм:

• сконфигурировать выводы PD0–PD3 на выход;

• в бесконечном цикле последовательно «двигать» единицу по этим выводам.

Пример кода (фрагмент):

```
// Настройка портов
DDRD |= (1 << DDD0) | (1 << DDD1) | (1 << DDD2) | (1 << DDD3);
```

```
uint8_t i = 0;
while (1) {
   // Очистить все биты
   PORTD &= ~((1 << PORTD0) | (1 << PORTD1) | (1 << PORTD2) | (1 << PORTD3));
   // Установить один бит в позиции i
   PORTD |= (1 << i);
   _delay_ms(200);
   i++;
   if (i > 3) i = 0;
}
```

Содержание отчета:

1. Титульный лист.
2. Цель работы.
3. Краткие теоретические сведения о регистрах GPIO (DDRx, PORTx, PINx).
4. Принципиальная схема подключения светодиодов и кнопок (фото или скриншот из симулятора).
5. Листинги программ для всех заданий с комментариями.
6. Результаты выполнения: описание работы схемы, возникшие проблемы и их решение.
7. Ответы на контрольные вопросы.
8. Выводы по работе.

Контрольные вопросы:

1. Каково назначение регистров DDRx, PORTx и PINx?
2. Как сконфигурировать вывод порта на вход с подключенным внутренним подтягивающим резистором?
3. Почему при подключении кнопки к выводу, сконфигурированному на вход с подтяжкой, нажатие кнопки читается как логический ноль?
4. Чем опасна работа выхода на короткое замыкание? Какой элемент схемы защищает светодиод?

5. Как можно реализовать антидребезг кнопки программным способом?

Варианты заданий:

• вариант 1. Реализовать светофор (три светодиода) с заданной последовательностью;

• вариант 2. Реализовать двоичный счетчик нажатий кнопки, выводящий значение на 4 светодиода;

• вариант 3. Управлять яркостью светодиода с помощью ШИМ (если изучалось) по нажатию кнопки.

Критерии оценки:

• *отлично:* схема собрана верно, все программы работают, отчет содержит полные ответы на вопросы;

• *хорошо:* есть незначительные ошибки в схеме или коде, исправленные в процессе работы;

• *удовлетворительно:* программы работают с помощью преподавателя, отчет оформлен неполно;

• *неудовлетворительно:* схема неработоспособна, программы не написаны.

ЛАБОРАТОРНОЕ ЗАНЯТИЕ № 4
ОРГАНИЗАЦИЯ ЦИКЛОВ И ВРЕМЕННЫХ ЗАДЕРЖЕК В МИКРОКОНТРОЛЛЕРАХ

Цель: изучить принципы организации программных задержек с использованием циклов. Освоить технику создания точных временных интервалов. Научиться управлять длительностью задержки с помощью регистров таймеров и программных циклов.

Теоретическая часть:

1. Программные задержки на основе циклов.

Простейший способ создания задержки — использование пустых циклов, которые «крутятся» заданное количество раз, потребляя такты процессора.

Длительность задержки зависит:

- от тактовой частоты микроконтроллера (F_CPU);
- количества итераций цикла;
- количества тактов, затрачиваемых на одну итерацию.

2. Использование таймеров/счетчиков.

Для создания точных и неблокирующих задержек используются аппаратные таймеры. Принцип:

- таймер настраивается на определенный режим работы (обычный, CTC);
- вычисляется значение регистра сравнения для получения нужной частоты прерываний;
- в обработчике прерывания таймера инкрементируется счетчик;
- главный цикл программы проверяет значение счетчика.

3. Функции задержки из стандартных библиотек.

Многие инструментальные среды (Arduino, Atmel Studio, MPLAB) предоставляют готовые функции задержки:

- Arduino: delay(), delayMicroseconds();
- AVR-GCC: _delay_ms(), _delay_us() (требуют определения F_CPU).

Практическая часть (на примере AVR ATmega).

Задание 1. Программная задержка с помощью цикла for:

1. Создайте проект для микроконтроллера ATmega328P.

2. Подключите светодиод к выводу PD0.

3. Реализуйте мигание светодиода с периодом 2 с (1 с — включен, 1 с — выключен), используя **пустой цикл for** для создания задержки.

4. **Алгоритм:**

- настройте вывод светодиода на выход;
- в бесконечном цикле переключайте состояние светодиода;
- используйте вложенные циклы for для создания паузы.

Пример кода (для AVR-GCC):

```
#include <avr/io.h>

#define LED_PIN PD0

void delay_ms(uint16_t ms) {
    for (uint16_t i = 0; i < ms; i++) {
        for (volatile uint16_t j = 0; j < 4000; j++) { // Подобрать экспериментально
            __asm__ __volatile__ ("nop");
        }
    }
}

int main(void) {
    DDRD |= (1 << LED_PIN);

    while (1) {
        PORTD ^= (1 << LED_PIN); // Инвертировать состояние светодиода
        delay_ms(1000);
    }
}
```

Задание 2. Точная задержка с использованием функции _delay_ms():

1. Используйте функции из библиотеки util/delay.h.

2. Реализуйте мигание светодиода с периодом 500 мс.

3. Убедитесь, что частота тактирования (F_CPU) определена правильно.

Пример кода:

```
#include <avr/io.h>
#include <util/delay.h>
```

```
#define LED_PIN PD0

int main(void) {
    DDRD |= (1 << LED_PIN);

    while (1) {
        PORTD ^= (1 << LED_PIN);
        _delay_ms(500);
    }
}
```

Задание 3. Реализация эффекта «дыхания» светодиода с помощью ШИМ и циклов:

1. Используйте циклы для изменения скважности ШИМ-сигнала.

2. Плавно увеличивайте и уменьшайте яркость светодиода.

Алгоритм.

Используйте два вложенных цикла:

- внешний цикл — для управления яркостью;
- внутренние циклы — для генерации ШИМ-сигнала.

Пример кода (фрагмент):

```
for (uint8_t brightness = 0; brightness < 255; brightness++) {
    for (uint16_t i = 0; i < 1000; i++) {
        if (i < brightness) {
            PORTD |= (1 << LED_PIN); // Включить
        } else {
            PORTD &= ~(1 << LED_PIN); // Выключить
        }
        _delay_us(10);
    }
}
```

Задание 4. Создание неблокирующей задержки с использованием системного таймера:

1. Настройте таймер на прерывание каждую 1 мс.

2. Создайте глобальную переменную-счетчик.
3. Реализуйте функцию проверки времени.

Пример кода:

```
#include <avr/io.h>
#include <avr/interrupt.h>

volatile uint32_t millis = 0;

ISR(TIMER0_COMPA_vect) {
    millis++;
}

void timer0_init() {
    TCCR0A = (1 << WGM01); // CTC mode
    TCCR0B = (1 << CS01) | (1 << CS00); // Prescaler 64
    OCR0A = 250 — 1; // Compare value
    TIMSK0 = (1 << OCIE0A); // Enable interrupt
}

int main(void) {
    timer0_init();
    sei();

    uint32_t previous_time = 0;

    while (1) {
        if (millis — previous_time >= 1000) {
            previous_time = millis;
            // Выполнять действие каждую секунду
        }
        // Здесь может выполняться другой код
    }
}
```

Содержание отчета:

1. Титульный лист.
2. Цель работы.
3. Теоретическая часть с описанием методов создания задержек.
4. Схема подключения светодиодов.
5. Листинги программ с комментариями.
6. Результаты экспериментов:

• измеренные временные параметры;

• осциллограммы сигналов (если возможно).

7. Ответы на контрольные вопросы.
8. Выводы.

Контрольные вопросы:

1. В чем недостатки программных задержек на основе циклов?
2. Как зависит длительность задержки от тактовой частоты процессора?
3. Почему важно использовать volatile для переменных в циклах задержки?
4. Чем неблокирующая задержка лучше блокирующей?
5. Как рассчитывается значение для регистра сравнения таймера?

Критерии оценки:

• *отлично:* все программы работают корректно, реализованы все задания, отчет содержит анализ точности задержек;

• *хорошо:* небольшие погрешности в длительностях задержек;

• *удовлетворительно:* работает только часть заданий;

• *неудовлетворительно:* программы не работают.

Дополнительное задание.

Реализуйте многозадачность с помощью конечного автомата и неблокирующих задержек:

• мигание светодиода 1 с периодом 1 с;

• мигание светодиода 2 с периодом 300 м;

• опрос кнопки с антидребезгом.

Все задачи должны работать параллельно в одном цикле.

ЛАБОРАТОРНОЕ ЗАНЯТИЕ № 5
ОРГАНИЗАЦИЯ ПОДПРОГРАММ В ЯЗЫКЕ СИ

Цель: изучить принципы модульного программирования. Освоить techniques создания и использования функций. Научиться передавать параметры в функции и возвращать результаты. Изучить области видимости переменных.

Теоретическая часть:

1. Понятие подпрограммы.

Подпрограмма (функция) — это логически завершенный фрагмент кода, который можно многократно вызывать из различных частей программы. Функции позволяют:

- структурировать программу;
- уменьшать дублирование кода;
- повышать читаемость и сопровождаемость;
- создавать многократно используемые компоненты.

2. Структура функции:

```
<возвращаемый_тип> <имя_функции>(<параметры>) {
    <тело_функции>
    return <значение>;
}
```

3. Передача параметров:

- по значению — создается копия параметра;
- по указателю — передается адрес переменной;
- по ссылке (в C++) — передается ссылка на переменную.

4. Области видимости:

- локальные переменные — видны только внутри функции;
- глобальные переменные — видны во всей программе;
- статические переменные — сохраняют значение между вызовами.

Практическая часть.

Задание 1. Создание простых функций.

Создайте программу, содержащую следующие функции:

1) функцию приветствия:

```
void print_hello() {
    printf("Hello, World!\n");
}
```

2) функцию сложения двух чисел:

```
int add(int a, int b) {
    return a + b;
}
```

3) функцию проверки числа на четность:

```
int is_even(int number) {
    return number % 2 == 0;
}
```

Пример использования:

```
int main() {
    print_hello();

    int result = add(5, 3);
    printf("5 + 3 = %d\n", result);

    if (is_even(4)) {
        printf("4 is even\n");
    }

    return 0;
}
```

Задание 2. Функции с передачей параметров по указателю.

Создайте функции для работы с двумя числами:

1) функцию обмена значений:

```
void swap(int *a, int *b) {
    int temp = *a;
```

```
    *a = *b;
    *b = temp;
}
```
2) функцию вычисления суммы и произведения:
```
void calculate(int a, int b, int *sum, int *product) {
    *sum = a + b;
    *product = a * b;
}
```
Пример использования:
```
int main() {
    int x = 5, y = 10;
    printf("Before swap: x=%d, y=%d\n", x, y);
    swap(&x, &y);
    printf("After swap: x=%d, y=%d\n", x, y);

    int s, p;
    calculate(4, 5, &s, &p);
    printf("Sum: %d, Product: %d\n", s, p);

    return 0;
}
```
Задание 3. Рекурсивные функции.

Реализуйте рекурсивные версии следующих функций:

1) вычисление факториала:
```
int factorial(int n) {
    if (n <= 1) return 1;
    return n * factorial(n — 1);
}
```
2) вычисление чисел Фибоначчи:
```
int fibonacci(int n) {
    if (n <= 1) return n;
    return fibonacci(n — 1) + fibonacci(n — 2);
}
```

Пример использования:

```
int main() {
    printf("Factorial of 5: %d\n", factorial(5));
    printf("Fibonacci(6): %d\n", fibonacci(6));

    return 0;
}
```

Задание 4. Функции работы с массивами.

Создайте функции для обработки массивов:

1) поиск максимального элемента:

```
int find_max(int arr[], int size) {
    int max = arr[0];
    for (int i = 1; i < size; i++) {
        if (arr[i] > max) {
            max = arr[i];
        }
    }
    return max;
}
```

2) сортировка массива:

```
void bubble_sort(int arr[], int size) {
    for (int i = 0; i < size — 1; i++) {
        for (int j = 0; j < size — i — 1; j++) {
            if (arr[j] > arr[j + 1]) {
                swap(&arr[j], &arr[j + 1]);
            }
        }
    }
}
```

Пример использования:

```
int main() {
    int numbers[] = {5, 2, 8, 1, 9};
    int size = sizeof(numbers) / sizeof(numbers[0]);
```

```
    printf("Max element: %d\n", find_max(numbers, size));

    bubble_sort(numbers, size);
    printf("Sorted array: ");
    for (int i = 0; i < size; i++) {
        printf("%d ", numbers[i]);
    }
    printf("\n");

    return 0;
}
```

Задание 5. Модульная организация.

Создайте многофайловый проект:

1) math_operations.h — заголовочный файл:

```
#ifndef MATH_OPERATIONS_H
#define MATH_OPERATIONS_H

int add(int a, int b);
int subtract(int a, int b);
int multiply(int a, int b);
double divide(int a, int b);

#endif
```

2) math_operations.c — реализация функций:

```
#include "math_operations.h"

int add(int a, int b) { return a + b; }
int subtract(int a, int b) { return a — b; }
int multiply(int a, int b) { return a * b; }
double divide(int a, int b) {
    if (b == 0) return 0;
    return (double)a / b;
}
```

3) main.c — основная программа:

```
#include <stdio.h>
#include "math_operations.h"

int main() {
    printf("Add: %d\n", add(10, 5));
    printf("Divide: %.2f\n", divide(10, 3));
    return 0;
}
```

Содержание отчета:

1. Титульный лист.
2. Цель работы.
3. Теоретическая часть с описанием принципов модульного программирования.
4. Листинги программ с комментариями.
5. Результаты выполнения программ.
6. Ответы на контрольные вопросы.
7. Выводы.

Контрольные вопросы:

1. В чем преимущества использования подпрограмм?
2. Чем отличается передача параметров по значению от передачи по указателю?
3. Что такое рекурсия и каковы ее особенности?
4. Как организовать многофайловый проект в языке Си?
5. Что такое прототип функции и для чего он нужен?

Критерии оценки:

- *отлично:* все программы работают корректно, реализована модульная организация, код хорошо документирован;
- *хорошо:* незначительные недочеты в оформлении кода;
- *удовлетворительно:* реализованы только базовые задания;
- *неудовлетворительно:* программы не работают или работают некорректно.

Дополнительное задание.

Реализуйте библиотеку функций для работы с комплексными числами:

- сложение и вычитание;
- умножение и деление;
- вычисление модуля;
- преобразование в строку.

Организуйте код в виде отдельного модуля с заголовочным файлом и файлом реализации.

ЛАБОРАТОРНОЕ ЗАНЯТИЕ № 6
РАБОТА С МАКРОСАМИ И ОБРАБОТКА ПРЕРЫВАНИЙ В МИКРОКОНТРОЛЛЕРАХ

Цель: изучить директивы препроцессора Си для создания макросов. Освоить techniques обработки аппаратных прерываний. Научиться конфигурировать источники прерываний и писать обработчики прерываний.

Теоретическая часть:

1. Макросы в Си.

Макросы — это инструкции препроцессору для замены текста перед компиляцией:

- простой макрос:

```
#define PI 3.14159
#define MAX(a, b) ((a) > (b) ? (a) : (b))
```

- многострочный макрос:

```
#define INIT_GPIO() \
    DDRB |= (1 << PB0); \
    PORTB &= ~(1 << PB0)
```

- макросы с защитой:

```
#ifndef CONFIG_H
#define CONFIG_H
```

// код заголовочного файла

#endif

2. Обработка прерываний.

Прерывание — это событие, которое временно приостанавливает выполнение основной программы для обработки специальной функции (обработчика прерывания).

Источники прерываний:

- аппаратные (таймеры, внешние сигналы, UART, ADC);
- программные (инструкция INT);
- исключения (деление на ноль, ошибка памяти).

Этапы обработки:

1. Запрос прерывания.
2. Сохранение контекста.
3. Вызов обработчика.
4. Восстановление контекста.
5. Возврат в основную программу.

Практическая часть (на примере AVR ATmega328P).

Задание 1. Работа с макросами.

Создайте заголовочный файл config.h с макросами для настройки микроконтроллера:

```
#ifndef CONFIG_H
#define CONFIG_H

// Тактовая частота
#define F_CPU 16000000UL

// Настройка светодиодов
#define LED1_PIN PB0
#define LED1_DDR DDRB
#define LED1_PORT PORTB

#define LED2_PIN PB1
#define LED2_DDR DDRB
```

```
#define LED2_PORT PORTB

// Настройка кнопки
#define BUTTON_PIN PD2
#define BUTTON_DDR DDRD
#define BUTTON_PORT PORTD
#define BUTTON_PINR PIND

// Макросы для управления пинами
#define SET_OUTPUT(ddr, pin) ((ddr) |= (1 << (pin)))
#define SET_INPUT(ddr, pin) ((ddr) &= ~(1 << (pin)))
#define SET_HIGH(port, pin) ((port) |= (1 << (pin)))
#define SET_LOW(port, pin) ((port) &= ~(1 << (pin)))
#define TOGGLE(port, pin) ((port) ^= (1 << (pin)))
#define READ(pinr, pin) ((pinr) & (1 << (pin)))

#endif
```

Задание 2. Обработка внешнего прерывания.

Настройте прерывание по изменению уровня на выводе INT0 (PD2):

```
#include <avr/io.h>
#include <avr/interrupt.h>
#include "config.h"

// Обработчик прерывания INT0
ISR(INT0_vect) {
   TOGGLE(LED1_PORT, LED1_PIN);
}

void init_interrupt(void) {
   // Настройка вывода кнопки как входа с подтяжкой
   SET_INPUT(BUTTON_DDR, BUTTON_PIN);
```

```
    SET_HIGH(BUTTON_PORT, BUTTON_PIN);

    // Настройка прерывания INT0 по фронту
    EICRA |= (1 << ISC01) | (1 << ISC00); // Прерывание по rising edge
    EIMSK |= (1 << INT0); // Разрешить прерывание INT0

    sei(); // Глобально разрешить прерывания
}

int main(void) {
    SET_OUTPUT(LED1_DDR, LED1_PIN);
    SET_OUTPUT(LED2_DDR, LED2_PIN);

    init_interrupt();

    while (1) {
        // Основной цикл — мигаем вторым светодиодом
        TOGGLE(LED2_PORT, LED2_PIN);
        _delay_ms(500);
    }
}
```

Задание 3. Обработка прерывания таймера.

Настройте прерывание таймера для создания точных временных интервалов:

```
#include <avr/io.h>
#include <avr/interrupt.h>
#include "config.h"

volatile uint8_t counter = 0;

// Обработчик прерывания таймера
ISR(TIMER0_COMPA_vect) {
```

```
    counter++;
    if (counter >= 50) { // 50 * 20ms = 1 second
        counter = 0;
        TOGGLE(LED1_PORT, LED1_PIN);
    }
}

void init_timer(void) {
    // Настройка таймера 0 в режиме CTC
    TCCR0A = (1 << WGM01); // CTC mode
    TCCR0B = (1 << CS02) | (1 << CS00); // Prescaler 1024
    OCR0A = 155; // 20ms при 16 МГц и предделителе 1024
    TIMSK0 = (1 << OCIE0A); // Разрешить прерывание по совпадению

    sei();
}

int main(void) {
    SET_OUTPUT(LED1_DDR, LED1_PIN);
    init_timer();

    while (1) {
        // Основная программа может выполнять другие задачи
    }
}
```

Задание 4. Комбинированная обработка прерываний.

Создайте программу, которая использует оба типа прерываний:

```
#include <avr/io.h>
#include <avr/interrupt.h>
#include <util/delay.h>
#include "config.h"
```

```
volatile uint8_t timer_flag = 0;
volatile uint8_t button_pressed = 0;

// Обработчик прерывания таймера
ISR(TIMER0_COMPA_vect) {
    static uint8_t counter = 0;
    if (++counter >= 50) {
        counter = 0;
        timer_flag = 1;
    }
}

// Обработчик прерывания кнопки
ISR(INT0_vect) {
    // Простая защита от дребезга
    _delay_ms(50);
    if (!READ(BUTTON_PINR, BUTTON_PIN)) {
        button_pressed = 1;
    }
}

void init_system(void) {
    // Настройка прерываний
    EICRA = (1 << ISC01) | (1 << ISC00); // INT0 no rising edge
    EIMSK = (1 << INT0);

    TCCR0A = (1 << WGM01);
    TCCR0B = (1 << CS02) | (1 << CS00);
    OCR0A = 155;
    TIMSK0 = (1 << OCIE0A);

    sei();
```

```
}

int main(void) {
    SET_OUTPUT(LED1_DDR, LED1_PIN);
    SET_OUTPUT(LED2_DDR, LED2_PIN);
    SET_INPUT(BUTTON_DDR, BUTTON_PIN);
    SET_HIGH(BUTTON_PORT, BUTTON_PIN);

    init_system();

    while (1) {
        if (timer_flag) {
            timer_flag = 0;
            TOGGLE(LED1_PORT, LED1_PIN);
        }

        if (button_pressed) {
            button_pressed = 0;
            TOGGLE(LED2_PORT, LED2_PIN);
        }

        // Режим пониженного энергопотребления
        __asm__ __volatile__ ("sleep");
    }
}
```

Содержание отчета:

1. Титульный лист.
2. Цель работы.
3. Теоретическая часть:

- принципы работы препроцессора;
- виды макросов и их применение;
- архитектура системы прерываний.

4. Схема подключения оборудования.

5. Листинги программ с комментариями.

6. Результаты экспериментов.

7. Ответы на контрольные вопросы.

8. Выводы.

Контрольные вопросы:

1. Чем макрос отличается от функции?

2. Что такое защита заголовочного файла и для чего она нужна?

3. Какие этапы обработки прерывания вы знаете?

4. Почему в обработчиках прерываний следует избегать сложных вычислений?

5. Как организовать защиту от дребезга контактов в обработчике прерывания?

Критерии оценки:

• *отлично:* все программы работают корректно, макросы грамотно оформлены, обработчики прерываний оптимизированы;

• *хорошо:* незначительные недочеты в коде;

• *удовлетворительно:* реализованы только базовые задания;

• *неудовлетворительно:* программы не работают.

Дополнительное задание.

Реализуйте систему управления с помощью:

1) прерывания от таймера — генерация ШИМ для управления яркостью светодиода;

2) прерывания от кнопки — изменение режима работы (увеличение/уменьшение яркости);

3) прерывания от UART — удаленное управление параметрами.

Используйте макросы для настройки всех параметров системы.

ЗАКЛЮЧЕНИЕ

Изучение курса «Микроконтроллеры и встраиваемые системы» выстраивает логическую связь от архитектуры вычислительных систем до создания сложных, автономных и интеллектуальных устройств, способных взаимодействовать с физическим миром. Раскрыта суть встраиваемых систем — «мозгов», управляющих множеством аспектов современной жизни.

Подводя итоги, можно выделить несколько фундаментальных основ, на которых строится вся эта область:

1. От аппаратного обеспечения к программному обеспечению: единство противоположностей. Разработка встраиваемых систем стирает грань между «железом» и «софтом». Программист микроконтроллеров должен мыслить категориями электрических сигналов, временных диаграмм, регистров периферии и прерываний. Успех проекта определяется не только оптимизированным алгоритмом, но и корректно настроенным тактированием, правильно рассчитанными обвязочными цепями и грамотно обработанными аппаратными событиями. Умение читать datasheet и схемы становится таким же критичным навыком, как и умение писать код.

2. Архитектурное разнообразие как инструмент выбора. Не существует «универсального» микроконтроллера. Выбор между 8-битными AVR, мощными 32-битными ARM Cortex-M, открытой архитектурой RISC-V или специализированными решениями определяется конкретной задачей — требованием к энергопотреблению, вычислительной мощности, стоимости или наличию специфической периферии. Понимание этого позволяет выбрать оптимальный инструмент, избегая как избыточности, так и недостаточности вычислительных ресурсов.

3. Дисциплина реального времени и надежности. В отличие от универсальных компьютеров встраиваемые системы часто работают в режиме жесткого реального времени, где пропуск deadline (крайнего срока) может привести к катастрофическим последствиям. Это накладывает отпечаток на все аспекты разработки — от использования прерываний и таймеров для гарантированного отклика до приме-

нения специальных методик тестирования и создания отказоустойчивых программ. На первый план выходят такие качества кода, как предсказуемость, стабильность и надежность.

4. Энергоэффективность как парадигма. Подавляющее большинство встраиваемых систем работают от автономных источников питания, поэтому каждый микроампер тока и каждое тактовое колебание на счету. Изученные techniques — от применения энергосберегающих режимов (Sleep, Power-down) и динамического изменения частоты процессора до грамотного проектирования алгоритмов — превращают инженера не только в программиста, но и в энергетика, отвечающего за месяцы и годы автономной работы устройства.

5. Экосистема и сообщество. Разработка не происходит в вакууме. Мы увидели, насколько важно умение работать с инструментами (IDE, компиляторы, отладчики), использовать готовые библиотеки и middleware, искать поддержку в сообществе. Современные фреймворки, такие как Arduino, PlatformIO и Mbed OS, значительно снижают порог входа, позволяя быстро создавать прототипы, но при этом требуют понимания происходящего «под капотом» для создания финальных коммерческих продуктов.

Тенденции развития встраиваемых систем:

- распространение IoT: микроконтроллеры становятся ключевыми элементами интернета вещей, требуя от разработчиков знаний в области беспроводных протоколов связи (Wi-Fi, BLE, LoRaWAN, Zigbee) и облачной интеграции;
- интеграция ИИ на периферии (TinyML): возможность запуска lightweight-моделей машинного обучения прямо на микроконтроллерах открывает эру truly intelligent edge devices (по-настоящему интеллектуальных периферийных устройств), способных к распознаванию образов, голоса и аномалий без подключения к облаку;
- повышение уровня абстракции: появление таких технологий, как MicroPython и Zephyr RTOS, упрощает разработку сложных систем, но одновременно требует от инженера еще более глубокого понимания основ для отладки и оптимизации.

Данный курс является основой для дальнейшей специализации в изучении микроконтроллеров.

СПИСОК ЛИТЕРАТУРЫ

1. *Таненбаум, Э., Остин, Т.* Архитектура компьютера. — 6-е изд. — Санкт-Петербург : Питер, 2013. — 874 с. *(Фундаментальный труд по архитектуре вычислительных систем, необходимый для понимания принципов работы МК.)*

2. *Гибсон, Г., Лю, Б.* Проектирование встраиваемых систем. Подход на основе микроконтроллеров PIC и dsPIC. — Москва : ДМК Пресс, 2018. — 384 с.

3. *Новиков, Ю. В.* Основы цифровой схемотехники. — Москва : Мир, 2001. — 379 с. *(Классика для понимания аппаратной логики, работы с шинами и памяти.)*

4. *Александров, К. К., Кузьмич, А. И.* Электронные устройства на микроконтроллерах. — Санкт-Петербург : Корона-Век, 2017. — 256 с.

5. *Баранов, В. Н.* Применение микроконтроллеров в электронной аппаратуре. — Москва : Радио и связь, 2019. — 198 с.

6. *Рюмик, С. М.* Современные микроконтроллеры. Архитектура, программирование, разработка устройств. — Москва : ДМК Пресс, 2020. — 272 с.

7. *Керниган, Б., Ритчи, Д.* Язык программирования Си. — 3-е изд. — Москва : Вильямс, 2021. — 304 с. *(«Библия» программиста на Си, must-have для понимания основ языка.)*

8. *Прата, С.* Язык программирования C. Лекции и упражнения. — 6-е изд. — Москва : Вильямс, 2015. — 928 с. *(Более современный и подробный учебник с большим количеством примеров.)*

9. *Эндрюс, Д.* Программирование на Си для микроконтроллеров. — Москва : Додэка-XXI, 2018. — 416 с. *(Специализированное издание, ориентированное именно на embedded-разработку.)*

10. *Евстифеев, А. В.* Микроконтроллеры AVR семейства Mega. Руководство пользователя. — Москва : Додэка-XXI, 2008. — 512 с. *(Одно из лучших отечественных руководств по AVR.)*

11. *Мортон, Дж.* Программирование микроконтроллеров AVR на языке Си. — Москва : ДМК Пресс, 2019. — 276 с.

12. *Бокселл, Д.* Arduino для изобретателей. — Москва : БХВ-Петербург, 2022. — 334 с. *(Практический подход к быстрому прототипированию.)*

13. *Яценков, В. С.* Микроконтроллеры ARM Cortex-M. Архитектура, программирование и разработка приложений. — Москва : ДМК Пресс, 2021. — 486 с. *(Лучшее современное русскоязычное руководство по ARM Cortex-M.)*

14. *Джозеф, Ю.* Полный справочник по микроконтроллерам ARM Cortex-M. — Москва : Вильямс, 2017. — 688 с. *(Исчерпывающая информация от инженера ARM.)*

15. *Трейси, К., Шмидт, Г.* STM32. Полное руководство. — Москва : Додэка-XXI, 2020. — 421 с. *(Практическое руководство по популярному семейству на ядре ARM.)*

16. *Гелль, П., Беккер, Х.* Датчики в электронных схемах: от простого к сложному. — Москва : ДМК Пресс, 2018. — 295 с.

17. *Петин, В. А.* Проекты с использованием контроллера Arduino. — 4-е изд. — Санкт-Петербург : БХВ-Петербург, 2021. — 448 с. *(Сборник практических проектов для начального и среднего уровня.)*

18. *Хоровиц, П., Хилл, У.* Искусство схемотехники. — 4-е изд. — Москва : ДМК Пресс, 2022. — 704 с. *(Фундаментальный труд по аналоговой и цифровой схемотехнике, незаменим для разработки аппаратной части.)*

19. *Таненбаум, Э., Вудхалл, А.* Операционные системы. Разработка и реализация. — 3-е изд. — Санкт-Петербург : Питер, 2019. — 704 с.

20. *Кузьминов, С. В.* Встраиваемые системы реального времени. — Москва : Горячая линия — Телеком, 2016. — 312 с.

21. *Столлингс, В.* Беспроводные линии связи и сети. — 2-е изд. — Москва : Вильямс, 2018. — 640 с. *(Для понимания основ IoT и беспроводной передачи данных.)*

22. ISO/IEC 9899:2018 «Информационные технологии. Языки программирования. Си» // Международная организация по стандарти-

зации. — URL: https://www.iso.org/standard/74528.html. *(Международный стандарт языка программирования Си.)*

23. Хабр (разделы «Embedded», «Программирование микроконтроллеров», «Аппаратная разработка») : сайт. — URL: https://habr.com/ru/feed/. *(Крупнейший русскоязычный ресурс со статьями и обсуждениями.)*

24. GitHub : сайт. — URL: https://github.com/?ysclid=mkq0ht79x6257654766. *(Открытые проекты, библиотеки и примеры кода для любых платформ.)*

25. Stack Overflow : сайт. — URL: https://stackoverflow.com/questions. *(Крупнейшее международное сообщество разработчиков для решения конкретных проблем.)*

26. Официальные форумы поддержки: Arduino Forum, ST Community, Espressif Forum. *(Прямое общение с инженерами и энтузиастами.)*

27. Datasheets и Reference Manuals производителей микроконтроллеров: Microchip (Atmel), STMicroelectronics, Texas Instruments, Espressif Systems, NXP. *(Первоисточник, обязательный к изучению для профессиональной разработки.)*

28. Technical Reference Manual (TRM) для ядер ARM Cortex-M. *(Детальное описание архитектуры ядра от создателей.)*